

Arthur Salvador de Almeida
Guilherme Castelo Branco de Brito
Renato Naves Fleury

**Sistema de Geração de Recomendações de
Treinos e Dicas Alimentares com uso de
Inteligência Artificial**

São Paulo, SP

2025

Arthur Salvador de Almeida
Guilherme Castelo Branco de Brito
Renato Naves Fleury

Sistema de Geração de Recomendações de Treinos e Dicas Alimentares com uso de Inteligência Artificial

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Compu-
tação e Sistemas Digitais da Escola Politéc-
nica da Universidade de São Paulo para ob-
tenção do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Jorge Luís Risco Becerra

São Paulo, SP

2025

Resumo

Este trabalho apresenta o desenvolvimento de um sistema colaborativo voltado ao segmento *fitness*, fundamentado na interação entre usuários finais e profissionais especializados, e no uso de mecanismos de recomendações personalizadas baseados em Inteligência Artificial (IA), por meio de *Large Language Models* (LLMs) com técnicas de *Retrieval Augmented Generation* (RAG). Para isso, adotou-se uma abordagem estruturada de engenharia de *software*, contemplando o levantamento de requisitos funcionais e não funcionais, a modelagem da arquitetura do sistema com o uso de diferentes padrões arquiteturais em camadas, microsserviços e arquitetura limpa, a prototipação das telas do aplicativo e o desenvolvimento e validação das funcionalidades principais. Como resultado, foi possível construir uma versão de testes do projeto, incluindo um aplicativo funcional para o cliente, com gerenciamento de treinos e planos alimentares, geração de recomendações personalizadas e acompanhamento de avaliações físicas e nutricionais, além do monitoramento da evolução do usuário ao longo de sua dieta ativa. Os resultados demonstram a viabilidade técnica da solução proposta, evidenciando o correto funcionamento dos fluxos de interação e das integrações entre módulos. Conclui-se que a arquitetura adotada favorece a escalabilidade, a manutenibilidade, a expansão futura do sistema e a incorporação de IA de forma segura e modular.

Palavras-chave: Saúde, Nutrição, Atividade física, IA Generativa, LLM, RAG, Microsserviços, Arquitetura em Camadas, Arquitetura Limpa.

Abstract

This work presents the development of a collaborative system aimed at the fitness segment, grounded in the interaction between end users and specialized professionals, and supported by personalized recommendation mechanisms based on Artificial Intelligence (AI). These recommendations are generated using Large Language Models (LLMs) combined with Retrieval Augmented Generation (RAG) techniques. A structured software engineering approach was adopted, encompassing the elicitation of functional and non-functional requirements, the modeling of the system architecture using different architectural patterns layered architecture, microservices, and clean architecture , the prototyping of the application screens, and the implementation and validation of the systems core functionalities. As a result, a test version of the project was developed, including a functional client application that enables workout and meal plan management, personalized recommendation generation, and monitoring of physical and nutritional assessments, as well as the users progress throughout their active diet. The results demonstrate the technical feasibility of the proposed solution, confirming the correct operation of interaction flows and module integrations. It is concluded that the adopted architecture promotes scalability, maintainability, future expansion, and the secure and modular incorporation of AI into the system.

Keywords: Health, Nutrition, Physical activity, Generative AI, LLM, RAG, Microservices, Layered Architecture, Clean Architecture.

Lista de ilustrações

Figura 1 – Arquitetura Básica de Microserviços	17
Figura 2 – Arquitetura Limpa	18
Figura 3 – Quadro de modelo de negócio proposto	25
Figura 4 – Informações coletadas de entrevistas com profissionais da saúde	30
Figura 5 – Diagrama de casos de uso para o cliente	36
Figura 6 – Diagrama de casos de uso para o nutricionista	37
Figura 7 – Diagrama de casos de uso para o <i>personal trainer</i>	37
Figura 8 – Arquitetura geral da solução	52
Figura 9 – Camada de Apresentação do sistema	53
Figura 10 – Camada de Negócio do sistema	54
Figura 11 – Camada de Infraestrutura do sistema	55
Figura 12 – Modelagem do banco de dados de usuários	61
Figura 13 – Modelagem do banco de dados de planos alimentares	66
Figura 14 – Modelagem do banco de dados do módulo de treinos	72
Figura 15 – Modelagem do banco de dados de avaliações	74
Figura 16 – Modelagem do banco de dados de IA	76
Figura 17 – Geração de planos alimentares personalizados	80
Figura 18 – Geração de treinos personalizados	80
Figura 19 – Monitoramento das interações com LLM pelo LangSmith	81
Figura 20 – <i>Pipeline</i> de <i>deploy</i> do <i>backend</i>	82
Figura 21 – Arquitetura do <i>deploy</i> na nuvem	83
Figura 22 – <i>Pipeline</i> de <i>deploy</i> do <i>frontend</i>	84
Figura 23 – Casos de uso implementados do cliente	85
Figura 24 – Casos de uso implementados do nutricionista	86
Figura 25 – Casos de uso implementados do <i>personal trainer</i>	86
Figura 26 – Telas da seção <i>home</i> e menu	87
Figura 27 – Registros no banco de dados com as informações da <i>home</i> e menu	88
Figura 28 – Telas da seção de treinos - Parte 1	90
Figura 29 – Telas da seção de treinos - Parte 2	91
Figura 30 – Registros no banco de dados com as informações dos treinos	92
Figura 31 – Telas da seção de planos alimentares - Parte 1	94
Figura 32 – Telas da seção de planos alimentares - Parte 2	95
Figura 33 – Registros no banco de dados com as informações dos planos alimentares	96
Figura 34 – Telas da seção de avaliações	97
Figura 35 – Registros no banco de dados das avaliações físicas	97
Figura 36 – Telas Iniciais	106

Figura 37 – Tela de <i>Home</i>	107
Figura 38 – Telas de treino	107
Figura 39 – Telas de nutrição	108
Figura 40 – Tela de avaliação	108

Lista de tabelas

Tabela 1 – Princípios de <i>design</i> SOLID	21
Tabela 2 – Requisitos identificados para a aplicação	35
Tabela 3 – <i>Endpoints</i> de nutricionistas do microserviço de Usuários	62
Tabela 4 – <i>Endpoints</i> de clientes do microserviço de Usuários	63
Tabela 5 – <i>Endpoints</i> de <i>personal trainers</i> do microserviço de Usuários	64
Tabela 6 – <i>Endpoints</i> de usuários genéricos do microserviço de Usuários	64
Tabela 7 – <i>Endpoints</i> de autenticação do microserviço de Usuários	64
Tabela 8 – <i>Endpoints</i> de ingredientes do microserviço de Planos alimentares	67
Tabela 9 – <i>Endpoints</i> de planos alimentares do microserviço de Planos alimentares	68
Tabela 10 – <i>Endpoints</i> de dias de planos alimentares do microserviço de Planos alimentares	69
Tabela 11 – <i>Endpoints</i> de refeições do microserviço de Planos alimentares	70
Tabela 12 – <i>Endpoints</i> de calendário de planos alimentares do microserviço de Planos alimentares	70
Tabela 13 – <i>Endpoints</i> de progresso de planos alimentares do microserviço de Planos alimentares	71
Tabela 14 – <i>Endpoints</i> de treinos do microserviço de Treinos	72
Tabela 15 – <i>Endpoints</i> de equipamentos do microserviço de Treinos	73
Tabela 16 – <i>Endpoints</i> de exercícios do microserviço de Treinos	73
Tabela 17 – <i>Endpoints</i> de avaliações físicas do microserviço de Avaliações	75
Tabela 18 – <i>Endpoints</i> de avaliações nutricionais do microserviço de Avaliações	75
Tabela 19 – <i>Endpoints</i> dos planos alimentares personalizados do microserviço de IA	77
Tabela 20 – <i>Endpoints</i> dos treinos personalizados do microserviço de IA	77

Lista de abreviaturas e siglas

IA	Inteligência Artificial
LGPD	Lei Geral de Proteção de Dados
LLM	<i>Large Language Models</i>
NLP	<i>Natural Language Processing</i>
PNS	Pesquisa Nacional de Saúde
API	<i>Application Programming Interface</i>
SRP	<i>Single Responsibility Principle</i>
OCP	<i>Open-Closed Principle</i>
LSP	<i>Liskov Substitution Principle</i>
ISP	<i>Interface Segregation Principle</i>
DIP	<i>Dependency Inversion Principle</i>
RAG	<i>Retrieval Augmented Generation</i>
URL	<i>Uniform Resource Locator</i>
JWT	<i>Json Web Token</i>
SGBD	Sistema Gerenciador de Banco de Dados
ORM	<i>Object Relational Mapper</i>

Sumário

1	INTRODUÇÃO	10
1.1	Objetivos	11
1.2	Justificativa	11
1.3	Organização do Trabalho	12
2	ASPECTOS CONCEITUAIS	13
2.1	Prática da Atividade Física	13
2.2	Nutrição	14
2.3	Inteligência Artificial Generativa e LLMs	14
2.4	Princípios para Desenvolvimento de <i>Software</i>	16
2.4.1	Arquitetura em Camadas	16
2.4.2	Arquitetura de Microserviços	17
2.4.3	Arquitetura Limpa	18
2.5	Aspectos de Segurança	21
2.5.1	LGPD	21
2.5.2	<i>Json Web Token</i>	22
2.5.3	Bcrypt	23
3	CONTEXTO INICIAL E PROCESSO DE DESENVOLVIMENTO	24
3.1	Contexto Inicial	24
3.1.1	Modelo de negócios	24
3.2	Processo de desenvolvimento	28
3.2.1	Estudo bibliográfico	28
3.2.2	Entrevistas com profissionais da saúde	29
3.2.3	Especificação	30
3.2.4	<i>Design</i> e prototipação	30
3.2.5	Implementação	30
3.2.6	Testes e validação	31
4	ESPECIFICAÇÃO E ARQUITETURA	32
4.1	Requisitos Funcionais	32
4.2	Atributos de Qualidade	33
4.3	Casos de Uso	35
4.3.1	Casos de Uso Básicos	36
4.3.2	Cliente	41
4.3.3	Nutricionista	46

4.3.4	<i>Personal Trainer</i>	48
4.4	Arquitetura	49
4.4.1	Camada de Apresentação	52
4.4.2	Camada de Negócio	53
4.4.3	Camada de Infraestrutura	54
5	DESENVOLVIMENTO DO TRABALHO	56
5.1	Prototipação das Telas	56
5.2	Tecnologias Utilizadas	57
5.2.1	<i>Frameworks</i> de desenvolvimento <i>mobile</i> multiplataforma	57
5.2.2	<i>Frameworks</i> utilizados no <i>Backend</i>	58
5.2.3	Linguagem de programação	59
5.2.4	Bancos de dados	59
5.3	Projeto e Implementação	60
5.3.1	Microserviço de Usuários	60
5.3.2	Microserviço de Planos alimentares	65
5.3.3	Microserviço de Treinos	71
5.3.4	Microserviço de Avaliações	73
5.3.5	Microserviço de IA	76
5.4	Deploy do projeto	82
5.5	Resultados do Desenvolvimento	85
5.5.1	Seção <i>home</i>	87
5.5.2	Seção de treinos	89
5.5.3	Seção de dietas	92
5.5.4	Seção de avaliação	96
6	CONSIDERAÇÕES FINAIS	99
6.1	Conclusões do Projeto de Formatura	99
6.2	Contribuições	100
6.3	Perspectivas de Continuidade	100
	REFERÊNCIAS	102
	APÊNDICES	105
	APÊNDICE A – PROTOTIPAÇÃO DAS TELAS	106

1 Introdução

O cenário atual de qualidade de vida da população brasileira mostra um panorama permeado de questões estruturais, que incluem o estilo de vida sedentário, a predominância do sobrepeso e a alimentação baseada em produtos ultraprocessados e pouco saudáveis.

Na Pesquisa Nacional de Saúde realizada em 2019, constatou-se que 40,3% da população adulta foi enquadrada como insuficientemente ativa, que não praticam atividade física ou praticam por menos de 150 minutos semanais, considerando a prática por motivos de lazer, trabalho ou deslocamento ao trabalho. Estimou-se, ainda, que 23,9% da população dessa faixa etária recebeu o diagnóstico de hipertensão arterial e que 7,7% foram diagnosticadas com diabetes ([Instituto Brasileiro de Geografia e Estatística \(IBGE\), 2020a](#)).

Além disso, aponta-se a influência da desigualdade social e das condições econômicas desfavoráveis de grande parte da população na incidência de obesidade e nutrição inadequada. No artigo "Desigualdades socioeconômicas associadas ao excesso de peso e sedentarismo em adolescentes brasileiros" ([FERREIRA; ANDRADE, 2021](#)), discute-se a predominância de alimentos não saudáveis (salgados fritos, doces e refrigerantes) na dieta da população com renda familiar menor, enquanto pessoas com situação econômica melhor tendem a adotar uma alimentação mais equilibrada (frutas, verduras, laticínios). Tal comportamento tem relação com o baixo custo de produtos ultraprocessados, diante dos preços mais elevados dos alimentos saudáveis, mas também está atrelado ao acesso limitado da faixa mais vulnerável da população à conscientização alimentar e a dietas personalizadas.

Outro fator a ser considerado é a maior dificuldade de obtenção de ajuda especializada, quando se tenta adotar hábitos saudáveis. A prática de musculação, por exemplo, frequentemente depende da presença de um *personal trainer*, que auxilia o aluno a montar treinos adequados aos seus objetivos, de forma a prezar pela segurança e bem-estar do praticante, e ao mesmo tempo se adequar às suas metas. No entanto, uma parcela grande das pessoas não tem condições de contratar esse tipo de profissional, o que evidencia a necessidade de uma alternativa prática e econômica para orientar as atividades físicas.

Nesse cenário, faz-se necessária a busca de soluções para auxiliar na mudança de estilo de vida da população no geral.

Diante desse panorama, surge a motivação para o desenvolvimento deste projeto: criar uma plataforma acessível e integrada capaz de orientar usuários finais de forma personalizada, incorporando treinos, nutrição e acompanhamento físico em um único ambiente. A proposta visa não apenas suprir uma lacuna tecnológica existente, mas também

democratizar o acesso a recomendações qualificadas, oferecendo suporte contínuo e estruturado mesmo para aqueles que não possuem condições de contratar acompanhamento profissional.

1.1 Objetivos

Pensando na problemática descrita anteriormente, este trabalho tem como objetivo desenvolver um protótipo de um produto voltada para o segmento *fitness*, na qual seja possível garantir um acompanhamento personalizado para cada usuário final, usando como base a colaboração de usuários finais com profissionais da área da saúde e a geração de recomendações com Inteligência Artificial. Essa plataforma desenvolvida deve ser de fácil acesso à população, de modo a se tornar uma alternativa viável e democrática. Em contraposição às ferramentas existentes, pretende-se centralizar em uma única solução o processo de geração de recomendações de planos de treinos e de dicas alimentares, de forma a facilitar o acompanhamento do progresso individual de cada usuário e a criação de um plano mais completo e personalizado.

1.2 Justificativa

A adoção de hábitos saudáveis é um grande desafio à população brasileira conforme visto anteriormente, especialmente no que tange à manutenção de uma rotina de exercícios físicos e alimentação equilibrada. Mesmo entre indivíduos motivados, a ausência de acompanhamento especializado, a dificuldade em definir metas adequadas e a falta de *feedback* contínuo comprometem a adesão e a consistência das práticas de bem-estar. Nesse contexto, plataformas digitais surgem como alternativas promissoras, capazes de ampliar o acesso a orientações de qualidade de forma mais acessível e democrática.

Estudos recentes reforçam o potencial das tecnologias móveis no incentivo à prática de atividade física. Revisões sistemáticas indicam que aplicativos de *mHealth*, isto é, soluções de saúde móvel que utilizam *smartphones* para monitoramento, educação e apoio ao bem-estar, podem reduzir comportamentos sedentários e aumentar os níveis de atividade física, demonstrando o impacto positivo desse tipo de solução na promoção da saúde (ZHANG et al., 2022). No entanto, análises críticas do ecossistema atual mostram que muitos aplicativos ainda apresentam baixa qualidade metodológica, funcionalidades limitadas e pouca integração entre diferentes dimensões do bem-estar (NEGREIROS et al., 2022). Além disso, pesquisas nacionais apontam que, apesar dos avanços no monitoramento digital de atividades físicas, a maioria das soluções disponíveis não oferece suporte consistente para mudanças comportamentais duradouras (PRADO et al., 2022). Em conjunto, esses estudos evidenciam: a falta de plataformas integradas capazes de combinar

recomendações inteligentes, adaptação contínua ao usuário e acompanhamento multiprofissional em um único ambiente.

Diante disso, justifica-se o desenvolvimento de uma plataforma completa que una treinos, nutrição, acompanhamento físico e geração de recomendações personalizadas com IA, suprimindo limitações presentes nas soluções existentes. Diferentemente dos aplicativos convencionais, que tendem a focar exclusivamente em um aspecto da saúde, a proposta aqui apresentada busca oferecer um ecossistema integrado, apoiando tanto usuários finais quanto profissionais da área. A incorporação de modelos de IA generativa e em conjunto com a técnica de RAG permite adaptar recomendações ao perfil individual do usuário, ampliando a qualidade do suporte oferecido e tornando a solução mais responsiva, inteligente e alinhada às necessidades reais de cada pessoa.

Portanto, este projeto mostra-se uma abordagem integrada e acessível, utilizando IA para facilitar e aprimorar o processo de adoção de hábitos saudáveis.

1.3 Organização do Trabalho

O restante deste trabalho será organizado da seguinte forma: o [Capítulo 2](#) apresenta os principais conceitos teóricos utilizados para o desenvolvimento do projeto, com explicações acerca dos temas.

O [Capítulo 3](#) aborda o planejamento para a realização do trabalho, com as respectivas fases e metodologias usadas para auxiliar no desenvolvimento do projeto.

No [Capítulo 4](#), são levantados e especificados os requisitos do sistema, tanto os funcionais quanto os não funcionais.

O [Capítulo 5](#) discute o desenvolvimento e a implementação do trabalho, abordando a arquitetura concebida e as tecnologias utilizadas. Além disso, são mostrados os resultados obtidos com o projeto no final desta seção.

Por fim, o [Capítulo 6](#) traz as considerações finais do trabalho acerca dos resultados atingidos, das contribuições realizadas e das perspectivas de continuidade do projeto.

2 Aspectos Conceituais

2.1 Prática da Atividade Física

A prática regular de atividade física possui um papel essencial na promoção da saúde e qualidade de vida, alguns exemplos são a redução do risco de morte prematura e a menor incidência de doenças crônicas não transmissíveis, como hipertensão, doenças cardiovasculares, obesidade, diabetes tipo 2, alguns tipos de câncer, condições autoimunes e doenças neurodegenerativas (REINER et al., 2013). Além disso, a atividade física contribui para o fortalecimento dos ossos e articulações, prevenindo condições como a osteoporose e melhorando a mobilidade e o equilíbrio, fatores fundamentais para a longevidade e a independência ao longo dos anos.

Diante desse cenário, diversas associações nacionais e internacionais estabeleceram diretrizes para incentivar a prática de exercícios físicos, o próprio Ministério da Saúde que recomenda pelo menos 150 minutos semanais de atividade física moderada ou pelo menos 75 minutos de exercício vigoroso (Ministério da Saúde, 2021). No entanto, apesar das comprovações científicas e das recomendações, a adesão da população às diretrizes ainda é baixa, de acordo com os dados da Pesquisa Nacional de Saúde (PNS), apenas três em cada dez adultos cumprem as exigências mínimas de atividade física (Instituto Brasileiro de Geografia e Estatística (IBGE), 2020b).

Com base nas evidências científicas acumuladas relacionadas à saúde e ao condicionamento físico, o ACSM recomenda que os programas de exercícios sigam o princípio FITT-VP (frequência, intensidade, tempo, tipo, volume e progressão) para proporcionar benefícios à saúde, evitando lesões e eventos adversos à saúde. De acordo com as diretrizes do ACSM, cada sessão de exercício ou treino deve incluir os seguintes componentes: precauções de segurança, aquecimento, condicionamento (incluindo fortalecimento) e desaquecimento. Os programas de exercícios devem progredir em um ritmo seguro e adequado ao nível de condicionamento físico e aos objetivos do indivíduo. Além disso, os princípios de prescrição de exercícios do ACSM recomendam que os programas de exercícios ideais incluam três componentes principais: exercício aeróbico, exercício de força e resistência, e flexibilidade. Seguir programas de exercícios com esses componentes melhora o condicionamento cardiovascular, a aptidão neuromuscular e a saúde geral. (PESCATELLO; MEDICINE, 2014)

2.2 Nutrição

A nutrição desempenha um papel essencial na manutenção da saúde, na prevenção de doenças e na promoção do bem-estar geral. As diretrizes nutricionais são estabelecidas por instituições como a Organização Mundial da Saúde (OMS), a Organização das Nações Unidas para a Alimentação e a Agricultura (FAO), e órgãos nacionais como o Departamento de Agricultura dos Estados Unidos (USDA) e a Autoridade Europeia para a Segurança dos Alimentos (EFSA). Essas organizações baseiam suas recomendações em evidências científicas atualizadas para orientar o consumo adequado de macronutrientes (carboidratos, proteínas e gorduras) e micronutrientes (vitaminas e minerais), além da ingestão calórica ideal para diferentes faixas etárias e condições fisiológicas.

As principais métricas utilizadas para avaliar a nutrição incluem o Índice de Massa Corporal (IMC), que relaciona peso e altura para classificar o estado nutricional de um indivíduo, e a Taxa Metabólica Basal (TMB), que estima a quantidade de energia gasta em repouso. Além disso, a distribuição de macronutrientes na dieta é frequentemente monitorada por meio da ingestão de calorias provenientes de carboidratos (45-65%), proteínas (10-35%) e gorduras (20-35%), conforme estabelecido pelo Instituto de Medicina dos Estados Unidos (IOM). Outros indicadores relevantes incluem os níveis de colesterol, glicemia e marcadores inflamatórios, que ajudam a avaliar o impacto da alimentação na saúde metabólica e cardiovascular.

As diretrizes nutricionais variam entre as principais instituições, mas há um consenso sobre a importância do consumo de alimentos naturais e minimamente processados, ricos em fibras, vitaminas e minerais. A OMS, por exemplo, recomenda uma ingestão diária de pelo menos 400g de frutas e vegetais, enquanto o USDA propõe o conceito de "MyPlate", que sugere a distribuição equilibrada dos grupos alimentares em cada refeição. Além disso, a FAO enfatiza a sustentabilidade alimentar, promovendo dietas que respeitam a biodiversidade e a produção sustentável de alimentos.

2.3 Inteligência Artificial Generativa e LLMs

A Inteligência Artificial generativa consiste em um modelo de IA focado na criação de materiais novos que sejam semelhantes às informações geradas pelos seres humanos por meio de aprendizado não supervisionado. Além da geração de conteúdo, tal modelo também é capaz de reconhecer e reproduzir estruturas básicas, como textos, imagens, áudios, entre outros.

Dentro deste contexto, existe um tipo específico de IA Generativa conhecido como *Large Language Models*, o qual é caracterizado por ser um modelo de NLP treinado com uma grande quantidade de textos de fontes diversas, o que garante a esse modelo a ca-

pacidade de compreender e gerar textos sobre vários contextos. Por isso, LLMs têm sido usadas cada vez mais em aplicações práticas, como em *chatbots* e ferramentas de criação de conteúdo. Além da capacidade de compreender e gerar textos em diferentes contextos, LLMs podem ser aprimoradas para aplicações específicas por meio de um processo conhecido como *Fine-Tuning*. Essa técnica consiste no ajuste fino de um modelo previamente treinado. Para isso, treina-se novamente o modelo, mas utilizando um conjunto de dados menor e mais direcionado, permitindo que ele se especialize em um determinado domínio. Dessa forma, um LLM pode ser adaptado para gerar textos técnicos, jurídicos ou médicos com maior precisão e coerência, aumentando sua aplicabilidade em cenários específicos.

O impacto dessa tecnologia é significativo em diversas áreas. No setor empresarial, LLMs ajustados podem aprimorar assistentes virtuais, personalizar interações com clientes e automatizar tarefas de processamento de linguagem natural. Na pesquisa acadêmica, esses modelos auxiliam na análise e síntese de grandes volumes de informações, facilitando a produção de conhecimento. No entanto, a adoção de LLMs também levanta desafios, como a necessidade de garantir que os modelos gerem respostas confiáveis e minimizem vieses indesejáveis.

Dessa forma, o avanço e a aplicação de LLMs dependem não apenas de sua capacidade técnica, mas também do uso criterioso dessas ferramentas. Com o refinamento contínuo da tecnologia, espera-se que esses modelos desempenhem um papel cada vez mais relevante na automação de tarefas, transformando a forma como interagimos com informações e conteúdos digitais (BALASUBRAMANIAM, S.; KADRY, Seifedine; RAJESH KUMAR, D.; ARUCHAMY, Prasanth, 2024).

Embora os LLMs sejam uma ferramenta revolucionária no sentido de melhorar a eficiência dos processos, eles apresentam algumas limitações consideráveis, como frequentes alucinações, bases de dados desatualizadas usadas para o treinamento do modelo e limites de contexto. Essas limitações motivaram uma abordagem de uso das IAs generativas, na qual se busca trazer dados adicionais específicos do contexto para aumentar a precisão das respostas, tal abordagem ficou conhecida como RAG (*Retrieval Augmented Generation*).

A técnica de geração aumentada de recuperação foi introduzida em 2020 em uma publicação do Facebook. Essa técnica também pode ser vista como um *framework* de IA o qual permite que modelos de inteligência artificial generativa acessem informações externas não incluídas nos dados de treinamento dos modelos e incorporá-las no processo de geração da resposta. As etapas dessa técnica podem ser resumidas em: o sistema recebe uma consulta (*prompt*), realiza uma busca em uma base de conhecimento (como documentos indexados semanticamente) e, em seguida, utiliza os trechos mais relevantes recuperados como contexto adicional para a geração da resposta (GAO et al., 2023).

A recuperação de informações no RAG pode ocorrer a partir de diferentes tipos

de fontes, incluindo documentos não estruturados (como arquivos PDF ou páginas web), bancos de dados estruturados ou sistemas de busca semântica baseados em *embeddings* vetoriais. Entre os principais benefícios do RAG, destacam-se: maior precisão factual; possibilidade de especialização em domínios específicos; e aumento da interpretabilidade e verificabilidade das respostas. No entanto, o método também impõe desafios, como a dependência da qualidade da recuperação e o aumento da latência durante a inferência, além da necessidade de estratégias eficazes para garantir que o modelo utilize corretamente as informações recuperadas.

2.4 Princípios para Desenvolvimento de *Software*

O desenvolvimento de *software*, quando realizado com pouco planejamento e padronização, pode gerar diversas complicações para o andamento do projeto, como a dificuldade na manutenção da base de código, a necessidade constante de refatorações para atender a novas demandas do sistema, entre outros. Todos os pontos negativos ocasionados pela não adoção de boas práticas no desenvolvimento resultam em atrasos no cronograma previsto e, com isso, em prejuízos às partes interessadas no sistema. Por isso, faz-se necessária a definição de princípios de arquitetura e de projeto, os quais devem ser considerados durante a etapa de implementação do *software*.

2.4.1 Arquitetura em Camadas

Além da arquitetura de *software*, é importante definir a arquitetura a ser usada no sistema. Existem diversos padrões que podem ser utilizados, o abordado nesta seção será a arquitetura em camadas ([RICHARDS, 2015](#)).

A arquitetura em camadas é um padrão bastante consolidado no desenvolvimento de aplicações empresariais, sendo frequentemente adotado como ponto de partida para a construção de sistemas organizados e com manutenção facilitada. Nele, propõe-se que os componentes do sistema sejam organizados em camadas horizontais, cada uma com responsabilidade claramente definida, por exemplo: apresentação, lógica de negócios, persistência e banco de dados.

Um dos conceitos centrais da arquitetura em camadas é o da separação de responsabilidades: cada camada cuida de um determinado aspecto da aplicação e não precisa conhecer os detalhes internos das camadas adjacentes. Com isso, garante-se que cada camada seja desacoplada das demais, o que resulta em maior controle da solução desenvolvida, evitando que mudanças no código em uma parte do sistema afetem diretamente outra.

Outro ponto importante refere-se ao fluxo de requisições entre camadas. No modelo clássico, as requisições movem-se da camada superior para a camada inferior de forma

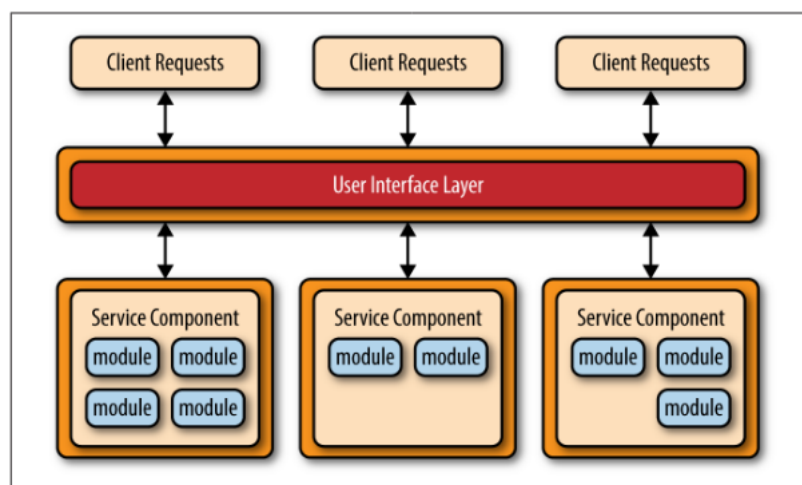
sequencial, por exemplo, da apresentação para o negócio, depois para a persistência, até atingir o banco. Essa transição controlada garante o isolamento das mudanças realizadas.

Embora a adoção da arquitetura em camadas possa, em alguns casos, direcionar o sistema para uma estrutura monolítica, é possível aplicá-la a um nível mais alto de abstração. Nesse contexto, o padrão pode ser combinado com outras abordagens arquiteturais, como a utilização de microsserviços, mantendo os benefícios da separação de responsabilidades sem comprometer a modularidade e a escalabilidade da solução.

2.4.2 Arquitetura de Microsserviços

Além da abordagem em camadas, outra forma de orientar a arquitetura do sistema é a divisão em microsserviços ([ALSHUQAYRAN; ALI; EVANS, 2016](#)). Diferentemente de arquiteturas monolíticas, em que toda a lógica de negócio e a base de código são mantidas em uma única aplicação, a abordagem com microsserviços permite segmentar a solução em módulos independentes entre si. Embora os microsserviços sejam independentes, ainda é necessário que tenham interações com os demais, para validações, registro de operações que afetam outro módulo, entre outros. Nesse caso, a comunicação entre eles pode ocorrer por meio de requisições diretas feitas em protocolos como HTTP/HTTPS ou gRPC, para comunicação síncrona, ou por meio de serviços de mensageria e fila, para comunicação assíncrona. Os principais benefícios desta abordagem são a escalabilidade e modularidade da solução final, enquanto alguns desafios enfrentados estão relacionados a complexidade de rastreabilidade e maiores custos de implantação. Na [Figura 1](#), encontra-se um exemplo de uma arquitetura simples de microsserviços, onde é possível observar a divisão dos módulos em componentes distintos do sistema.

Figura 1 – Arquitetura Básica de Microsserviços

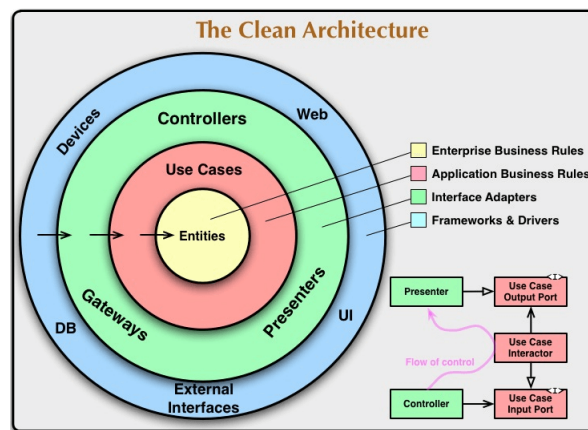


Fonte: ([RICHARDS, 2015](#), cap. 4)

2.4.3 Arquitetura Limpa

Existem diversos padrões de arquiteturas de *software* e, por mais que elas variem em seus detalhes de implementação, normalmente existem alguns fatores comuns às principais arquiteturas, como a divisão do sistema em camadas, especificamente contendo uma para as regras de negócio, outra para interfaces de sistema e usuário; a independência da implementação com relação a *frameworks*, bancos de dados; entre outros. Dessa forma, é introduzido o conceito de Arquitetura Limpa (*Clean Architecture*), com o intuito de combinar ideias das diferentes arquiteturas existentes em uma única ideia executável (MARTIN, 2017, cap. 22). Pode-se observar a arquitetura proposta na Figura 2:

Figura 2 – Arquitetura Limpa



Fonte: (MARTIN, 2017, cap. 22)

A regra principal que rege esta arquitetura e que garante a escalabilidade do sistema em conjunto com a independência de tecnologias como banco de dados e *frameworks* é a regra de dependências. Neste cenário, as dependências devem apontar apenas para dentro do sistema, ou seja, as camadas superiores devem ter dependências para as inferiores, mas nunca o contrário. Entrar-se-á em mais detalhes sobre a responsabilidade de cada uma das quatro camadas definidas na Arquitetura Limpa.

A camada mais interna é destinada ao encapsulamento das regras de negócio da aplicação, para isso, são usadas as Entidades (*Entities*), estruturas que contêm as lógicas de negócio e a estrutura de dados relativas ao negócio. Normalmente, são representadas por classes com métodos e atributos, mas podem ser apenas um conjunto de funções e estruturas de dados. Como é a camada mais interna, esta deve ser a que tem menos chance de sofrer modificações para implementações de cunho operacional.

A segunda camada é a de Casos de Uso (*Use Cases*), a qual deve conter toda a lógica da aplicação. Nesta camada, são implementados os casos de uso do sistema. Aqui, orchestra-se o fluxo de dados para as entidades e onde ocorre o acionamento das regras

de negócio para a execução do caso de uso. Não se espera que mudanças nesta camada afetem a camada das entidades do sistema, nem que a camada de casos de uso seja afetada por alterações de *frameworks* ou funcionalidades externas similares.

Entre a camada de casos de uso e a camada mais externa da aplicação, reside a camada de Adaptadores (*Interface Adapters*). Esta camada é responsável por realizar conversões dos dados entre as duas camadas explicitadas anteriormente, ou seja, pegando os dados que foram exportados da camada de casos de uso e alterando-os para um formato conveniente para a camada de tecnologias e vice-versa. O objetivo central é transformar os dados no formato mais adequado para a camada de destino.

Por fim, a camada mais externa é aquela que contém *frameworks*, *drivers*, bancos de dados, serviços externos, entre outros. Aqui devem ser implementados códigos relacionados a tecnologias específicas que não dizem respeito à lógica da aplicação.

Para a utilização da Arquitetura Limpa de maneira adequada, existem alguns conceitos de *design* de *software* que suportam a escalabilidade de arquiteturas como esta. Tais princípios são denominados pelo acrônimo SOLID (MARTIN, 2017, caps. 7–11). O objetivo deste grupo de valores é definir boas práticas para como funções e estruturas de dados devem ser agrupadas em classes e como essas diferentes classes devem se comunicar entre si, fortalecendo a construção de componentes de *software* tolerantes a mudanças, de fácil compreensão e que servem como base para a utilização em diversos pontos do sistema. Os conceitos discutidos no SOLID são: *Single Responsibility Principle*, *Open-Closed Principle*, *Liskov Substitution Principle*, *Interface Segregation Principle* e *Dependency Inversion Principle*.

O primeiro princípio do SOLID, SRP, afirma que cada módulo deve ter apenas uma única responsabilidade e um único motivo para mudar dentro do sistema, o qual está associado a um ator. Quando se tem uma classe coesa, em que todas as funções e estruturas de dados presentes nela estão ligadas a um único ator, essa classe está seguindo o princípio do SRP. Com isso, ambiguidades são evitadas no seu módulo, pois as funções definidas nele são usadas dentro de um único contexto e com um único propósito. Também é discutido o SRP no contexto de funções. Neste caso, define-se que uma função deve executar uma única ação, sendo tal definição utilizada em processos de refatoração de uma função grande em diversas funções menores, a fim de garantir maior entendimento do código.

O OCP define que um artefato de *software* deve ser aberto para extensões e fechado para modificações, ou seja, deve ser possível adicionar novos comportamentos ao sistema sem a necessidade de alterar significativamente os componentes já definidos no sistema. Para atingir tal comportamento, a principal prática a ser aplicada é fazer um *design* hierárquico dos componentes do sistema, segmentando-o em diferentes camadas, nas quais as mais internas são protegidas de alterações nas camadas superiores. Além disso, para

garantir que essa "proteção" entre camadas seja possível, existem dois conceitos importantes no *design* dos componentes: deve-se ter apenas um único fluxo de dependências no sistema, em que os componentes de camadas superiores dependem de outros presentes em camadas inferiores, mas nunca o contrário; e é preciso criar abstrações entre as camadas para garantir que a camada superior não precise saber todos os detalhes de uma camada abaixo dela, o que é feito com o uso de interfaces.

O terceiro princípio é o LSP, o qual é um conceito presente em Orientação a Objetos e estabelece que um programa com dependências em uma certa classe T, por exemplo, deve ter seu comportamento inalterado ao receber uma instância de outra classe S, a qual é uma subclasse de T. Além do contexto de classes e herança, é possível visualizar o LSP como um princípio que fundamenta a ideia de interfaces com diferentes implementações, onde as dependências ficam restritas a interfaces bem definidas, sem a necessidade de se conhecer a implementação realizada. Com isso, mesmo em cenários com diferentes lógicas que podem ser aplicadas, a depender do contexto, são reduzidas as dependências entre classes distintas e a existência de diferentes mecanismos para um propósito similar no sistema.

O quarto princípio trabalhado, ISP, aborda a utilização de interfaces para garantir que uma classe implemente apenas os métodos necessários para ela. Assim, se existir um caso em que a classe implemente uma interface mais geral e acabe sendo forçada a implementar métodos não utilizados, observa-se uma violação do ISP. Para resolver isso, basta criar interfaces mais específicas que façam mais sentido no contexto da classe final. Isso garante que comportamentos e erros ocasionados por uma dependência sem uso sejam evitados no sistema.

Por fim, o DIP diz que, para aumentar a flexibilidade do sistema, as dependências existentes no código devem ser com relação a interfaces, e não a classes concretas. A explicação para este princípio reside no fato de que a implementação é alterada constantemente, ou seja, pode ser classificada como instável. Assim, ao se criarem interfaces sujeitas a poucas alterações e com alto grau de estabilidade, garante-se uma maior escalabilidade do sistema e maior estabilidade entre as suas dependências. Embora o ideal seja não termos nenhuma dependência com classes concretas, por vezes tal cenário é inatingível, portanto, os esforços devem atuar para minimizar ao máximo a quantidade desse tipo de dependência na aplicação.

A fim de sumarizar as principais ideias dos princípios do SOLID, foi realizada a construção da [Tabela 1](#):

Tabela 1 – Princípios de *design* SOLID

Princípio	Definição
SRP	Cada classe deve ter uma única responsabilidade e um único motivo para mudar dentro do sistema, estando associada a um ator
OCP	Os módulos do sistema devem ser abertos para extensões e fechados para modificações
LSP	Um programa com dependências em uma certa classe T, por exemplo, deve ter seu comportamento inalterado ao receber uma instância de outra classe S, a qual é uma subclasse de T
ISP	Interfaces devem ser usadas para garantir que uma classe implemente apenas os métodos necessários para ela
DIP	Para aumentar a flexibilidade do sistema, as dependências existentes no código devem ser com relação a interfaces, e não a classes concretas

Fonte: Produzido pelos autores

2.5 Aspectos de Segurança

2.5.1 LGPD

A Lei Geral de Proteção de Dados ([BRASIL, 2018](#)) regula, em território nacional, o tratamento de dados pessoais em serviços digitais, a fim de proteger os direitos à privacidade e à liberdade dos cidadãos. A LGPD estabelece princípios fundamentais para o tratamento de dados, incluindo a necessidade de transparência, finalidade específica, minimização de dados e segurança. As organizações que coletam, armazenam ou processam dados pessoais, os controladores e operadores, devem garantir que o tratamento seja realizado de forma lícita, justa e informada, assegurando a proteção contra acessos não autorizados e vazamentos de informações.

Para implementar a LGPD, as empresas precisam adotar medidas técnicas e administrativas, como a elaboração de uma política de privacidade clara, a nomeação de um encarregado para atuar como canal de comunicação entre o controlador, os titulares dos dados e a Autoridade Nacional de Proteção de Dados (ANPD). Além disso, devem estabelecer mecanismos para atender aos direitos dos titulares, que incluem o acesso, a retificação, a exclusão e a portabilidade dos dados pessoais.

O descumprimento da LGPD pode acarretar penalidades severas, como advertências, multas que podem chegar a 2% do faturamento da empresa, com limite de R\$ 50 milhões por infração, e até mesmo a suspensão das atividades de tratamento de dados. Dessa forma, a adequação à legislação não apenas evita sanções, mas também fortalece a confiança do público nas organizações que lidam com informações pessoais.

Além do cumprimento legal, a LGPD impulsiona uma mudança cultural na forma como os dados são tratados, promovendo uma abordagem baseada na privacidade desde a concepção (*privacy by design*) e, por padrão (*privacy by default*). Essas diretrizes incentivam a adoção de boas práticas e reforçam a responsabilidade das empresas na gestão de informações sensíveis.

Devido ao contexto no qual o sistema a ser desenvolvido está inserido, tratando de aspectos relacionados a atividades físicas e à nutrição de usuários, a conformidade com a LGPD torna-se essencial. Isso porque, além dos dados considerados pessoais a serem tratados pelo sistema, têm-se ainda os dados sensíveis, aqueles que contêm informações referentes à saúde. Por isso, a adoção de medidas de segurança no desenvolvimento é primordial para assegurar o cumprimento da LGPD, fazendo uso, por exemplo, de criptografia, tanto para o armazenamento de informações em bancos de dados quanto para garantir uma comunicação segura na qual os dados serão transmitidos.

2.5.2 *Json Web Token*

O *JSON Web Token* é um padrão aberto amplamente utilizado para autenticação e troca segura de informações entre partes em um sistema distribuído. O JWT tem como principal característica a capacidade de transmitir dados de forma auto-contida, ou seja, todas as informações necessárias para validar a identidade e as permissões de um usuário são incluídas dentro do próprio *token* (JONES; BRADLEY; SAKIMURA, 2015).

Um *token* JWT é composto por três partes principais, separadas por pontos e codificadas em Base64: *header*, *payload* e *signature*. O *header* contém informações sobre o tipo de *token* e o algoritmo de assinatura utilizado, como HMAC-SHA256 ou RSA-SHA256. O *payload* inclui as *claims*, que são as declarações sobre uma entidade e metadados relacionados, como emissor e tempo de expiração. As *claims* podem ser padronizadas ou definidas de forma personalizada, conforme as necessidades da aplicação. Por fim, a *signature* é gerada a partir da combinação codificada do *header* e do *payload*, juntamente com uma chave secreta ou par de chaves assimétricas, dependendo do algoritmo escolhido. Essa assinatura garante a integridade e autenticidade do JWT.

O processo de validação consiste em decodificar o *token*, recalcular a assinatura com a chave correspondente e verificar se as *claims* ainda são válidas, analisando o tempo de expiração do JWT por exemplo. Para a assinatura do *token*, existem algumas opções de algoritmos que podem ser utilizados: algoritmos simétricos, como HMAC em conjunto com SHA (HS256, HS512), utilizam a mesma chave para assinar e verificar; já os algoritmos assimétricos, como RSA (RS256, RS512) e ECDSA (ES256), utilizam um par de chaves pública e privada, conferindo maior segurança em arquiteturas distribuídas. Além disso, é recomendada a adoção de boas práticas, como a definição de tempos curtos de expiração e o uso de canais seguros para o tráfego dos *tokens*.

Por se configurar como uma autenticação *stateless*, o JWT é uma ferramenta eficiente e escalável para cenários de autenticação moderna, especialmente em sistemas baseados em microsserviços. Isso ocorre porque o seu uso permite eliminar a necessidade de manter sessões no servidor e possibilita que cada um dos diferentes serviços valide *tokens* de forma independente.

Entretanto, é preciso ter atenção ao adotar o JWT em um sistema, uma vez que não há um mecanismo nativo para invalidá-los antes do término do seu tempo de vida. Uma abordagem comum é a utilização de *blacklists* de *tokens* em cache ou a emissão de JWT com períodos curtos de expiração, combinados com *refresh tokens*, permitindo que a aplicação revogue ou renove credenciais de forma segura.

2.5.3 Bcrypt

O Bcrypt é um algoritmo de derivação de chave baseado em hash, desenvolvido por Niels Provos e David Mazieres em 1999, com o objetivo de proteger senhas contra ataques de força bruta e de dicionário (PROVOS, 1999). Ele é derivado do algoritmo Blowfish e foi projetado especificamente para ser computacionalmente custoso, de modo que cada tentativa de adivinhar uma senha exija um tempo considerável de processamento.

Uma das principais características do Bcrypt é o uso de um *salt*, um valor aleatório adicionado à senha antes do cálculo do *hash*. O *salt* garante que senhas idênticas resultem em *hashes* diferentes. Isso impede o uso de tabelas pré-computadas (*rainbow tables*), uma técnica comum para tentar reverter funções de hash.

Além de aplicar o *salt*, o Bcrypt executa múltiplas iterações no processo de hashing sobre a combinação inicial de *salt* + senha, reforçando a segurança contra ataques de força bruta. Cada iteração depende do resultado da anterior, tornando inviável o paralelismo dessa operação e aumentando o custo computacional de cada tentativa de quebra. O número de iterações é controlado pelo fator de custo (*cost factor*), que determina quantas vezes o algoritmo será repetido.

O *hash* gerado pelo Bcrypt a ser armazenado no banco de dados inclui o fator de custo, o *salt* e o resultado final, concatenados em uma única *string*. Esse formato facilita a validação da senha posteriormente, uma vez que todos os elementos necessários para recalculá-lo estão presentes, não dependendo de informações externas.

3 Contexto Inicial e Processo de Desenvolvimento

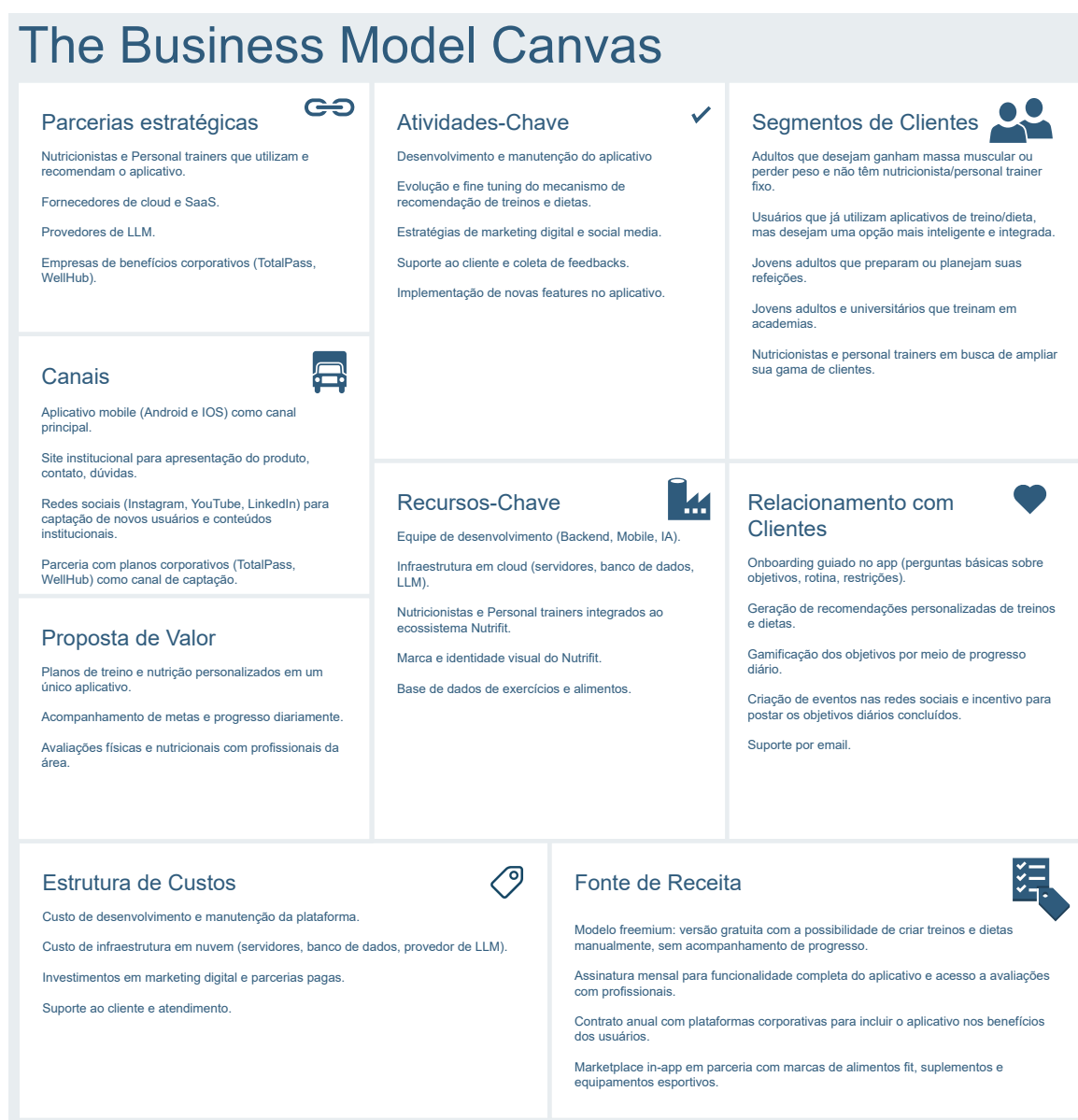
3.1 Contexto Inicial

A fim de discutir a viabilidade e o posicionamento da solução proposta no mercado brasileiro, foi idealizada uma empresa, de nome NutriFit, responsável por planejar, desenvolver e distribuir o sistema em questão. Assim, o aplicativo não é visto apenas como um software isolado, mas como o canal por meio do que a empresa NutriFit cria, entrega e captura valor a partir de serviços como a recomendação e acompanhamento de treinos e dietas, avaliações com profissionais da área e monitoramento de metas individuais.

3.1.1 Modelo de negócios

Torna-se necessário, portanto, explicitar o modelo de negócio associado a essa instituição, o que será feito à luz do modelo *Business Model Canvas* (BMC) (OSTERWALDER; PIGNEUR, 2010, cap. 1), que organiza a lógica de valor da empresa em nove blocos principais. A seguir, apresenta-se o modelo de negócio proposto para o NutriFit, detalhando cada elemento no contexto do sistema, de modo a evidenciar como a empresa se sustenta como solução empresarial.

Figura 3 – Quadro de modelo de negócio proposto



Fonte: Produzido pelos autores

A partir do quadro da Figura 3, é possível detalhar cada um dos blocos do *Business Model Canvas* no contexto da NutriFit.

• Segmentos de clientes

A NutriFit atende, de forma complementar, segmentos de clientes finais (*business-to-consumer* – B2C) e organizacionais/profissionais (*business-to-business* – B2B). No segmento B2C, o público-alvo principal é composto por adultos que desejam ganhar massa muscular ou perder peso, mas que não possuem acompanhamento contínuo de

nutricionistas ou *personal trainers*, bem como por usuários que já utilizam soluções digitais de treino e dieta e buscam uma alternativa mais integrada e inteligente. Jovens adultos e universitários que treinam em academias constituem um grupo relevante dentro desse segmento, em função da familiaridade com tecnologias móveis e com o uso de aplicativos no dia a dia.

No segmento B2B, a NutriFit volta-se a nutricionistas e *personal trainers* interessados em ampliar sua base de clientes e em utilizar o aplicativo como ferramenta de prescrição e acompanhamento, além de empresas e programas corporativos de bem-estar que desejam oferecer a plataforma como benefício de saúde aos seus colaboradores. Esses diferentes segmentos orientam tanto o desenho da proposta de valor quanto as estratégias de canais e de geração de receita da empresa.

- **Proposta de valor**

A proposta de valor central da NutriFit consiste em oferecer, em um único aplicativo, planos personalizados de treino e nutrição, ajustados aos objetivos, rotina e restrições de cada usuário. A solução procura diferenciar-se por permitir o acompanhamento diário de metas e progresso, bem como a realização de avaliações físicas e nutricionais com profissionais integrados ao ecossistema. Dessa forma, o aplicativo se posiciona como um hub de saúde e bem-estar, conectando recomendações automatizadas baseadas em dados com o acompanhamento especializado de profissionais.

- **Canais**

O principal canal de entrega da proposta de valor é o aplicativo móvel, disponível para dispositivos Android e iOS, por meio do qual o usuário interage com as funcionalidades de treino, dieta e acompanhamento. Complementarmente, um site institucional é utilizado para apresentação do produto, esclarecimento de dúvidas e contato com a empresa. Canais digitais, como redes sociais (por exemplo, Instagram, YouTube e LinkedIn), são empregados tanto para divulgação da marca quanto para captação de novos usuários e compartilhamento de conteúdos educativos. Além disso, parcerias com planos corporativos de bem-estar (como programas de academias conveniadas) funcionam como canais adicionais de aquisição de usuários.

- **Relacionamento com clientes**

O relacionamento com os clientes é estruturado de forma predominantemente digital e automatizada, com suporte humano quando necessário. O processo de onboarding no aplicativo é guiado por perguntas básicas sobre objetivos, rotina e restrições, permitindo configurar um perfil inicial personalizado. A partir disso, o sistema gera recomendações de treinos e dietas sob medida, reforçando a sensação de acompanhamento contínuo. Elementos de gamificação, como metas diárias e indicadores

de progresso, são utilizados para aumentar o engajamento. A interação com a comunidade é estimulada por meio da criação de eventos e desafios em redes sociais, enquanto o suporte a dúvidas e problemas é realizado principalmente via e-mail.

- **Recursos-chave**

Para viabilizar o modelo de negócio, a NutriFit depende de uma equipe de desenvolvimento capaz de manter e evoluir o aplicativo (incluindo componentes de *backend*, *mobile* e inteligência artificial), bem como de infraestrutura em nuvem para hospedagem de servidores, bases de dados e modelos de linguagem. Outro recurso-chave é a rede de nutricionistas e *personal trainers* integrados ao ecossistema, que agrega credibilidade às recomendações e aos serviços oferecidos. A marca e a identidade visual do produto, juntamente com uma base de dados estruturada de exercícios e alimentos, também constituem ativos importantes para a diferenciação e o funcionamento adequado da plataforma.

- **Atividades-chave**

Entre as atividades essenciais da NutriFit estão o desenvolvimento e a manutenção contínua do aplicativo, incluindo correções, melhorias de desempenho e atualização de funcionalidades. Destaca-se, ainda, a evolução do mecanismo de recomendação de treinos e dietas, que envolve ajustes (*fine tuning*) com base em dados de uso e feedback dos usuários. Atividades de marketing digital e gestão de redes sociais são fundamentais para divulgação da solução e aquisição de novos clientes. Complementarmente, o suporte ao usuário, a coleta sistemática de feedbacks e a implementação de novas *features* compõem o ciclo de melhoria constante do sistema.

- **Parcerias estratégicas**

O modelo de negócios prevê parcerias com nutricionistas e *personal trainers* que utilizem e recomendem o aplicativo como ferramenta de apoio aos atendimentos. Do ponto de vista tecnológico, a NutriFit depende de provedores de computação em nuvem, serviços *SaaS* e fornecedores de modelos de linguagem, que permitem viabilizar recursos de inteligência artificial em escala. Parcerias com empresas de benefícios corporativos relacionados à saúde e bem-estar (como programas de acesso a academias) ampliam a capilaridade da solução e facilitam a inclusão do aplicativo em pacotes oferecidos a colaboradores de organizações parceiras.

- **Estrutura de custos**

A estrutura de custos da NutriFit é composta, principalmente, pelas despesas de desenvolvimento e manutenção da plataforma, envolvendo equipe técnica, licenças e ferramentas de apoio. Há também custos recorrentes de infraestrutura em nuvem, englobando servidores, armazenamento de dados e utilização de modelos de IA. Investimentos em marketing digital, campanhas pagas e ações de comunicação com

parceiros representam outro componente relevante. Por fim, despesas com suporte ao cliente, atendimento e eventuais comissões ou remunerações para profissionais parceiros integram o conjunto de custos a ser coberto pelo modelo de receita.

- **Fontes de receita**

No que diz respeito às fontes de receita, adota-se um modelo freemium, em que uma versão gratuita do aplicativo permite a criação manual de treinos e dietas, com funcionalidade limitada. Recursos avançados, como acompanhamento detalhado de progresso, planos personalizados e acesso a avaliações com profissionais, são disponibilizados mediante assinatura mensal. Adicionalmente, prevê-se a celebração de contratos com plataformas e programas corporativos de benefícios, que remuneram a inclusão do aplicativo como serviço oferecido aos colaboradores. Por fim, um potencial canal adicional de monetização é a criação de um *marketplace* interno, por meio do qual marcas de alimentos, suplementos e equipamentos esportivos possam ofertar produtos diretamente aos usuários do NutriFit.

3.2 Processo de desenvolvimento

Esta seção apresenta as principais fases para a criação e o desenvolvimento do trabalho, fornecendo uma breve descrição das ações realizadas e dos objetivos de cada etapa.

3.2.1 Estudo bibliográfico

Primeiramente, foram feitos estudos sobre temas importantes relacionados ao projeto, os quais estão concentrados no [Capítulo 2](#). Fez-se uma revisão de aspectos importantes da área da saúde no contexto do projeto, com foco em atividades físicas e nutrição, a fim de levantar os principais pontos e componentes a serem considerados durante o desenvolvimento da solução.

Além disso, fez-se uma pesquisa sobre temas mais técnicos, como LGPD e IA Generativa, para melhorar o entendimento de como a solução poderia ser executada. Com o estudo da LGPD, pode-se levantar a necessidade de incluir a segurança dos dados pessoais e sensíveis dos usuários como prioridade no projeto e, a partir disso, pensou-se em estratégias de implementá-la adequadamente. Já com o estudo sobre IA Generativa e LLMs, foi possível entender a estrutura e casos de uso dessa tecnologia e conhecer métodos para aumentar a eficiência das respostas geradas pelo modelo para o contexto específico do projeto, o que pode ser feito com *Fine-Tuning*.

3.2.2 Entrevistas com profissionais da saúde

Em conjunto com a pesquisa bibliográfica, foram realizadas entrevistas com profissionais da área a fim de ampliar o entendimento das necessidades do projeto com relação a esse contexto. Durante essas entrevistas, pode-se levantar informações importantes a serem usadas no processo de geração de treino e de dicas alimentares do sistema. Definiram-se as informações essenciais do cliente a serem coletadas para uso no aplicativo e os principais dados a serem coletados dos processos de avaliação física e de avaliação nutricional. Tais informações são destacadas na [Figura 4](#).

Figura 4 – Informações coletadas de entrevistas com profissionais da saúde

Informações essenciais:	Avaliação Física:	Avaliação nutricional:
• Idade • Sexo • Peso • Meta de peso • Estatura • Tipo de corpo • Local de treino • Frequência semanal de treino • Sono • Objetivo (emagrecimento, hipertrofia, cfg, reabilitação, saúde, taf) • Restrições de exercícios físico • Restrições alimentares * (discutir melhor) • Há quanto tempo treina • Preferências de estilo de treino	• Histórico de saúde • Risco Coronariano • Anamnese específica • Composição Corporal • Perimetria • RCQ • Avaliação Funcional • Dobras cutâneas • Restrições de exercícios físicos • Facilidade/Dificuldade hipertrofia • Facilidade/Dificuldade ganho de gordura • Alimentação • Gasto basal	• Alergias e intolerâncias alimentares • Exames laboratoriais recentes • Quantidade de refeições por dia • Descrição das refeições (café da manhã, almoço, lanches e jantas) • Prepara as refeições ou come fora? • Quantos litros de água diários? • Suco, chá, refrigerante? • Suplementos alimentares? • Alcool? Qual a frequência?

Fonte: Produzido pelos autores

3.2.3 Especificação

Com os estudos e as entrevistas realizadas, definiram-se as funcionalidades essenciais e diferenciais da solução. Para modelar tais pontos, foram feitos os diagramas de casos de uso para os atores do sistema e, com isso, foi possível chegar à definição dos requisitos funcionais e não funcionais do projeto. Por fim, definiu-se a arquitetura do sistema, que foi trabalhado com o conceito de microsserviços e arquitetura em camadas.

3.2.4 Design e prototipação

Esta etapa consistiu na elaboração do *design* e prototipação da solução do aplicativo a ser desenvolvido. Para isso, foi elaborada a estrutura dos fluxos de tela, visando uma experiência de usuário intuitiva e eficiente. Já o processo de prototipação envolveu a definição de elementos visuais e fluxos interativos, permitindo uma melhor visualização da interface antes da fase de desenvolvimento.

3.2.5 Implementação

Uma vez definidas as telas do aplicativo e a arquitetura a ser seguida para o projeto, partiu-se para a fase de implementação da solução. Primeiramente, foram analisadas quais tecnologias poderiam ser utilizadas, incluindo opções de LLMs, *frameworks mobile* multiplataforma, *frameworks* para desenvolvimento *Backend*, entre outros. Com as escolhas das tecnologias fundamentadas, iniciou-se o processo de implementação da solução, em que foram desenvolvidas as APIs com as funcionalidades do sistema e foi realizado o desenvolvimento do aplicativo que utilizava as APIs feitas. Além disso, estudou-se diferentes estratégias de *deploy* em diferentes provedores de nuvem.

3.2.6 Testes e validação

Para garantir a qualidade da solução, foram feitos testes de integração do aplicativo. Esta etapa assegurou que todos os componentes funcionassem corretamente e atendessem aos requisitos definidos na fase de planejamento.

4 Especificação e Arquitetura

Nesta seção, definem-se os requisitos necessários para o desenvolvimento do aplicativo proposto. Eles especificam e restringem as funcionalidades e qualidades do sistema, de forma que devem ser utilizados como guia na etapa de desenvolvimento. Os requisitos definidos para o trabalho se dividem em Requisitos Funcionais e Atributos de Qualidade, sendo os primeiros relativos às funcionalidades a serem implementadas, e os últimos são as restrições de qualidade, desempenho, segurança, acessibilidade e compatibilidade.

4.1 Requisitos Funcionais

A seguir, são apresentados os requisitos funcionais definidos para o sistema:

- **Usuários devem ser capazes de fazer cadastro na plataforma, bem como excluir suas contas:** O aplicativo contará com tela de cadastro, onde o usuário criará sua conta. Será possível excluir a conta posteriormente também.
- **Usuários devem ser capazes de fazer login e logout na plataforma:** O aplicativo contará com tela de login, que cria a sessão do usuário, além de opção de logout, para encerrar a sessão.
- **O cliente deve ser capaz de inserir e atualizar suas informações pessoais:** Informações de saúde, medidas corporais, objetivos a serem alcançados com o plano poderão ser gerenciados pelo cliente.
- **Um *personal trainer* deve ser capaz de criar uma avaliação física de seu cliente:** Caso um cliente esteja atrelado a um *personal trainer*, esse profissional poderá criar uma avaliação física para ele, composta de informações complementares sobre a saúde o condicionamento físico do cliente.
- **Um nutricionista deve ser capaz de criar uma avaliação nutricional de seu cliente:** Caso um cliente esteja atrelado a um nutricionista, esse profissional poderá criar uma avaliação nutricional para ele, composta de informações complementares sobre alergias, preferências e gastos calóricos do cliente.
- **O cliente deverá ser capaz de gerar um treino personalizado:** O aplicativo utilizará inteligência artificial para gerar um treino personalizado, a partir das informações pessoais do cliente.
- **O cliente deverá ser capaz de criar um treino manualmente:** O aplicativo deverá contar com uma função manual de criação de treinos.

- **Um *personal trainer* deve ser capaz de criar um treino para seu cliente:** Caso um cliente esteja atrelado a um *personal trainer*, esse profissional terá a opção de criar um treino para ele, criando-o manualmente ou gerando um treino personalizado para o cliente.
- **Todos os treinos de um cliente poderão ser gerenciados por ele:** O cliente terá a possibilidade de gerenciar (editar, apagar, substituir exercícios) os seus treinos existentes.
- **O cliente deverá ser capaz de gerar um plano alimentar personalizado:** O aplicativo utilizará inteligência artificial para gerar um plano alimentar para o cliente, a partir de suas informações pessoais.
- **O cliente deve ser capaz de criar um plano alimentar manualmente:** O aplicativo permitirá que o cliente crie novos planos alimentares, mostrando informações nutricionais sobre os alimentos e pratos selecionados.
- **Um nutricionista deverá ser capaz de criar um plano alimentar para seu cliente:** Caso um cliente esteja atrelado a um nutricionista, esse profissional terá a opção de criar um plano alimentar para ele, criando-o manualmente ou gerando um plano alimentar personalizado para o cliente.
- **Todos os planos alimentares de um cliente poderão ser gerenciados por ele:** O cliente terá a possibilidade de gerenciar (editar, apagar, substituir alimentos) os seus planos alimentares existentes.
- **Um *personal trainer* deve ser capaz de gerenciar os treinos de seus clientes:** O *personal trainer* terá a possibilidade de gerenciar (editar, apagar, substituir exercícios) os treinos existentes de seus clientes.
- **Um nutricionista deve ser capaz de gerenciar os planos alimentares de seus clientes:** O nutricionista terá a possibilidade de gerenciar (editar, apagar, substituir alimentos) os planos alimentares existentes de seus clientes.
- **Os colaboradores devem poder gerenciar seus clientes:** *Personal trainers* e nutricionistas devem poder gerenciar (adicionar, excluir) os clientes atrelados a eles.
- **Usuário poderá ter mais de um papel no sistema:** Um colaborador (*personal trainer*, nutricionista), também poderá ser um cliente.

4.2 Atributos de Qualidade

Apresentam-se os atributos de qualidade definidos para o sistema:

- **Login persistente:** Após o usuário efetuar o login no aplicativo, ele permanecerá logado, desde que sua sessão seja válida. Isso evita que o login seja necessário a cada vez que o aplicativo é iniciado.
- **Suporte a plataformas móveis:** O aplicativo estará disponível para dispositivos Android e dispositivos iOS.
- **Criptografia nas requisições:** As requisições entre a interface mobile e o servidor remoto serão criptografadas, a fim de prezar pela confidencialidade dos dados.
- **Backend escalável:** O backend será capaz de se adaptar dinamicamente em relação aos recursos usados. A utilização de mais CPU e memória em caso de aumento da carga será automática.
- **Compliance com a LGPD:** Os dados dos usuários deverão ser utilizados em conformidade com as determinações da Lei Geral de Proteção de Dados.

Com isso, é possível chegar ao seguinte resumo acerca dos requisitos levantados para a aplicação presente na [Tabela 2](#):

Tabela 2 – Requisitos identificados para a aplicação

Requisitos funcionais	
1	Usuários devem ser capazes de fazer cadastro na plataforma, bem como excluir suas contas
2	Usuários devem ser capazes de fazer login e logout na plataforma
3	O cliente deve ser capaz de inserir e atualizar suas informações pessoais
4	Um <i>personal trainer</i> deve ser capaz de criar uma avaliação física de seu cliente
5	Um nutricionista deve ser capaz de criar uma avaliação nutricional de seu cliente
6	O cliente deverá ser capaz de gerar um treino personalizado
7	O cliente deverá ser capaz de criar um treino manualmente
8	Todos os treinos de um cliente poderão ser gerenciados por ele
9	O cliente deverá ser capaz de gerar um plano alimentar personalizado
10	O cliente deve ser capaz de criar um plano alimentar manualmente
11	Um <i>personal trainer</i> deve ser capaz de criar um treino para seu cliente
12	Um nutricionista deverá ser capaz de criar um plano alimentar para seu cliente
13	Todos os planos alimentares de um cliente poderão ser gerenciados por ele
14	Um <i>personal trainer</i> deve ser capaz de gerenciar os treinos de seus clientes
15	Um nutricionista deve ser capaz de gerenciar os planos alimentares de seus clientes
16	Os colaboradores devem poder gerenciar seus clientes
17	Usuário poderá ter mais de um papel no sistema

Atributos de qualidade	
1	Login persistente
2	Suporte a plataformas móveis
3	Criptografia nas requisições
4	Backend escalável
5	Compliance com a LGPD

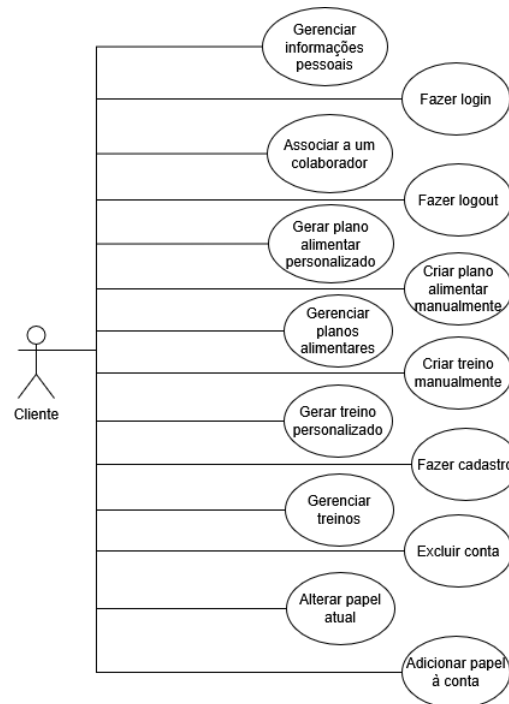
Fonte: Produzido pelos autores

4.3 Casos de Uso

Com a definição dos requisitos do sistema, pode-se aprofundar mais sobre as interações previstas entre os usuários e o *software*, o que é feito com a elaboração dos

casos de uso da aplicação. Dessa forma, foram feitos os diagramas de casos de uso para os três atores presentes no sistema, cliente, nutricionista e *personal trainer*, encontrados, respectivamente, na Figura 5, Figura 6 e Figura 7:

Figura 5 – Diagrama de casos de uso para o cliente



Fonte: Produzido pelos autores

A partir disso, foram feitas as descrições detalhadas dos casos de uso de cada ator.

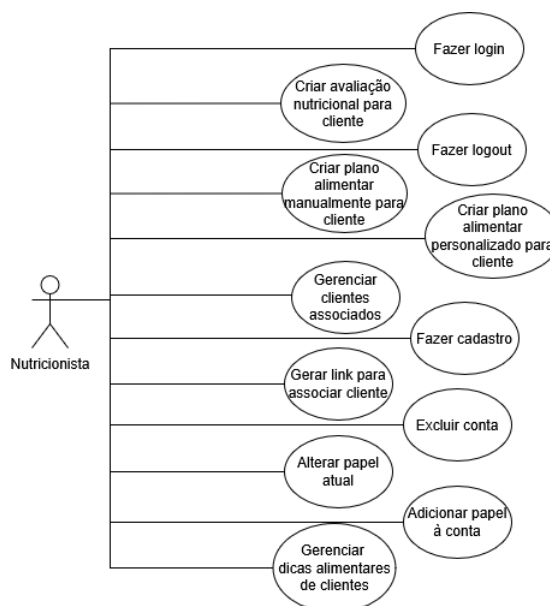
4.3.1 Casos de Uso Básicos

Com base nos diagramas mostrados anteriormente, percebe-se que existem alguns casos de uso básicos e comuns aos três atores do sistema. Dessa forma, as descrições desses casos foram concentradas nesta seção:

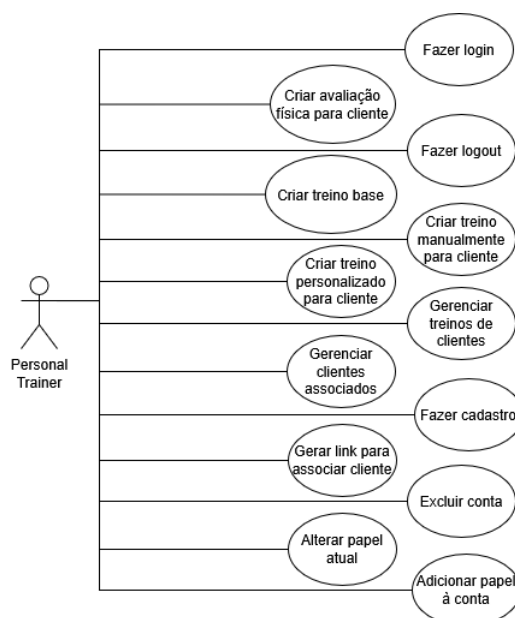
1. Fazer *login*

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para fazer o *login* na plataforma.
- **Evento iniciador:** Usuário abrir a tela de *login* do aplicativo.
- **Ator:** Todos.
- **Pré-condição:** Usuário possuir uma conta.
- **Sequência de eventos:**

Figura 6 – Diagrama de casos de uso para o nutricionista



Fonte: Produzido pelos autores

Figura 7 – Diagrama de casos de uso para o *personal trainer*

Fonte: Produzido pelos autores

- Usuário clica em "Fazer *login*" na tela de boas-vindas do aplicativo.
- Sistema redireciona o usuário para tela de *login*, a qual necessita das credenciais do usuário para autenticá-lo no sistema.
- Usuário insere suas credencias de identificação, *email* e senha, e envia os

dados.

- d) Sistema verifica as credencias.
- e) Sistema autentica o usuário e o redireciona para a tela principal do aplicativo.
- **Pós-condição:** Usuário é autenticado no sistema e redirecionado para as demais telas do aplicativo.
- **Extensões:**
 - a) Credencias fornecidas pelo usuário não cumprem os requisitos mínimos do sistema, como *email* inválido ou senha com segurança menor que a permitida (passo c): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.
 - b) Usuário não possui conta (passo d): Sistema apresenta mensagem de que não tem nenhuma conta associada às credencias fornecidas.
 - c) Senha incorreta (passo d): Sistema informa ao usuário que a senha está incorreta.

2. Fazer *logout*

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para fazer o *logout* na plataforma.
- **Evento iniciador:** Usuário clicar na opção de fazer *logout* da sua conta.
- **Ator:** Todos.
- **Pré-condição:** Usuário estar *logado* na plataforma.
- **Sequência de eventos:**
 - a) Usuário clica em "Fazer *logout*" na página da sua conta do aplicativo.
 - b) Sistema realiza o processo de remover a sessão do usuário.
 - c) Sistema redireciona para a tela de boas-vindas do aplicativo.
- **Pós-condição:** Usuário tem sua sessão removida do sistema e é redirecionado para a tela de boas-vindas.

3. Fazer cadastro

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para criar a sua conta na plataforma.
- **Evento iniciador:** Usuário clicar em fazer cadastro no aplicativo.
- **Ator:** Todos.
- **Pré-condição:** Usuário não possuir uma conta.

- **Sequência de eventos:**

- a) Usuário clica em "Fazer cadastro" na tela de boas-vindas aplicativo.
- b) Sistema redireciona o usuário para uma tela onde ele escolhe qual perfil ele irá utilizar no aplicativo, cliente, nutricionista ou *personal trainer*.
- c) Usuário escolhe o papel desejado.
- d) Sistema redireciona o usuário para a tela de cadastro do papel selecionado.
- e) Usuário insere as informações requisitadas para o cadastro e envia os dados.
- f) Sistema verifica as informações fornecidas.
- g) Sistema autentica o usuário e o redireciona para a tela principal do aplicativo.

- **Pós-condição:** Usuário é autenticado no sistema e redirecionado para as demais telas do aplicativo.

- **Extensões:**

- a) Informações fornecidas pelo usuário não cumprem os requisitos mínimos do sistema, como *email* inválido ou senha com segurança menor que a permitida (passo e): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.
- b) Usuário já possui conta (passo f): Sistema apresenta mensagem de que já existe uma conta associada às informações fornecidas.

4. Excluir conta

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para fazer excluir a sua conta da plataforma.

- **Evento iniciador:** Usuário clicar na opção de excluir conta.

- **Ator:** Todos.

- **Pré-condição:** Usuário estar *logado* na plataforma.

- **Sequência de eventos:**

- a) Usuário clica em "Excluir conta" na página da sua conta do aplicativo.
- b) Sistema pede uma ação de confirmação do usuário e explica que todos os dados associados à conta dele serão removidos da plataforma.
- c) Usuário confirma exclusão da conta.
- d) Sistema remove todas as informações daquele usuário do sistema.
- e) Sistema redireciona o usuário para a tela de boas-vindas do aplicativo.

- **Pós-condição:** Usuário tem todos os seus dados removidos do sistema e é redirecionado para a tela de boas-vindas.

- **Extensões:**

- a) Usuário cancela exclusão da conta (passo c): Nenhuma ação é realizada pelo sistema e o usuário continua com as suas informações disponíveis no aplicativo.

5. Alterar papel atual

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para alterar o papel ativo da sua conta no aplicativo.
- **Evento iniciador:** Usuário clicar na opção de alterar papel ativo.
- **Ator:** Todos.
- **Pré-condição:** Usuário estar *logado* na plataforma e possuir mais de um papel associado a sua conta.
- **Sequência de eventos:**
 - a) Usuário clica em "Alterar papel ativo" na página da sua conta do aplicativo.
 - b) Sistema retorna a lista de papéis associados à conta do usuário.
 - c) Usuário seleciona um dos papéis disponíveis, diferente do ativo naquele momento.
 - d) Sistema altera as informações da sessão do usuário para comportar a troca de papel.
 - e) Sistema redireciona o usuário para a tela inicial do papel selecionado no aplicativo.
- **Pós-condição:** Usuário tem o seu papel ativo alterado.
- **Extensões:**
 - a) Usuário seleciona papel que já está ativo (passo c): Nenhuma ação é realizada pelo sistema e o usuário continua com o mesmo papel ativo.

6. Adicionar papel à conta

- **Descrição:** Este caso de uso descreve a interação do usuário com o sistema para adicionar um novo papel a sua conta no aplicativo.
- **Evento iniciador:** Usuário clicar na opção de adicionar papel.
- **Ator:** Todos.
- **Pré-condição:** Usuário estar *logado* na plataforma e ter apenas um ou dois papéis associados a sua conta.
- **Sequência de eventos:**
 - a) Usuário clica em "Adicionar papel" na página da sua conta do aplicativo.

- b) Sistema retorna a lista de papéis aos quais o usuário pode adicionar um papel, aqueles que ainda não estão associados a sua conta.
 - c) Usuário seleciona um dos papéis disponíveis.
 - d) Sistema redireciona o usuário para a tela de preenchimento das informações referentes ao papel escolhido.
 - e) Usuário preenche as informações requisitadas e envia os dados.
 - f) Sistema verifica as informações fornecidas.
 - g) Sistema cria novo papel para a conta do usuário e altera as informações da sessão do usuário para comportar a troca de papel.
 - h) Sistema redireciona o usuário para a tela inicial do papel criado no aplicativo.
- **Pós-condição:** Usuário tem um novo papel atribuído a sua conta.
 - **Extensões:**
 - a) Informações fornecidas pelo usuário não cumprem os requisitos mínimos do sistema (passo e): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.

4.3.2 Cliente

Com base no diagrama de casos de uso do cliente, o qual pode ser visualizado na [Figura 5](#), faz-se a descrição dos demais casos não detalhados na [subseção 4.3.1](#):

1. Gerenciar informações pessoais

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para gerenciar as suas informações pessoais de saúde, objetivo, medidas corporais, entre outros na plataforma.
- **Evento iniciador:** Cliente clicar no seu perfil do aplicativo.
- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado*.
- **Sequência de eventos:**
 - a) Cliente clica no seu perfil do aplicativo.
 - b) Sistema redireciona o usuário para a tela com todas as suas informações pessoais.
 - c) Cliente clica em alguma das informações e altera o valor do campo.
 - d) Cliente envia o perfil atualizado depois de alterar todos os campos desejados.

- e) Sistema verifica a consistência dos campos.
- f) Sistema atualiza as informações do cliente.
- **Pós-condição:** Cliente tem suas informações atualizadas no sistema.
- **Extensões:**
 - a) Informações atualizadas pelo usuário não cumprem os requisitos mínimos do sistema (passo d): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.
 - b) Alterações feitas pelo cliente são inválidas (passo e): Sistema sinaliza quais campos precisam ser ajustados e invalida a tentativa de atualização das informações.

2. Associar a um colaborador

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para se associar a um colaborador na plataforma.
- **Evento iniciador:** Cliente clicar no *link* de associação gerado pelo colaborador.
- **Ator:** Cliente.
- **Pré-condição:** Cliente ter uma conta, não estar associado a nenhum colaborador e possuir o *link* gerado por um colaborador da plataforma.
- **Sequência de eventos:**
 - a) Cliente clica no *link* gerado e fornecido pelo colaborador.
 - b) Sistema processa requisição de associação.
 - c) Sistema faz a associação do cliente com o colaborador.
- **Pós-condição:** Cliente é vinculado ao colaborador.
- **Extensões:**
 - a) Cliente já associado a um colaborador (passo b): Sistema sinaliza que o cliente já está associado a um colaborador e cancela a operação de associação.

3. Gerar plano alimentar personalizado

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para gerar um plano alimentar personalizado na plataforma.
- **Evento iniciador:** Cliente clicar em gerar plano alimentar personalizado no aplicativo.
- **Ator:** Cliente.

- **Pré-condição:** Cliente estar *logado*.
- **Sequência de eventos:**
 - a) Cliente clica em "Gerar plano alimentar personalizado" na tela de planos alimentares do aplicativo.
 - b) Sistema solicita alguns dados necessários para a geração do plano alimentar personalizado.
 - c) Cliente insere as informações requisitadas e envia os dados.
 - d) Sistema verifica as informações fornecidas.
 - e) Sistema gera o plano alimentar do cliente e o adiciona a sua lista de planos alimentares.
- **Pós-condição:** Plano alimentar personalizado é adicionado à lista de planos alimentares do cliente.
- **Extensões:**
 - a) Informações fornecidas pelo cliente não cumprem os requisitos mínimos do sistema (passo c): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.

4. Criar plano alimentar manualmente

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para criar um plano alimentar manualmente na plataforma.
- **Evento iniciador:** Cliente clicar em criar plano alimentar manualmente na plataforma.
- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado* na plataforma.
- **Sequência de eventos:**
 - a) Cliente clica em "Criar plano alimentar manualmente" na tela de planos alimentares do aplicativo.
 - b) Sistema retorna as opções de alimentos disponíveis no aplicativo.
 - c) Cliente escolhe os alimentos e os distribui para montar o seu plano alimentar.
 - d) Cliente confirma o plano feito e envia os dados para o sistema.
 - e) Sistema verifica o plano alimentar montado.
 - f) Sistema adiciona o plano alimentar montado à lista de planos alimentares do cliente.
- **Pós-condição:** Plano alimentar manual é adicionado à lista de planos alimentares do cliente.

- **Extensões:**

- a) Plano alimentar montado pelo cliente é inválido (passo e): Sistema sinaliza quais pontos precisam ser ajustados e invalida a criação do plano alimentar.

5. Gerenciar planos alimentares

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para gerenciar os seus planos alimentares na plataforma.
- **Evento iniciador:** Cliente abrir a tela de planos alimentares no aplicativo.
- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado*.
- **Sequência de eventos:**
 - a) Cliente abre a tela de planos alimentares.
 - b) Sistema redireciona o cliente para uma tela com todos os planos alimentares daquele cliente.
 - c) Cliente clica em um plano alimentar.
 - d) Sistema retorna as informações específicas do plano alimentar selecionado.
 - e) Cliente clica em "Editar plano alimentar".
 - f) Sistema redireciona o cliente para a tela de edição do plano alimentar.
 - g) Cliente faz alterações no plano alimentar.
 - h) Cliente envia o plano alimentar atualizado.
 - i) Sistema verifica o plano alimentar atualizado.
 - j) Sistema atualiza as informações do plano alimentar.
- **Pós-condição:** Cliente tem o plano alimentar especificado atualizado.
- **Extensões:**
 - a) Cliente clica em "Apagar plano alimentar"(passo e): Sistema pede confirmação de ação para o cliente e, caso seja confirmado, remove o plano alimentar do sistema.
 - b) Plano alimentar atualizado pelo cliente é inválido (passo i): Sistema sinaliza quais pontos precisam ser ajustados e invalida a atualização do plano alimentar.

6. Gerar treino personalizado

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para gerar um treino personalizado na plataforma.
- **Evento iniciador:** Cliente clicar em gerar treino personalizado no aplicativo.

- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado*.
- **Sequência de eventos:**
 - a) Cliente clica em "Gerar treino personalizado" na tela de treinos do aplicativo.
 - b) Sistema solicita alguns dados necessários para a geração do treino personalizado.
 - c) Cliente insere as informações requisitadas e envia os dados.
 - d) Sistema verifica as informações fornecidas.
 - e) Sistema gera o treino do cliente e o adiciona a sua lista de treinos.
- **Pós-condição:** Treino personalizado é adicionado à lista de treinos do cliente.
- **Extensões:**
 - a) Informações fornecidas pelo cliente não cumprem os requisitos mínimos do sistema (passo c): Sistema sinaliza quais campos precisam ser ajustados e não permite o envio dos dados até que todos os campos estejam corretos.

7. Criar treino manualmente

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para criar um treino manualmente na plataforma.
- **Evento iniciador:** Cliente clicar em criar treino manualmente na plataforma.
- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado* na plataforma.
- **Sequência de eventos:**
 - a) Cliente clica em "Criar treino manualmente" na tela de treinos do aplicativo.
 - b) Sistema retorna as opções de exercícios disponíveis no aplicativo.
 - c) Cliente escolhe os exercícios e os distribui para montar o seu treino.
 - d) Cliente confirma o treino feito e envia os dados para o sistema.
 - e) Sistema verifica o treino montado.
 - f) Sistema adiciona o treino montado à lista de treinos do cliente.
- **Pós-condição:** Treino manual é adicionado à lista de treinos do cliente.
- **Extensões:**
 - a) Treino montado pelo cliente é inválido (passo e): Sistema sinaliza quais pontos precisam ser ajustados e invalida a criação do treino.

8. Gerenciar treinos

- **Descrição:** Este caso de uso descreve a interação do cliente com o sistema para gerenciar os seus treinos na plataforma.
- **Evento iniciador:** Cliente abrir a tela de treinos no aplicativo.
- **Ator:** Cliente.
- **Pré-condição:** Cliente estar *logado*.
- **Sequência de eventos:**
 - a) Cliente abre a tela de treinos.
 - b) Sistema redireciona o cliente para uma tela com todos os treinos daquele cliente.
 - c) Cliente clica em um treino.
 - d) Sistema retorna as informações específicas do treino selecionado.
 - e) Cliente clica em "Editar treino".
 - f) Sistema redireciona o cliente para a tela de edição do treino.
 - g) Cliente faz alterações no treino.
 - h) Cliente envia o treino atualizado.
 - i) Sistema verifica o treino atualizado.
 - j) Sistema atualiza as informações do treino.
- **Pós-condição:** Cliente tem o treino especificado atualizado.
- **Extensões:**
 - a) Cliente clica em "Apagar treino"(passo e): Sistema pede confirmação de ação para o cliente e, caso seja confirmado, remove o treino do sistema.
 - b) Treino atualizado pelo cliente é inválido (passo i): Sistema sinaliza quais pontos precisam ser ajustados e invalida a atualização do treino.

4.3.3 Nutricionista

O diagrama de casos de uso presente na [Figura 6](#) ilustra os casos a serem considerados para este ator. Nesta descrição, além dos casos básicos descritos na [subseção 4.3.1](#), os casos de uso **Criar plano alimentar personalizado para cliente**, **Criar plano alimentar manualmente para cliente** e **Gerenciar dicas alimentares de clientes** serão desconsiderados, uma vez que são bem similares aos casos de uso 3, 4 e 5 da [subseção 4.3.2](#). Assim, chega-se às seguintes descrições:

1. Gerar *link* para associar cliente

- **Descrição:** Este caso de uso descreve a interação do nutricionista com o sistema para gerar um *link* para associar um cliente a ele na plataforma.
- **Evento iniciador:** Nutricionista clicar em gerar *link* de associação na tela de clientes.
- **Ator:** Nutricionista.
- **Pré-condição:** Nutricionista estar *logado*.
- **Sequência de eventos:**
 - a) Nutricionista clica em "Gerar *link* de associação" na tela de clientes do aplicativo.
 - b) Sistema processa requisição para gerar *link*.
 - c) Sistema retorna o *link* que o nutricionista deve compartilhar com seus clientes.
- **Pós-condição:** *Link* de associação ao nutricionista é gerado.

2. Gerenciar clientes associados

- **Descrição:** Este caso de uso descreve a interação do nutricionista com o sistema para gerenciar os clientes na plataforma.
- **Evento iniciador:** Nutricionista deve abrir a tela de clientes do aplicativo.
- **Ator:** Nutricionista.
- **Pré-condição:** Nutricionista estar *logado*.
- **Sequência de eventos:**
 - a) Nutricionista abre a tela de clientes do aplicativo.
 - b) Sistema retorna todos os clientes associados ao nutricionista.
 - c) Nutricionista seleciona um cliente específico.
 - d) Sistema retorna as informações do cliente e dos planos alimentares dele.
 - e) Nutricionista clica em "Desvincular cliente".
 - f) Sistema pede confirmação de ação para o nutricionista.
 - g) Nutricionista confirma ação.
 - h) Sistema remove o cliente selecionado da lista de cliente do nutricionista.
- **Pós-condição:** Cliente especificado é removido da lista de clientes do nutricionista.

3. Criar avaliação nutricional para cliente

- **Descrição:** Este caso de uso descreve a interação do nutricionista com o sistema para criar uma avaliação nutricional para um cliente na plataforma.

- **Evento iniciador:** Nutricionista clicar em criar avaliação nutricional na tela de um cliente na plataforma.
- **Ator:** Nutricionista.
- **Pré-condição:** Nutricionista estar *logado* na plataforma.
- **Sequência de eventos:**
 - a) Nutricionista clica em "Criar avaliação nutricional" na tela de um cliente selecionado pelo nutricionista no aplicativo.
 - b) Sistema requisita as informações a serem preenchidas na avaliação nutricional.
 - c) Nutricionista preenche as informações e envia os dados para o sistema.
 - d) Sistema verifica a avaliação nutricional criada.
 - e) Sistema adiciona a avaliação nutricional criada a lista de avaliações do cliente.
- **Pós-condição:** Avaliação nutricional é adicionada à lista de avaliações do cliente.
- **Extensões:**
 - a) Avaliação nutricional criada pelo nutricionista é inválida (passo d): Sistema sinaliza quais pontos precisam ser ajustados e invalida a criação da avaliação nutricional.

4.3.4 *Personal Trainer*

O diagrama de casos de uso presente na [Figura 7](#) mostra os casos a serem considerados para o *personal trainer*. Nesta descrição, além dos casos básicos descritos na [subseção 4.3.1](#), os casos de uso **Criar treino personalizado para cliente**, **Criar treino manualmente para cliente** e **Gerenciar treinos de clientes** serão desconsiderados, uma vez que são bem similares aos casos de uso 6, 7 e 8 da [subseção 4.3.2](#). Ademais, os casos de uso **Gerenciar clientes associados** e **Gerar link para associar cliente** já foram descritos na [subseção 4.3.3](#). Por fim, devido a similaridade entre os casos de uso **Criar avaliação física para cliente** e **Criar avaliação nutricional para cliente**, sendo esse último descrito também na [subseção 4.3.3](#), o caso de criação de avaliação nutricional também não será descrito nesta seção. Assim, chega-se à descrição do caso de uso **Criar treino base**:

1. Criar treino base

- **Descrição:** Este caso de uso descreve a interação do *personal trainer* com o sistema para criar um treino base, o qual pode ser aproveitado para diversos clientes, na plataforma.

- **Evento iniciador:** *Personal trainer* clicar em criar treino base na plataforma.
- **Ator:** *Personal trainer*.
- **Pré-condição:** *Personal trainer* estar *logado* na plataforma.
- **Sequência de eventos:**
 - a) *Personal trainer* clica em "Criar treino base" na tela de treinos do aplicativo.
 - b) Sistema retorna as opções de exercícios disponíveis no aplicativo.
 - c) *Personal trainer* escolhe os exercícios e os distribui para montar o seu treino.
 - d) *Personal trainer* confirma o treino feito e envia os dados para o sistema.
 - e) Sistema verifica o treino montado.
 - f) Sistema adiciona o treino montado à lista de treinos base do *personal trainer*.
- **Pós-condição:** Treino base é adicionado à lista de treinos do *personal trainer*.
- **Extensões:**
 - a) Treino montado pelo *personal trainer* é inválido (passo e): Sistema sinaliza quais pontos precisam ser ajustados e invalida a criação do treino.

4.4 Arquitetura

Com base nos requisitos e atores levantados para o sistema, foi possível agrupar as funcionalidades necessárias em cinco módulos, cada um com uma responsabilidade dentro do sistema:

- **Usuários:** responsável pela gestão de clientes, nutricionistas e *personal trainers*, além da autenticação e controle de acesso no aplicativo.
- **Treinos:** responsável pela definição, personalização e acompanhamento dos treinos de um cliente ou *personal trainer*, e gerenciamento dos exercícios e equipamentos cadastrados no sistema.
- **Planos alimentares:** responsável pelo gerenciamento das dietas, ingredientes e refeições.
- **Avaliações:** responsável pelo registro e acompanhamento de indicadores de saúde e progresso do cliente.
- **IA:** responsável pela interação com a LLM usada para a geração de recomendações de treinos e planos alimentares.

A partir disso, foram utilizados padrões de arquitetura de *software* para modelar o sistema. A adoção de diversos padrões para a arquitetura em diferentes níveis de abstração traz como vantagens a possibilidade de combinar perspectivas complementares, garantindo organização em todas as camadas. Padrões de alto nível fornecem uma visão estrutural mais estável, facilitam a evolução global e oferecem uma divisão clara das responsabilidades de cada componente do sistema, enquanto padrões de baixo nível permitem maior flexibilidade na implementação, reutilização de código e redução de acoplamento. Essa combinação contribui para o desenvolvimento de um sistema mais robusto, escalável e sustentável.

Para a arquitetura de mais alto nível do sistema, adotou-se a abordagem em camadas para definir uma clara divisão de responsabilidades entre os diversos componentes do sistema. Com isso, foram definidos os seguintes níveis:

- **Apresentação:** camada do sistema responsável pela interação com o usuário.
- **Negócio:** camada com as regras de negócio e com as implementações das funcionalidades do sistema.
- **Infraestrutura:** camada com serviços externos de infraestrutura necessários para a execução e persistência das operações.

Nesse contexto, a camada de negócio torna-se central para o projeto, uma vez que nela são orquestradas as operações e os processos importantes para a execução das funcionalidades do sistema. Por isso, foi utilizado o padrão de microsserviços para a implementação desta camada, onde cada um dos módulos definidos anteriormente será um microsserviço do sistema final. Assim, garante-se uma maior escalabilidade e modularidade da solução. Em um cenário de expansão de escopo para o sistema, por exemplo, pode-se adicionar novos microsserviços nesta camada e realizar as integrações apenas com os módulos necessários, evitando alterações em pontos já existentes.

Esses microsserviços comunicam-se entre si por meio de APIs RESTful com contratos bem definidos, garantindo a consistência das operações realizadas. Optou-se por não usar comunicação assíncrona entre os módulos devido ao custo adicional de manter uma infraestrutura com serviço de mensageria.

Dentro de cada microsserviço, orientou-se o desenvolvimento do *software* seguindo a arquitetura limpa, a fim de garantir um isolamento entre as regras de negócios e os *frameworks* e *drivers* específicos. Considerando a divisão de camadas explicada na [subseção 2.4.3](#), segue uma representação da aplicação dos conceitos teóricos dentro do contexto do projeto:

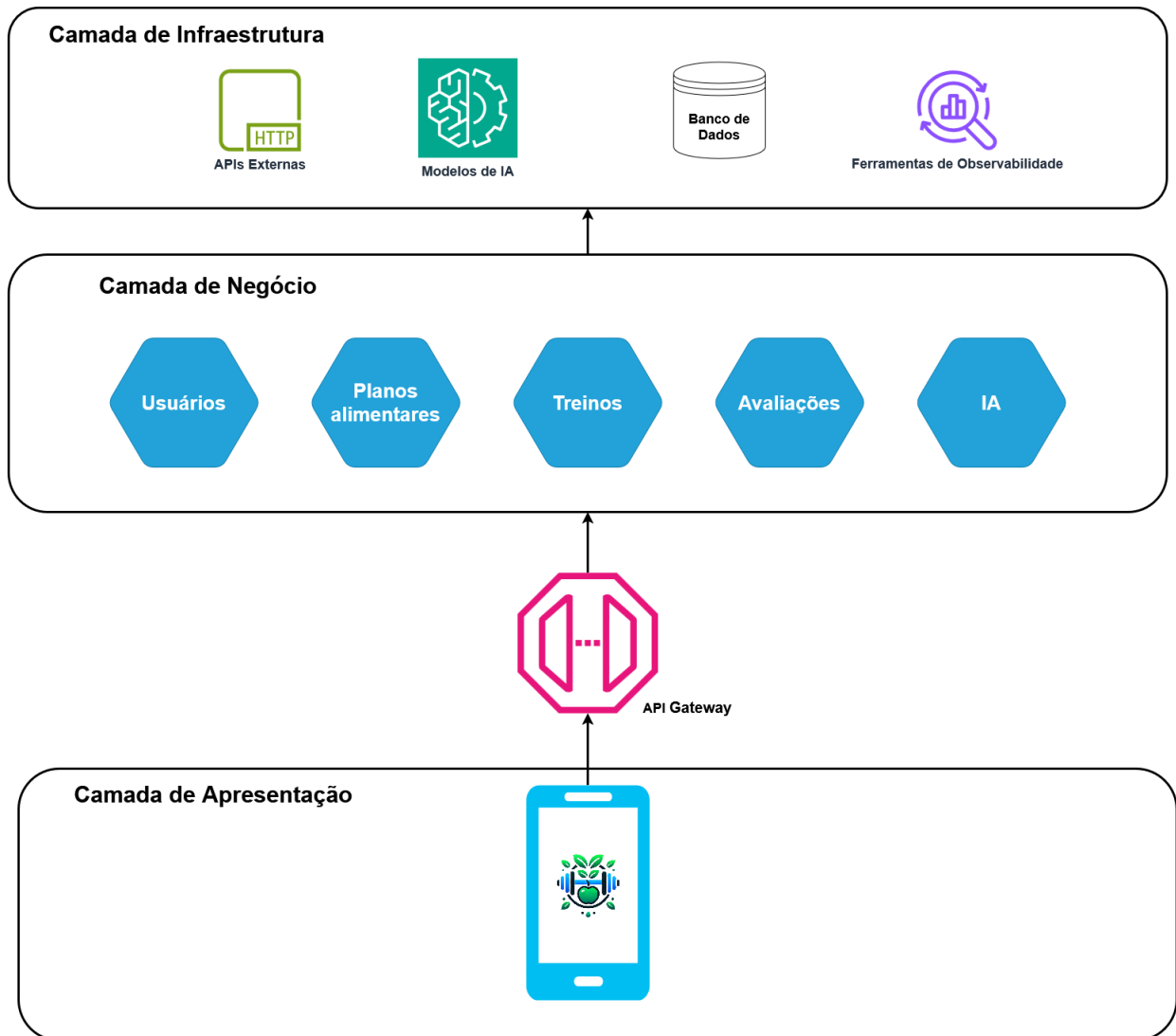
- **Entidades:** modelos centrais do módulo, como Cliente, Plano Alimentar, Treino e Avaliação, contendo apenas regras essenciais do domínio.
- **Casos de Uso:** serviços responsáveis por orquestrar validações e operações, como Criar Treino, Atualizar Plano Alimentar ou Gerar Recomendação, sem ter referências ou dependências diretas com elementos de infraestrutura.
- **Adaptadores (*Interface Adapters*):** controladores REST e mapeadores utilizados para traduzir dados entre APIs e os casos de uso.
- **Frameworks e Drivers:** implementações específicas de infraestrutura (ORM, comunicação com o modelo de LLM, integração com serviços externos, configuração de *frameworks*).

Para o consumo dos microsserviços pela camada de apresentação *mobile*, adotou-se o padrão de API Gateway. Nele, quem irá consumir as funcionalidades disponibilizadas pelos microsserviços comunica-se com uma única aplicação, a qual servirá como uma interface de entrada para o ecossistema interno com os módulos. Assim, o *gateway* traz diversos benefícios, como:

- **Redução da complexidade:** Ao concentrar a comunicação do aplicativo móvel em uma única interface, reduz-se a complexidade do cliente em gerenciar a comunicação com diversos *hosts* de microsserviços.
- **Segurança:** a autenticação, autorização e validação de requisições ficam centralizadas no *gateway*, garantindo que as requisições que são encaminhadas aos microsserviços já foram autenticadas e validadas. Além disso, o *gateway* torna-se a única aplicação exposta do *Backend*, mitigando as chances de ataques diretamente aos serviços internos.
- **Flexibilidade:** permitindo a inclusão de novas APIs sem impacto direto na aplicação cliente, sem a necessidade de incluir um novo *host* no aplicativo móvel por exemplo.

Com isso, a arquitetura geral do sistema pode ser dividida em camadas, conforme a [Figura 8](#).

Figura 8 – Arquitetura geral da solução



Fonte: Produzido pelos autores

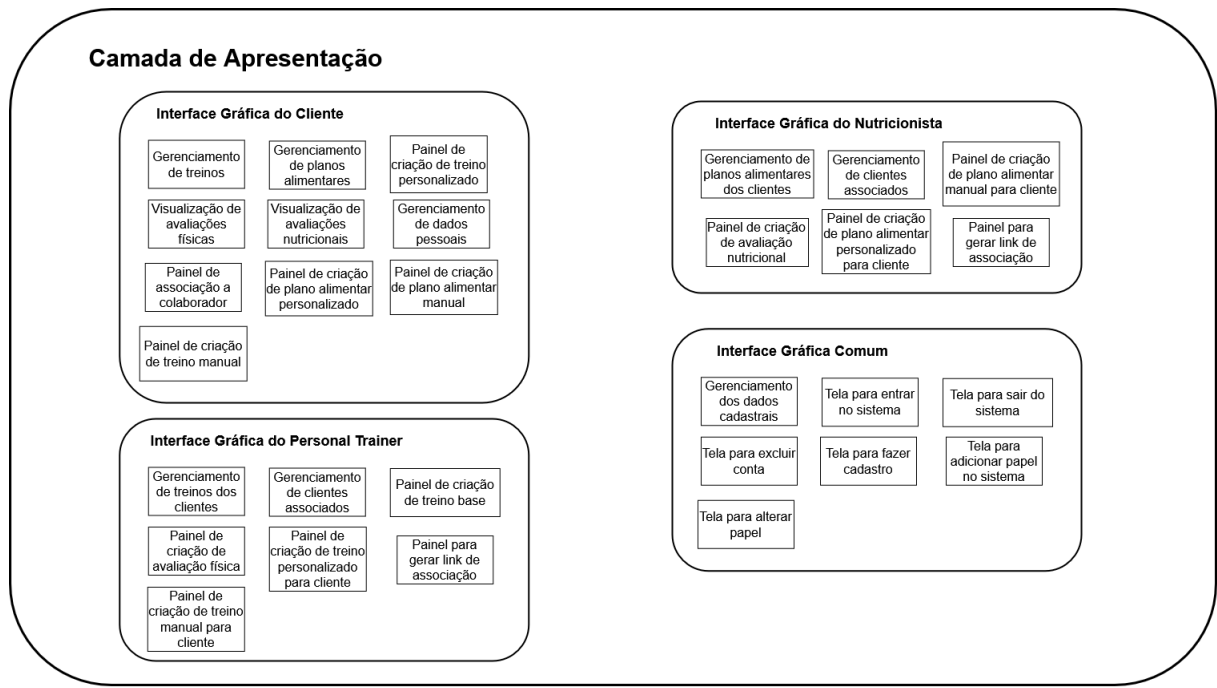
4.4.1 Camada de Apresentação

Esta camada é responsável por gerenciar a interface com o usuário final do sistema. Nela, encontram-se *frameworks* de renderização dos elementos visuais, tem-se a coleta de dados inseridos pelo usuário e a comunicação constante com a camada de negócio do sistema. Seu papel é garantir uma experiência de uso intuitiva, sem conter regras de negócio, atuando apenas como meio de comunicação entre o usuário e a lógica da aplicação.

Além disso, essa camada precisa gerenciar as interfaces existentes para diferentes tipos de usuários, garantindo que o usuário tenha acesso às interfaces condizentes com seu perfil dentro do sistema. Considerando os atores presentes na solução e suas funcionalida-

des, pode-se detalhar a Camada de Apresentação com relação às suas interfaces gráficas conforme a Figura 9.

Figura 9 – Camada de Apresentação do sistema



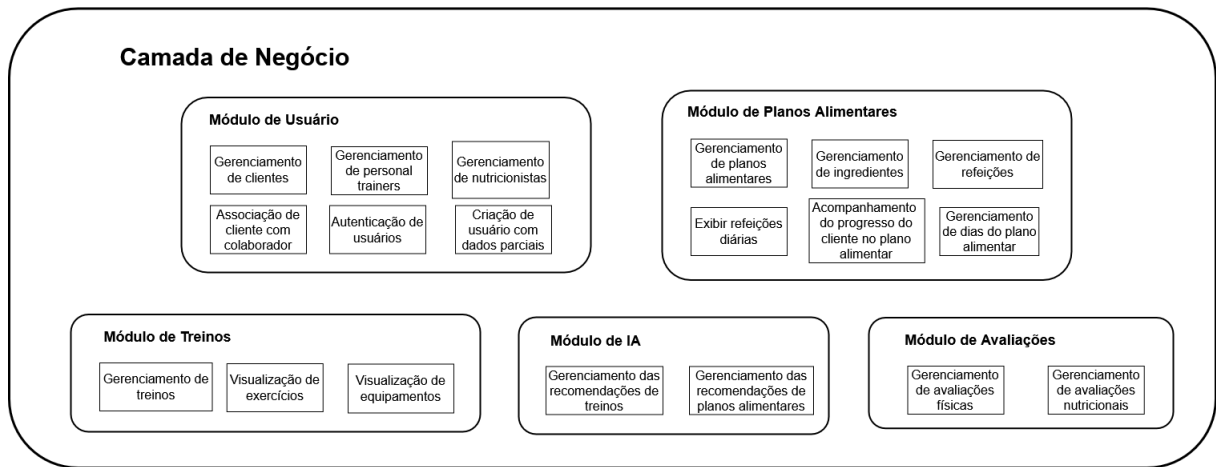
Fonte: Produzido pelos autores

4.4.2 Camada de Negócio

Esta camada concentra as regras de negócio e os processos que definem o funcionamento do sistema. Nela, são tomadas as decisões e aplicadas as lógicas necessárias para atender aos objetivos do domínio, como validações e geração de resultados. A Camada de Negócio deve ter uma implementação agnóstica de elementos de infraestrutura, como *frameworks*, tecnologias específicas, entre outros.

Considerando os módulos discutidos anteriormente, juntamente com suas responsabilidades, detalha-se na Figura 10 as funcionalidades de negócio importantes que cada microserviço deve implementar.

Figura 10 – Camada de Negócio do sistema



Fonte: Produzido pelos autores

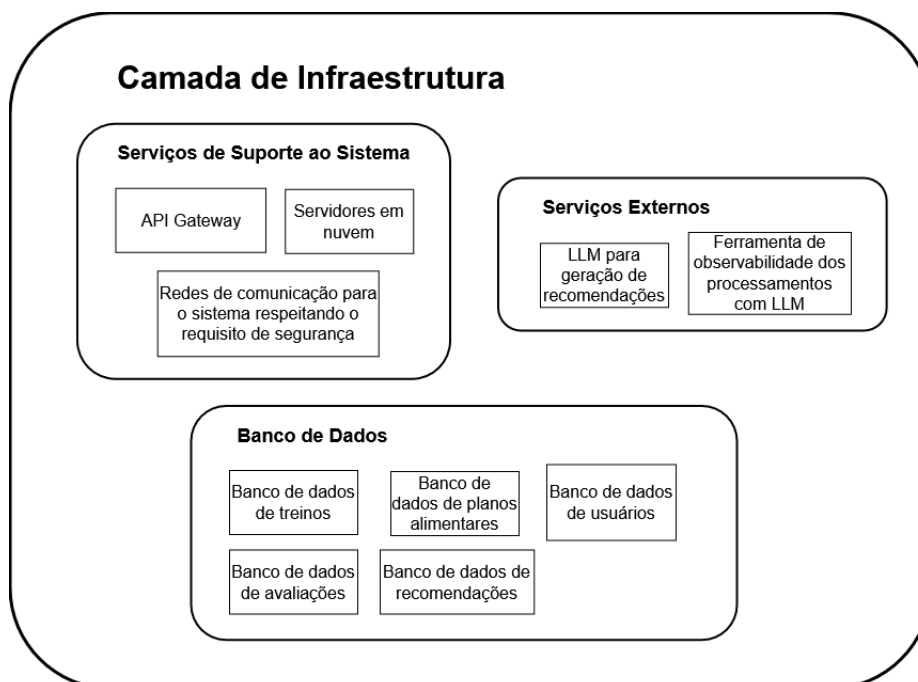
A organização interna dos microsserviços segue os padrões descritos anteriormente, garantindo que as regras de negócio permaneçam independentes de detalhes de infraestrutura e possam evoluir de maneira modular.

4.4.3 Camada de Infraestrutura

A Camada de Infraestrutura fornece o suporte necessário para a execução e persistência das operações do sistema. Ela é responsável por armazenar, recuperar e disponibilizar dados, além de realizar a comunicação com serviços externos e gerenciar recursos do sistema. Seu propósito é dar sustentação à camada de negócios, garantindo a confiabilidade das operações realizadas. Nela, encontram-se as tecnologias e *frameworks* específicos, como *drivers* para comunicação com SGBDs, ferramentas para observabilidade da solução e o modelo de IA que será usado para a geração de recomendações.

O detalhamento dos principais elementos encontrados nesta camada é apresentado na [Figura 11](#). Vale ressaltar que, para garantir a segregação proposta com o uso de microsserviços, os bancos de dados presentes na infraestrutura também são segregados para cada módulo do sistema, mantendo o compromisso com a modularidade da solução.

Figura 11 – Camada de Infraestrutura do sistema



Fonte: Produzido pelos autores

5 Desenvolvimento do Trabalho

5.1 Prototipação das Telas

A fim de facilitar o desenvolvimento da interface *mobile* para o aplicativo, foi utilizado o Figma ([Figma, Inc., 2025](#)) como ferramenta de prototipação. Ele permite criar protótipos interativos das telas de forma colaborativa e acessível. Ela permite validar o design e a experiência do usuário antes do início da implementação, garantindo que as telas atendam aos requisitos funcionais e estéticos do projeto.

Essa forma de trabalhar agilizou o desenvolvimento *frontend* do projeto, na medida em que definiu os elementos que cada tela deveria conter, como deveria ser seu comportamento e como deveria ser o fluxo de navegação do usuário pelo aplicativo. Além disso, os protótipos das telas foram importantes para o desenvolvimento do *backend*, uma vez que expuseram a necessidade de *endpoints* específicos do servidor para implementar as funcionalidades previstas.

A seguir, apresentam-se as descrições das principais telas produzidas no Figma, que ajudaram a nortear o desenvolvimento do aplicativo. O resultado da prototipação pode ser encontrado no [Apêndice A](#).

- **Telas iniciais:** Essas telas servem para apoiar o usuário nos cenários não autenticados de uso do aplicativo. A partir delas, é possível registrar-se como cliente, nutricionista ou *personal trainer*. Além disso, o usuário pode realizar seu login com *email* e senha.
- **Homescreen:** Após fazer login no aplicativo, o usuário é direcionado para a página de *home*, onde ele pode acessar seu treino do dia, o plano alimentar do dia, além de consultar seu progresso atual diário. A navegação para outras telas pode ser feita utilizando a aba inferior.
- **Treinos:** O usuário pode consultar seus treinos e gerenciá-los nesta aba do aplicativo.
- **Nutrição:** O usuário pode consultar seus planos alimentares e gerenciá-los nesta aba do aplicativo.
- **Avaliações:** O usuário pode consultar suas avaliações físicas e nutricionais nessa aba do aplicativo.

5.2 Tecnologias Utilizadas

Durante o desenvolvimento do trabalho, foram utilizadas tecnologias e ferramentas para facilitar a implementação da solução idealizada. Para a escolha das tecnologias, foram analisados os seguintes quesitos: desempenho, facilidade de integração, curva de aprendizado para os membros, suporte da comunidade e alinhamento com os requisitos do projeto.

5.2.1 *Frameworks* de desenvolvimento *mobile* multiplataforma

Para o desenvolvimento *mobile*, optou-se pelo React Native ([Meta Open Source, 2025](#)), um dos *frameworks* mais consolidados para a criação de aplicativos móveis multiplataforma. Sua principal vantagem é a possibilidade de, em um único código, gerar aplicações nativas tanto para Android quanto para iOS, o que reduz o tempo de desenvolvimento e manutenção. Além disso, sua ampla comunidade garante suporte contínuo, documentação extensa e bibliotecas complementares para resolver problemas e necessidades encontrados durante o desenvolvimento. No contexto deste projeto, tal característica foi essencial para garantir agilidade sem perda de qualidade, visto que a equipe se concentrou em manter uma única base de código.

Junto ao React Native, foi utilizada a biblioteca de componentes *gluestack-ui* ([GeekyAnts, 2025](#)), responsável por fornecer um conjunto de componentes visuais padronizado e com ampla utilização no mercado. Essa escolha teve como objetivo manter a padronização visual do aplicativo e acelerar o desenvolvimento das telas, com foco em garantir uma experiência de usuário intuitiva e agradável. Ao mesmo tempo em que a biblioteca de componentes garante a consistência visual para o aplicativo, ainda é possível customizá-la de modo a permitir a incorporação do *design* do sistema para a ferramenta.

Além disso, fez-se o uso do Expo ([650 Industries, Inc, 2025](#)), uma plataforma que simplifica o desenvolvimento de aplicativos em React Native ao fornecer um conjunto de ferramentas, bibliotecas e serviços integrados. Um recurso importante presente na solução do Expo é o aplicativo Expo Go, que permite executar o projeto diretamente no dispositivo, sem a necessidade de gerar um *build* completo e instalá-lo manualmente. Com tal ferramenta, basta escanear um código QR gerado pela aplicação no ambiente de desenvolvimento para visualizar o resultado das alterações realizadas. Essa funcionalidade contribuiu de forma significativa para acelerar o ciclo de testes e validações no aplicativo *mobile*, tanto em dispositivos Android quanto iOS. Ademais, o Expo disponibiliza bibliotecas adicionais que permitem interagir com recursos do dispositivo por meio de APIs fornecidas pela ferramenta, como a disponibilização de um serviço de persistência de dados com criptografia no dispositivo, importante para ofuscação de dados de autenticação do usuário.

5.2.2 Frameworks utilizados no Backend

Para o desenvolvimento do *Backend*, utilizou-se o Node.js ([OpenJS Foundation, 2025b](#)), ambiente de execução que permite usar JavaScript/TypeScript no lado do servidor. Do ponto de vista de desempenho, essa escolha foi motivada pela sua eficiência em lidar com operações de entrada e saída não bloqueantes, característica relevante em sistemas que demandam alta escalabilidade e múltiplas requisições simultâneas, especialmente levando em consideração o caso de uso com serviços de IA externos dentro sistema, os quais costumam ter um tempo de execução elevado. Para estruturar os microsserviços, adotou-se como padrão o *framework* Fastify ([OpenJS Foundation, 2025a](#)), reconhecido por seu desempenho superior em relação a alternativas como Express.js, além de oferecer um ecossistema de *plugins* com módulos e suporte nativo para diversas operações necessárias para aplicações *backend*, como a validação de objetos recebidos via APIs, roteamento das requisições que chegam no servidor e *logging* das ações realizadas.

Dentro da perspectiva do uso da Inteligência Artificial no projeto, a camada responsável pela interação com modelos de linguagem utilizados foi desenvolvida com a utilização do *framework* LangChain ([LangChain, Inc., 2025](#)), uma ferramenta projetada para facilitar a integração de aplicações com LLMs e fontes externas de informação. O LangChain fornece abstrações que simplificam a comunicação com os provedores de modelos de IA, permitindo que o sistema se torne menos dependente de um provedor específico ao codificar sua solução. Ademais, gerenciamento de prompts e implementação de técnicas como o RAG, essenciais para reduzir alucinações do modelo e aumentar a relevância das respostas, também são facilitados pelo *framework*.

Outra vantagem de usar o LangChain para integração com a LLM é a observabilidade garantido pela solução. Dentro do ecossistema do *framework*, existe uma ferramenta para monitoramento das interações com a LLM, o LangSmith. Nesta ferramenta, é possível visualizar o consumo total de *tokens*, custo total das chamadas e *otracing* de todas as execuções dentro do período de tempo selecionado. Assim, é possível ter maior controle sobre o uso do modelo pelo sistema e gerar *insights* relevantes para a plataforma desenvolvida.

O provedor de LLM adotado foi o Gemini ([Google DeepMind, 2025](#)), desenvolvido pelo Google. O seu bom custo-benefício e a sua integração facilitada com a Google Cloud Platform (GCP), opção de infraestrutura para a solução, foram fatores que motivaram essa tomada de decisão. Além do aspecto econômico, a confiabilidade do ecossistema Google, que oferece suporte contínuo e estabilidade para seus produtos, foi outro ponto considerado para a escolha do Gemini.

Por fim, para facilitar a integração entre o *backend* e o *frontend* do projeto, utilizou-se a ferramenta Swagger ([SmartBear Software, 2025](#)), responsável por gerar automati-

camente uma página em HTML contendo a documentação interativa das interfaces do servidor. Essa página exibe informações detalhadas sobre cada rota implementada, como URL, método HTTP, parâmetros de entrada e respostas esperadas, permitindo que a equipe de desenvolvimento compreenda e utilize corretamente os serviços disponibilizados. Além disso, o Swagger possibilita testar diretamente os endpoints a partir dessa interface gerada, o que agiliza o processo de validação durante o desenvolvimento, permitindo verificar parâmetros opcionais e obrigatórios, formatos dos objetos de resposta e exceções retornadas em diferentes cenários.

5.2.3 Linguagem de programação

Por fim, a linguagem de programação TypeScript ([Microsoft, 2025](#)) foi escolhida como a linguagem unificadora do projeto, tanto no *Frontend* quanto no *Backend*. O TypeScript adiciona tipagem estática ao JavaScript, o que aumenta a confiabilidade do código e reduz erros em tempo de execução. A tipagem dos dados é essencial dentro de um projeto com múltiplos módulos, garantindo maior segurança na integração entre diferentes microserviços e maior clareza na definição de contratos de dados.

A unificação da linguagem também trouxe vantagens à equipe, que pôde compartilhar conhecimento para as diferentes partes do sistema sem a necessidade de lidar com múltiplas sintaxes ou paradigmas, o que reduziu a curva de aprendizado para o andamento do projeto. Por fim, a adoção de uma linguagem comum, com suporte ao Node.js e ao *Node Package Manager* (NPM), permitiu a utilização de algumas bibliotecas auxiliares em comum, como utilitários para datas e *timestamps*, além de validadores de formatação de objetos recebidos.

Desse modo, as escolhas de tecnologias possibilitaram a construção de uma solução escalável, segura e de fácil manutenção, alinhada às melhores práticas de desenvolvimento e às necessidades identificadas durante a fase de levantamento de requisitos.

5.2.4 Bancos de dados

A fim de manter a modularidade inerente ao modelo de microserviços, optou-se por trabalhar com bancos de dados separados para cada uma das aplicações implementadas no *backend* do projeto. Desta forma, cada aplicação passa a ser responsável por definir e implementar a modelagem de seu domínio, na forma de tabelas e relacionamentos encapsulados em esquemas próprios. Além disso, essa estrutura modular facilita migrações de dados, evoluções na arquitetura das entidades e eventuais *rollbacks*, na medida em que o escopo de tais alterações passa a ser limitado pelo microserviço, ao invés de englobar o sistema inteiro. Por fim, essa organização promove a política de menor privilégio, ao evitar que aplicações possam ter acesso direto a dados de outros domínios.

Com relação as tecnologias usadas para o armazenamento de dados, adotou-se uma estratégia que combina diferentes paradigmas de bancos de dados, relacional e não relacional, de modo a atender melhor às demandas do sistema. O principal banco utilizado foi o PostgreSQL ([PostgreSQL Global Development Group, 2025](#)), um sistema gerenciador de banco de dados relacional amplamente reconhecido por sua robustez, confiabilidade e aderência ao padrão SQL. As principais razões que motivaram a sua escolha foram a garantia de consistência transacional, o suporte avançado a consultas complexas, possibilidade de usar extensões na base de dados e a disponibilidade de recursos adicionais, como o tipo de dado JSONB, que permite trabalhar com informações semiestruturadas dentro do paradigma relacional.

Paralelamente, explorou-se também o MongoDB ([MongoDB, Inc., 2025](#)), um banco de dados não relacional orientado a documentos. Sua utilização possibilitou entender os pontos positivos do modelo NoSQL, especialmente no que se refere à flexibilidade na definição de esquemas dos dados armazenados. Embora o foco da solução tenha permanecido no PostgreSQL, o aprendizado com o MongoDB contribuiu para ampliar a visão sobre outras abordagens de persistência de dados e sobre cenários em que modelos não relacionais podem se mostrar benéficos.

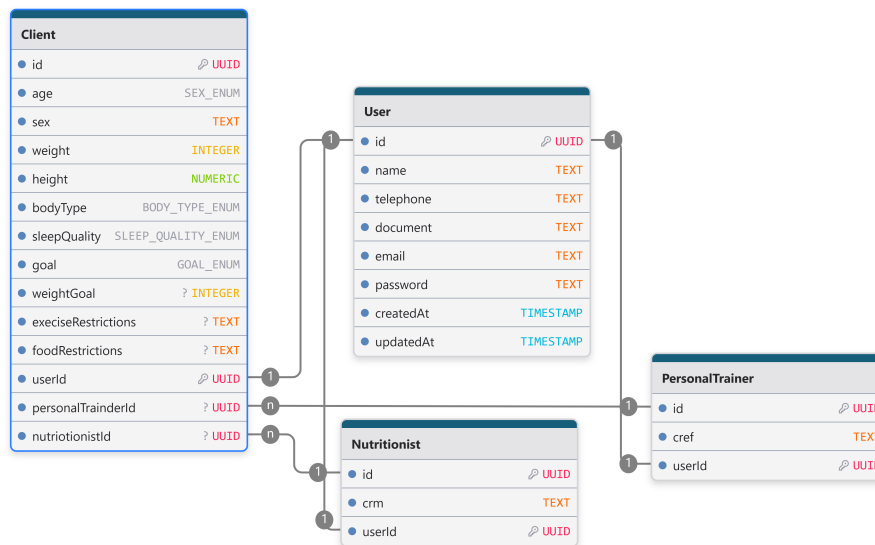
5.3 Projeto e Implementação

5.3.1 Microserviço de Usuários

O módulo de usuários do sistema é responsável pelo gerenciamento de clientes, nutricionistas e *personal trainers*. Ademais, neste microserviço encontram-se as APIs relativas ao processo de autenticação e autorização do aplicativo. Vale ressaltar que o gerenciamento dos usuários não é restrito apenas às operações de CRUD básicas (criação, edição, visualização e remoção de dados), foram desenvolvidas, também, operações de associação e desassociação de um colaborador a um cliente.

Para permitir que o usuário tenha mais de um papel no sistema, modelou-se o diagrama Entidade-Relacionamento do módulo com uma tabela principal, contendo as informações cadastrais comuns a todos os papéis do sistema, em conjunto com outras três tabelas para cada papel possível dentro do sistema: cliente, nutricionista e *personal trainer*, cada uma com as informações específicas referentes a cada papel. Além disso, a tabela de clientes possui uma relação com as tabelas de nutricionistas e de *personal trainers*, uma vez que um cliente pode estar associado a um colaborador. Assim, a modelagem final das tabelas do banco de dados é encontrada na [Figura 12](#). O SGBD utilizado para este microserviço foi o PostgreSQL.

Figura 12 – Modelagem do banco de dados de usuários



Fonte: Produzido pelos autores

Com isso, foram desenvolvidos os *endpoints* responsáveis pelo gerenciamento das entidades do microserviço, o que resulta em um conjunto de APIs para o gerenciamento dos nutricionistas (Tabela 3); um conjunto para o gerenciamento de clientes (Tabela 4); um conjunto para o gerenciamento de *personal trainers* (Tabela 5); e uma API para a criação de um usuário base (Tabela 6).

Sobre as funcionalidades de autenticação e de controle de acesso no aplicativo implementadas neste módulo, sua principal responsabilidade consiste na criação de um *token* válido com as informações do usuário que deseja entrar no sistema. Para isso, existe suporte para três cenários de autenticação do usuário:

1. **Novo usuário no sistema:** Para que um novo usuário tenha acesso direto ao sistema sem passar pelo fluxo de *login*, no momento em que o cadastro é bem sucedido, um JWT é gerado e retornado para ele.
2. **Usuário quer entrar no sistema pela primeira vez:** Quando o usuário tem um cadastro e precisa acessar o sistema pela primeira vez, ele deve fornecer suas informações de identificação, neste caso, o seu *email*, juntamente com a senha. Caso as informações sejam válidas, o JWT é gerado para o usuário e retornado a ele.
3. **Usuário autenticado com *token* expirado:** Este cenário ocorre quando o usuário já acessou o sistema previamente, mas seu *token* JWT expirou. Como o *login* persistente é um requisito não funcional do projeto, fez-se uma implementação para

re-autenticar o usuário na plataforma, gerando um *token* atualizado a partir das informações do antigo. Para isso, faz-se a validação do JWT expirado e, caso as informações sejam válidas e íntegras, o novo *token* é retornado ao usuário.

O primeiro cenário de autenticação descrito anteriormente é encontrado nas APIs de criação de nutricionistas, clientes e *personal trainers*, presentes na [Tabela 3](#), [Tabela 4](#) e [Tabela 5](#), respectivamente. Já os demais cenários de autenticação são implementados nos *endpoints* detalhados na [Tabela 7](#).

Outro aspecto importante deste microserviço é o tratamento adotado para o armazenamento das senhas no banco de dados. Mantendo o compromisso estabelecido com a LGPD, as senhas dos usuários não foram armazenadas às claras, e sim foram armazenados os *hashes* das senhas. Para isso, foi utilizado o algoritmo de *hash* Bcrypt, focado na geração de *hashes* seguros de senhas.

Tabela 3 – *Endpoints* de nutricionistas do microserviço de Usuários

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /users/nutritionists	Objeto com os dados cadastrais do nutricionista	Cria um nutricionista e retorna um <i>token</i> de autenticação para uso no sistema.
GET /users/nutritionists	-	Retorna os nutricionistas criados no sistema.
GET /users/nutritionists/{id}	Identificador do nutricionista	Retorna as informações de um nutricionista específico.
PUT /users/nutritionists/{id}	Identificador do nutricionista e informações atualizadas dele	Atualiza as informações do nutricionista especificado.
DELETE /users/nutritionists/{id}	Identificador do nutricionista	Remove a conta do sistema do nutricionista especificado.
GET /users/nutritionists/{id}/clients	Identificador do nutricionista	Retorna os clientes associados ao nutricionista especificado.

Fonte: Produzido pelos autores

Tabela 4 – *Endpoints* de clientes do microserviço de Usuários

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /users/clients	Objeto com os dados cadastrais do cliente	Cria um cliente e retorna um <i>token</i> de autenticação para uso no sistema.
GET /users/clients	-	Retorna os clientes criados no sistema.
GET /users/clients/complete-information/{userId}	Identificador do usuário com dados parciais	Completa as informações do cliente para um usuário com dados parciais previamente cadastrado.
GET /users/clients/{id}	Identificador do cliente	Retorna as informações do cliente especificado.
PUT /users/clients/{id}	Identificador do cliente e informações atualizadas dele	Atualiza as informações do cliente especificado.
DELETE /users/clients/{id}	Identificador do cliente	Remove a conta do sistema do cliente especificado.
POST /users/clients/{id}/personals	Identificador do cliente e do personal	Associa a conta do cliente especificado com o personal também especificado.
DELETE /users/clients/{id}/personals	Identificador do cliente	Remove a associação do cliente especificado com o seu personal, caso exista.
POST /users/clients/{id}/nutritionists	Identificador do cliente e do nutricionista	Associa a conta do cliente especificado com o nutricionista também especificado.
DELETE /users/clients/{id}/nutritionists	Identificador do cliente	Remove a associação do cliente especificado com o seu nutricionista, caso exista.

Fonte: Produzido pelos autores

Tabela 5 – *Endpoints* de *personal trainers* do microserviço de Usuários

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /users/personals	Objeto com os dados cadastrais do personal	Cria um <i>personal trainer</i> e retorna um <i>token</i> de autenticação para uso no sistema.
GET /users/personals	-	Retorna os <i>personal trainers</i> criados no sistema.
GET /users/personals/{id}	Identificador do personal	Retorna as informações do <i>personal trainer</i> especificado.
PUT /users/personals/{id}	Identificador do personal e informações atualizadas dele	Atualiza as informações do <i>personal trainer</i> especificado.
DELETE /users/personals/{id}	Identificador do personal	Remove a conta do sistema do <i>personal trainer</i> especificado.
GET /users/personals/{id}/clients	Identificador do personal	Retorna os clientes associados ao <i>personal trainer</i> especificado.

Fonte: Produzido pelos autores

Tabela 6 – *Endpoints* de usuários genéricos do microserviço de Usuários

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /users/users	Objeto com os dados cadastrais parciais de um usuário	Cadastro dos dados parciais de um usuário.

Fonte: Produzido pelos autores

Tabela 7 – *Endpoints* de autenticação do microserviço de Usuários

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /auth/login	Objeto com os dados de <i>login</i>	Efetua o <i>login</i> do usuário no sistema.
POST /auth/refresh-token	Token expirado do usuário	A partir de um token expirado, retorna novo token válido para uso do usuário.

Fonte: Produzido pelos autores

5.3.2 Microsserviço de Planos alimentares

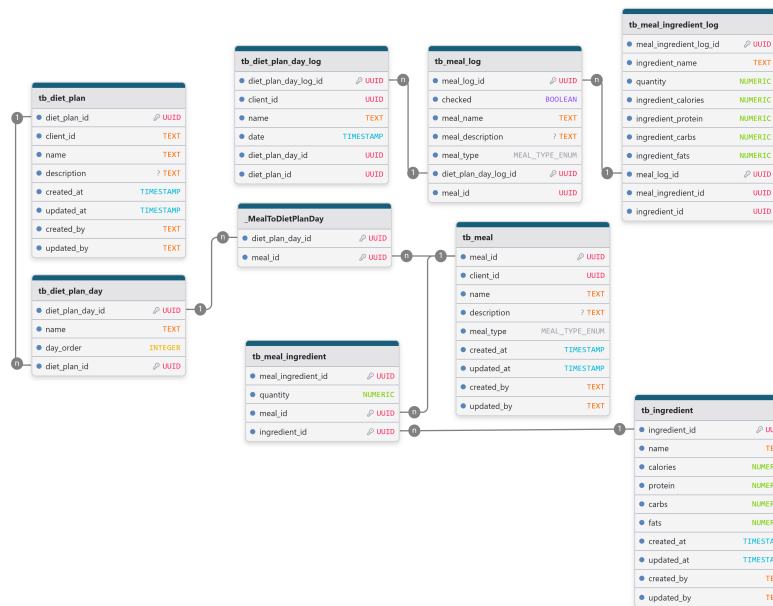
O módulo de planos alimentares é responsável pelo gerenciamento de planos alimentares, refeições e ingredientes. Além disso, este microsserviço oferece funcionalidades de acompanhamento da dieta atual do cliente e de seu progresso ao longo do tempo, permitindo uma visão clara da adesão ao plano e da evolução nutricional do usuário.

Para a modelagem do módulo, adotou-se uma abordagem granular sobre a estrutura do plano alimentar, visando maior flexibilidade e escalabilidade. O plano alimentar foi segmentado em dias, sendo cada dia composto por diversas refeições, e cada refeição por vários ingredientes, com suas respectivas quantidades. O sistema mantém, ainda, um plano alimentar ativo para cada cliente, utilizado para o monitoramento contínuo do progresso. Dessa forma, é possível registrar o histórico de refeições realizadas e exibir as refeições correspondentes ao dia atual, o que facilita o cumprimento e o planejamento diário da dieta.

Dessa forma, modelou-se o diagrama Entidade-Relacionamento do módulo conforme a [Figura 13](#). Nele, tem-se uma tabela para armazenar os planos alimentares dos clientes. Como cada plano alimentar é composto por vários dias, representa-se isso por meio de uma relação 1-N entre a tabela de plano alimentar e a tabela de dias do plano. Tem-se uma relação similar entre um dia do plano e suas refeições, com a diferença de que uma refeição pode ser reaproveitada para diferentes dias. Por isso, a relação entre essas entidades tem cardinalidade N-N. Analogamente, tem-se uma relação N-N entre um ingrediente e uma refeição, com a quantidade do ingrediente em uma refeição representada na tabela intermediária do relacionamento.

Ademais, tem-se a tabela que armazena o dia atual do plano alimentar do cliente para uma data específica. O dia do plano alimentar contém as suas refeições, representado por uma relação 1-N, em que cada uma pode ter sido concluída ou não pelo cliente, o que é usado para o acompanhamento do progresso. Por fim, cada refeição tem as informações de seus ingredientes, representado por uma relação 1-N também. Por fim, foi escolhido o PostgreSQL como SGBD do microsserviço.

Figura 13 – Modelagem do banco de dados de planos alimentares



Fonte: Produzido pelos autores

A partir disso, as funcionalidades do microsserviço foram desenvolvidas e expostas por meio das seguintes APIs:

- Gerenciamento dos ingredientes do sistema (Tabela 8). Operações básicas de CRUD dos ingredientes.
- Gerenciamento dos planos alimentares (Tabela 9). Além das operações básicas de CRUD, possui uma operação para selecionar um plano alimentar como ativo e um *endpoint* específico para o cenário de criação de um plano alimentar personalizado com IA.
- Gerenciamento dos dias de um plano alimentar (Tabela 10). Além das operações básicas de CRUD, possui uma operação para adicionar ou remover uma refeição específica de um dia do plano.
- Gerenciamento das refeições (Tabela 11). Além das operações básicas de CRUD, possui uma operação para adicionar ou remover um ingrediente específico e outra para alterar a quantidade de um ingrediente em uma refeição.
- Operação para recuperar as refeições para a data especificada, com base no plano alimentar ativo para ela (Tabela 12).

- Gerenciamento do progresso do cliente ([Tabela 13](#)). Possui uma operação para recuperar o progresso de um dia específico do cliente e operações para marcar uma refeição como concluída ou não concluída.

Tabela 8 – *Endpoints* de ingredientes do microserviço de Planos alimentares

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /nutrition/ingredients	Objeto com os dados do ingrediente a ser registrado	Cadastro do ingrediente no sistema.
GET /nutrition/ingredients	Nome do ingrediente e opções de paginação	Retorna uma lista paginada com os ingredientes que satisfazem a busca.
GET /nutrition/ingredients/{id}	Identificador do ingrediente	Retorna as informações do ingrediente especificado.
PUT /nutrition/ingredients/{id}	Identificador do ingrediente e informações atualizadas dele	Atualiza as informações de um ingrediente especificado.
DELETE /nutrition/ingredients/{id}	Identificador do ingrediente	Remove o ingrediente especificado do sistema.

Fonte: Produzido pelos autores

Tabela 9 – *Endpoints* de planos alimentares do microserviço de Planos alimentares

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /nutrition/diet-plans	Objeto com os dados de um plano alimentar e identificador do criador do plano alimentar	Criação de um novo plano alimentar no sistema.
GET /nutrition/diet-plans	Identificador do cliente	Retorna todos os planos alimentares associados a um cliente específico.
GET /nutrition/diet-plans/{id}	Identificador do plano alimentar	Retorna as informações do plano alimentar especificado.
POST /nutrition/diet-plans/ai	Objeto com os dados para geração por IA	Registra um plano alimentar personalizado feito com IA na base de dados de planos alimentares do cliente.
PATCH /nutrition/diet-plans/{id}	Identificador do plano alimentar e objeto com campos a serem atualizados	Atualiza as informações do plano alimentar especificado.
DELETE /nutrition/diet-plans/{id}	Identificador do plano alimentar	Exclui o plano alimentar especificado do sistema.
PUT /nutrition/diet-plans/activate/{id}	Identificador do plano alimentar	Ativa o plano alimentar especificado para o cliente.

Fonte: Produzido pelos autores

Tabela 10 – *Endpoints* de dias de planos alimentares do microserviço de Planos alimentares

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /nutrition/diet-plan-days	Objeto com os dados de um dia de plano alimentar	Criação de um novo dia de plano alimentar.
GET /nutrition/diet-plan-days	Identificador do plano alimentar e do cliente	Retorna a lista de dias do plano alimentar especificado associado ao cliente.
GET /nutrition/diet-plan-days/{id}	Identificador do dia do plano alimentar	Retorna as informações do dia do plano alimentar especificado.
PATCH /nutrition/diet-plan-days/{id}	Identificador do dia do plano alimentar e objeto com campos a serem atualizados	Atualiza as informações do dia do plano alimentar especificado.
DELETE /nutrition/diet-plan-days/{id}	Identificador do dia do plano alimentar	Remove o dia do plano alimentar especificado do sistema.
POST /nutrition/diet-plan-days/meal/{id}	Identificador do dia do plano alimentar e da refeição	Adiciona uma refeição ao dia do plano alimentar especificado.
DELETE /nutrition/diet-plan-days/meal/{id}	Identificador do dia do plano alimentar e da refeição	Remove uma refeição do dia do plano alimentar especificado.

Fonte: Produzido pelos autores

Tabela 11 – *Endpoints* de refeições do microserviço de Planos alimentares

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /nutrition/meals	Identificador do criador e objeto com os dados de uma refeição	Criação de uma nova refeição no sistema.
GET /nutrition/meals	Identificador do cliente, nome da refeição e opções de paginação	Retorna uma lista paginada de refeições associadas ao cliente.
GET /nutrition/meals/{id}	Identificador da refeição	Retorna as informações da refeição especificada.
PATCH /nutrition/meals/{id}	Identificador da refeição e objeto com campos a serem atualizados	Atualiza as informações da refeição especificada.
DELETE /nutrition/meals/{id}	Identificador da refeição	Remove a refeição especificada do banco de dados.
POST /nutrition/meals/{mealId}/ingredients/{ingredientId}	Identificador da refeição, identificador do ingrediente e objeto com quantidade do ingrediente em gramas	Adiciona a quantidade, em gramas, especificada de um ingrediente à refeição.
PUT /nutrition/meals/{mealId}/ingredients/{mealIngredientId}	Identificador da refeição, identificador do ingrediente e nova quantidade do ingrediente em gramas	Atualiza a quantidade do ingrediente especificado na refeição.
DELETE /nutrition/meals/{mealId}/ingredients/{mealIngredientId}	Identificador da refeição e identificador do ingrediente na refeição	Remove um ingrediente específico de uma refeição.

Fonte: Produzido pelos autores

Tabela 12 – *Endpoints* de calendário de planos alimentares do microserviço de Planos alimentares

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /calendar/{client}/date/{date}	Identificador do cliente e data no formato especificado	Busca ou cria um dia da dieta para o cliente na data especificada e o retorna.

Fonte: Produzido pelos autores

Tabela 13 – *Endpoints* de progresso de planos alimentares do microserviço de Planos alimentares

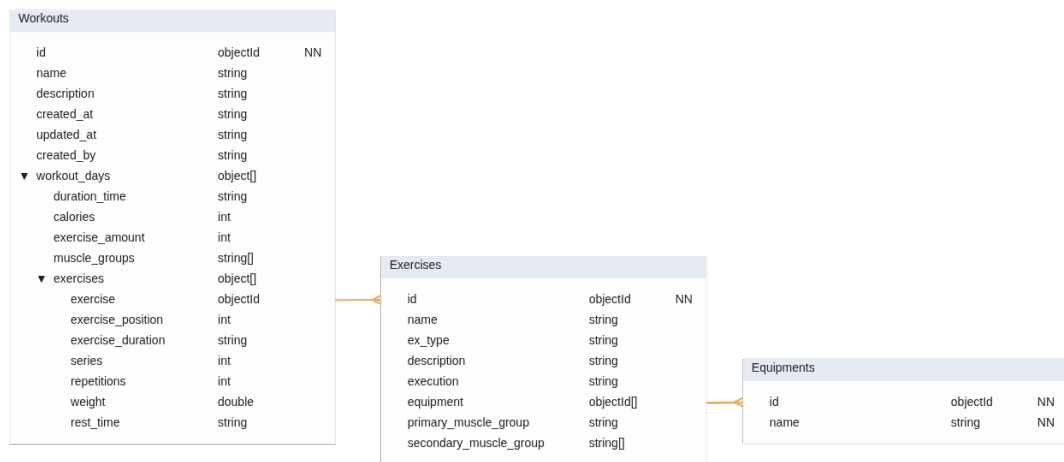
Rota HTTP	Parâmetros da Rota	Descrição da Operação
GET /progress/{client}/date /{date}	Identificador do cliente e data no formato especificado	Retorna valores do progresso atual para o dia especificado, informando quantidade de calorias, carboidratos, gorduras e proteínas totais, consumidas e remanescentes.
PATCH /progress/{mealLogId}/check	Identificador do registro de refeição	Marca uma refeição do registro diário como cumprida.
PATCH /progress/{mealLogId}/uncheck	Identificador do registro de refeição	Desmarca uma refeição do registro diário como cumprida.

Fonte: Produzido pelos autores

5.3.3 Microserviço de Treinos

O módulo de treinos é responsável pelo gerenciamento dos treinos, exercícios e equipamentos. Além disso, este serviço possui a lógica de cálculo do gasto calórico dos treinos. Para a modelagem do banco de dados do módulo de treinos, adotou-se uma abordagem simplificada, usando um banco de dados NoSQL, mais especificamente o MongoDB. Nessa modelagem, existem três *collections* como pode ser observado na [Figura 14](#). A *collections* de exercícios foi utilizada apenas como base de consulta pelo sistema, não sendo possível adicionar ou remover registros nela. Para a versão desenvolvida, optou-se por restringir a criação de exercícios, a fim de evitar possíveis exercícios sem representação visual no aplicativo (GIF da execução do movimento).

Figura 14 – Modelagem do banco de dados do módulo de treinos



Fonte: Produzido pelos autores

Consequentemente, as funcionalidades do microserviço de treinos foram desenvolvidas e expostas por meio das seguintes APIs: Gerenciamento dos treinos do sistema (Tabela 14); Criação e visualização de equipamentos, permitindo obter diferentes visões sobre os equipamentos registrados no sistema (Tabela 15); e um *endpoint* para resgate dos exercícios presentes no sistema (Tabela 16).

Tabela 14 – *Endpoints* de treinos do microserviço de Treinos

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /workouts/v1/workout /create	Objeto com os dados do treino a ser criado	Criação de um treino.
GET /workouts/v1/workout	Identificador do usuário que criou o treino	Retorna os treinos criados de um usuário específico no sistema.
GET /workouts/v1/workout/{id}	Identificador do treino	Retorna as informações do treino especificado.
PUT /workouts/v1/workout/{id}	Identificador do treino e informações atualizadas dele	Atualiza as informações de um treino especificado.
DELETE /workouts/v1/workout/{id}	Identificador do treino	Remove o treino especificado do sistema.

Fonte: Produzido pelos autores

Tabela 15 – *Endpoints* de equipamentos do microserviço de Treinos

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /workouts/v1/equipment	Informação do equipamento a ser cadastrado	Adiciona o equipamento ao sistema.
GET /workouts/v1/equipment/all	-	Recupera todos os equipamentos registrados no sistema.
GET /workouts/v1/equipment/byId/{id}	Identificador do equipamento	Retorna as informações do equipamento especificado pelo seu identificador.
GET /workouts/v1/equipment/byName/{name}	Nome do equipamento	Retorna as informações do equipamento especificado pelo seu nome.

Fonte: Produzido pelos autores

Tabela 16 – *Endpoints* de exercícios do microserviço de Treinos

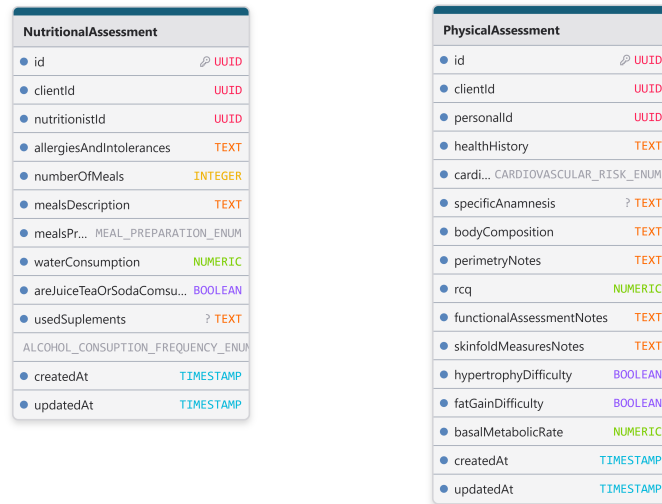
Rota HTTP	Parâmetros da Rota	Descrição da Operação
GET /workouts/v1/exercise	Lista de filtros para a busca (Identificador, grupo muscular, nome, equipamento ou tipo do exercício)	Recupera todos os exercícios que se enquadram nos filtros passados pelo usuário.

Fonte: Produzido pelos autores

5.3.4 Microserviço de Avaliações

O módulo de avaliações do sistema é responsável pelo gerenciamento de avaliações físicas e nutricionais. Neste microserviço, são implementadas as operações básicas de CRUD para os dois tipos de avaliação. Para a modelagem das entidades, foram consideradas as informações levantadas na [subseção 3.2.2](#) e a relação que as avaliações possuem com os usuários do aplicativo. Dessa forma, uma avaliação física deve estar associada a um *personal trainer* e a um cliente. Analogamente, a avaliação nutricional tem que estar associada a um nutricionista e a um cliente. Assim, o diagrama Entidade-Relacionamento do módulo é encontrado na [Figura 15](#). O SGBD utilizado para este microserviço foi o PostgreSQL.

Figura 15 – Modelagem do banco de dados de avaliações



Fonte: Produzido pelos autores

Com isso, foram desenvolvidos os *endpoints* responsáveis pelo gerenciamento das entidades do microserviço, o que resulta em um conjunto de APIs para o gerenciamento das avaliações físicas (Tabela 17) e um conjunto para o gerenciamento de clientes (Tabela 18).

Um ponto importante que foi considerado no desenvolvimento realizado foi a consistência dos dados entre o módulo de avaliações e o módulo de usuários. Uma vez que uma avaliação só pode existir se estiver associada a um cliente e a um colaborador, no processo de criação de uma nova avaliação, os identificadores do cliente e do colaborador recebidos na requisição são verificados no microserviço de usuários. Ou seja, caso sejam passados identificadores incorretos, que não correspondam a um usuário válido do sistema, o processo de criação da avaliação é interrompido.

Tabela 17 – *Endpoints* de avaliações físicas do microserviço de Avaliações

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /assessments/v1/physical-assessments	Objeto com os dados de uma avaliação física	Criação de uma avaliação física para um cliente.
GET /assessments/v1/physical-assessments	Identificador do cliente ou do personal	Retorna as avaliações físicas do sistema associadas a um cliente específico ou a um <i>personal trainer</i> .
GET /assessments/v1/physical-assessments/{id}	Identificador da avaliação física	Retorna as informações da avaliação física especificada.
PUT /assessments/v1/physical-assessments/{id}	Identificador da avaliação física	Atualiza as informações da avaliação física especificada.
DELETE /assessments/v1/physical-assessments/{id}	Identificador da avaliação física	Exclui a avaliação física especificada do sistema.

Fonte: Produzido pelos autores

Tabela 18 – *Endpoints* de avaliações nutricionais do microserviço de Avaliações

Rota HTTP	Parâmetros da Rota	Descrição da Operação
(POST) /assessments/v1/nutritional-assessments)	Objeto com os dados de uma avaliação nutricional	Criação de uma avaliação física para um cliente.
GET /assessments/v1/nutritional-assessments	Identificador do cliente ou do nutricionista	Retorna as avaliações nutricionais do sistema associadas a um cliente específico ou a um nutricionista.
GET /assessments/v1/nutritional-assessments/{id}	Identificador da avaliação nutricional	Retorna as informações da avaliação física especificada.
PUT /assessments/v1/nutritional-assessments/{id}	Identificador da avaliação nutricional	Atualiza as informações da avaliação física especificada.
DELETE /assessments/v1/nutritional-assessments/{id}	Identificador da avaliação nutricional	Exclui a avaliação física especificada do sistema.

Fonte: Produzido pelos autores

5.3.5 Microsserviço de IA

O módulo de Inteligência Artificial é responsável pela geração de treinos e planos alimentares personalizados, bem como pelo gerenciamento das recomendações produzidas.

Na modelagem deste microsserviço, foram definidos alguns pontos-chave para garantir a rastreabilidade das recomendações: é necessário registrar detalhadamente os dados de entrada que originaram cada sugestão gerada; o sistema deve armazenar o resultado final da recomendação, considerando possíveis alterações realizadas pelo cliente ou por um colaborador. Dessa forma, o sistema garante a transparência de todo o processo de geração das recomendações e permite a construção de uma base de conhecimento aprimorada, uma vez que as modificações feitas sobre uma sugestão por colaboradores e clientes indicam oportunidades de refinamento do modelo de IA utilizado. Com isso, fez-se o diagrama Entidade-Relacionamento deste módulo, o qual é evidenciado na [Figura 16](#). O SGBD utilizado foi o PostgreSQL.

Figura 16 – Modelagem do banco de dados de IA

DietAi	
id	UUID
clientId	UUID
input	JSONB
output	JSONB
editedOutput	JSONB
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

WorkoutAi	
id	UUID
clientId	UUID
input	JSONB
output	JSONB
editedOutput	JSONB
createdAt	TIMESTAMP
updatedAt	TIMESTAMP

Fonte: Produzido pelos autores

Com isso, foram desenvolvidos os *endpoints* responsáveis pelo gerenciamento das sugestões de planos alimentares ([Tabela 19](#)) e pelo gerenciamento das recomendações de treinos ([Tabela 20](#)) do microsserviço.

Tabela 19 – *Endpoints* dos planos alimentares personalizados do microserviço de IA

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /ai/v1/diets	Objeto com identificador do cliente e informações sobre o objetivo do plano alimentar a ser criado	Criação de um plano alimentar personalizado para o cliente.
GET /ai/v1/diets	Identificador do cliente	Retorna os planos alimentares personalizados de um cliente específico.
GET /ai/v1/diets/{id}	Identificador do plano alimentar personalizado	Recupera as informações do plano alimentar personalizado especificado.
PUT /ai/v1/diets/{id}	Identificador do plano alimentar personalizado	Atualiza as informações do plano alimentar personalizado especificado.

Fonte: Produzido pelos autores

Tabela 20 – *Endpoints* dos treinos personalizados do microserviço de IA

Rota HTTP	Parâmetros da Rota	Descrição da Operação
POST /ai/v1/workouts	Objeto com identificador do cliente e informações sobre o objetivo do treino a ser criado	Criação de um treino personalizado para o cliente.
GET /ai/v1/workouts	Identificador do cliente	Retorna os treinos personalizados de um cliente específico.
GET /ai/v1/workouts/{id}	Identificador do treino personalizado	Recupera as informações do treino personalizado especificado.
PUT /ai/v1/workouts/{id}	Identificador do treino personalizado	Atualiza as informações do treino personalizado especificado.

Fonte: Produzido pelos autores

A escolha do modelo de Inteligência Artificial utilizado para a geração de treinos e planos alimentares personalizados foi motivada por razões técnicas e questões de viabilidade prática do projeto. Inicialmente, avaliou-se a possibilidade de desenvolver um modelo próprio, treinado especificamente para o contexto do sistema. No entanto, essa abordagem exige uma grande quantidade de dados provenientes de duas áreas de conhe-

cimento distintas: nutrição e atividade física, além de demandar um processo complexo de categorização e tratamento dos parâmetros envolvidos. Por isso, optou-se por utilizar um modelo já treinado.

Para a seleção do modelo pré-treinado, optou-se pela utilização de LLMs, devido à sua flexibilidade na interpretação e no processamento dos dados de entrada. Diferentemente de outros tipos de modelos de IA, as LLMs são capazes de compreender informações textuais não estruturadas, permitindo que as diversas informações relevantes para um treino ou plano alimentar, como objetivos do usuário, restrições alimentares e preferências de treino, sejam consideradas na geração das recomendações. Além disso, a escolha por LLMs tem a vantagem da existência de diversas soluções que facilitam a integração com provedores consolidados, os quais disponibilizam modelos robustos e capazes de produzir respostas adequadas para o problema.

Embora as LLMs sejam modelos de propósito geral e, portanto, nem sempre forneçam recomendações com precisão absoluta, tal limitação é mitigada pela proposta de um sistema colaborativo, com a presença de profissionais, como *personal trainers* e nutricionistas, que podem revisar e ajustar as sugestões geradas automaticamente, além de o próprio cliente ser capaz de fazer as alterações que julgar necessárias. Além disso, para aumentar a precisão e a contextualização das recomendações geradas, emprega-se a técnica de RAG, que insere dados específicos do cliente armazenados no sistema no contexto do processamento da LLM. Essa abordagem permite que as respostas sejam baseadas em informações atualizadas, reduzindo o risco de alucinações e tornando as recomendações mais coerentes com o perfil e o histórico individual de cada usuário.

Conforme descrito na [subseção 5.2.2](#), a LLM escolhida foi o Gemini da Google, usando como ferramenta de integração com o modelo o *framework* LangChain.

Dada a importância da geração de recomendações personalizadas com IA no sistema, destaca-se o processo realizado para gerar isso no aplicativo. O processo de geração de plano alimentar pode ser resumido nos seguintes passos:

1. Requisição com dados de anamnese geral sobre o plano alimentar a ser gerado no microsserviço de IA.
2. Microsserviço de IA faz requisições paralelas para recuperar informações a serem usadas na geração do plano: uma para recuperar as informações do cliente no microsserviço de usuários e outra para recuperar os ingredientes e refeições disponíveis no microsserviço de planos alimentares.
3. Com as informações coletadas, usa-se o Langchain para a construção do *prompt* com as instruções sobre a tarefa a ser realizada em conjunto com os dados resgatados, técnica de RAG, e para fazer a chamada para a LLM com o *prompt* construído.

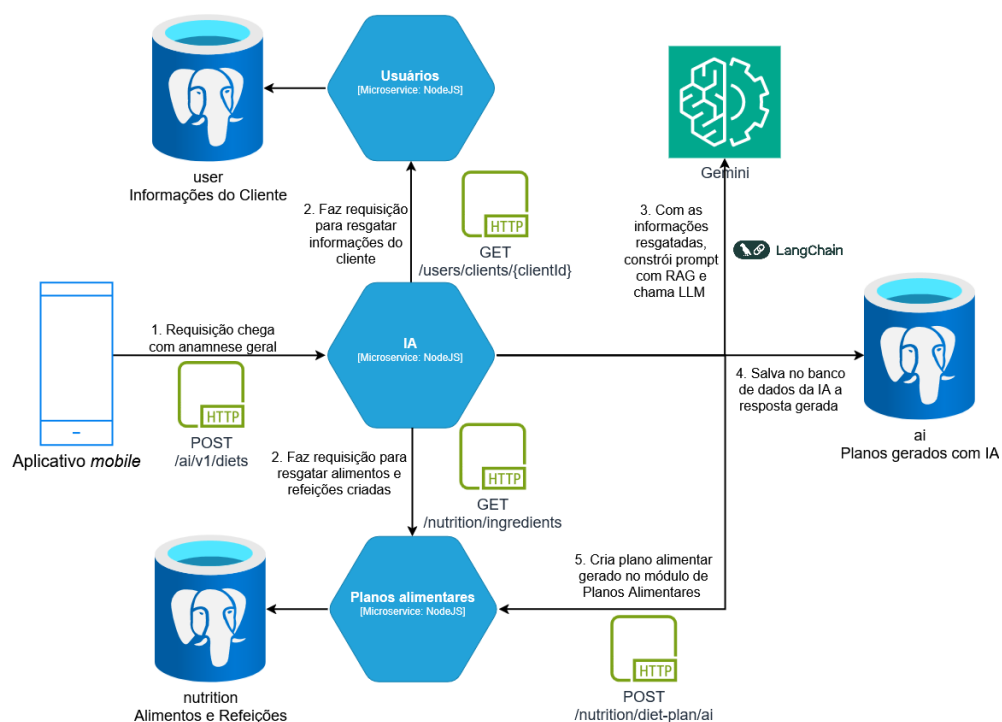
4. Quando recebe a resposta da LLM, o microserviço da IA armazena a resposta gerada juntamente com os dados de entrada em sua base de dados.
5. Por fim, faz uma requisição para o microserviço de planos alimentares a fim de cadastrar a recomendação gerada para o cliente.

O processo de gerar treinos personalizados é semelhante ao anterior, podendo ser descrito conforme apresentado abaixo:

1. Requisição com dados de anamnese geral sobre o treino a ser gerado no microserviço de IA.
2. Microserviço de IA faz requisições paralelas para recuperar informações a serem usadas na geração do plano: uma para recuperar as informações do cliente no microserviço de usuários e outra para recuperar os exercícios com equipamentos disponíveis no microserviço de treinos.
3. Com as informações coletadas, usa-se o Langchain para a construção do *prompt* com as instruções sobre a tarefa a ser realizada, em conjunto com os dados resgatados, técnica de RAG, e para fazer a chamada para a LLM com o *prompt* construído.
4. Quando recebe a resposta da LLM, o microserviço da IA armazena a resposta gerada juntamente com os dados de entrada em sua base de dados.
5. Por fim, faz uma requisição para o microserviço de treinos a fim de cadastrar a recomendação gerada para o cliente.

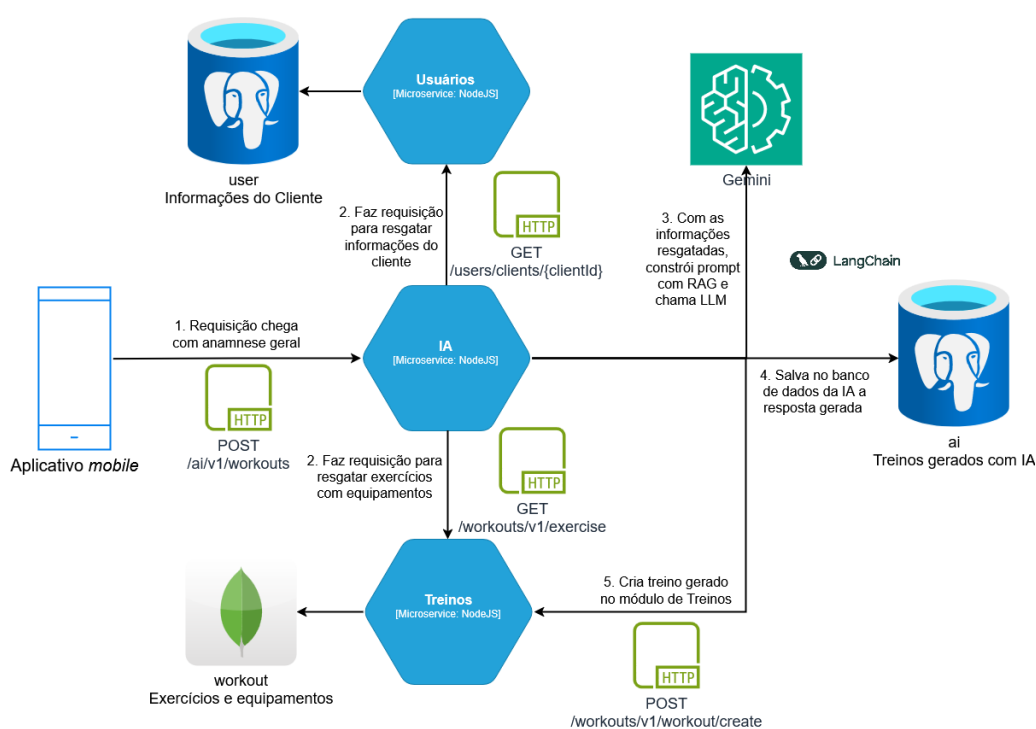
De forma a facilitar o entendimento desses fluxos, foram elaborados os diagramas exibidos na [Figura 17](#) e na [Figura 18](#), que detalham o processo de geração de um plano alimentar e de um treino, respectivamente.

Figura 17 – Geração de planos alimentares personalizados



Fonte: Produzido pelos autores

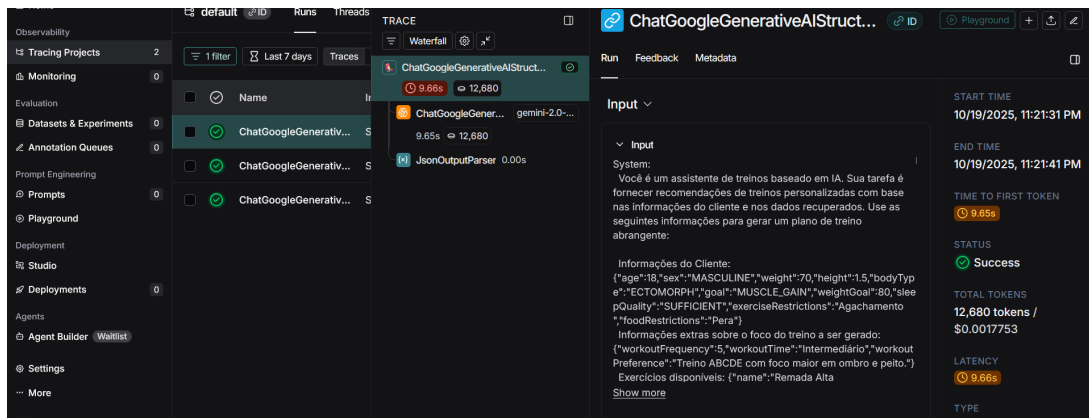
Figura 18 – Geração de treinos personalizados



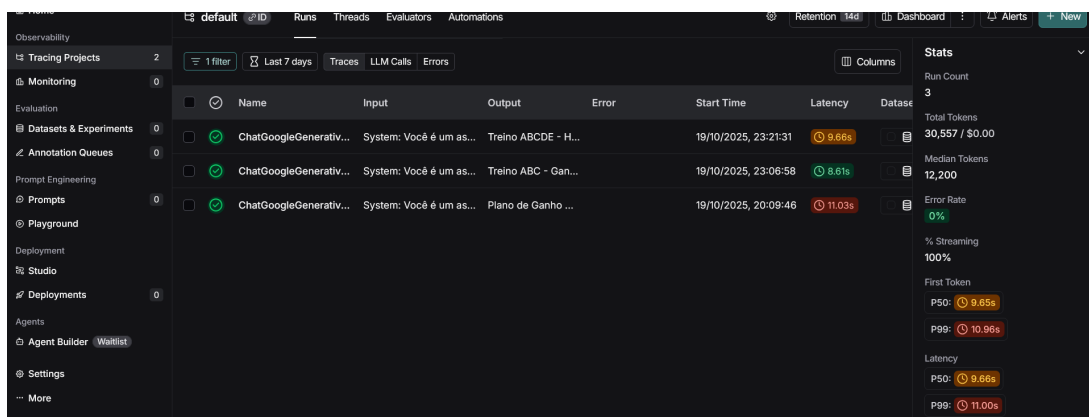
Fonte: Produzido pelos autores

Por fim, para demonstrar como o monitoramento das interações com a LLM é feito pela ferramenta do LangSmith, exibe-se na [Figura 19a](#) um exemplo de chamada feita para a LLM, na [Figura 19b](#) a visualização da lista de *traces* presentes no período de tempo selecionado e na [Figura 19c](#) a seção de *dashboards* para custos e uso de *tokens* da solução. Vale ressaltar que isso é apenas uma parte das visualizações existentes na ferramenta.

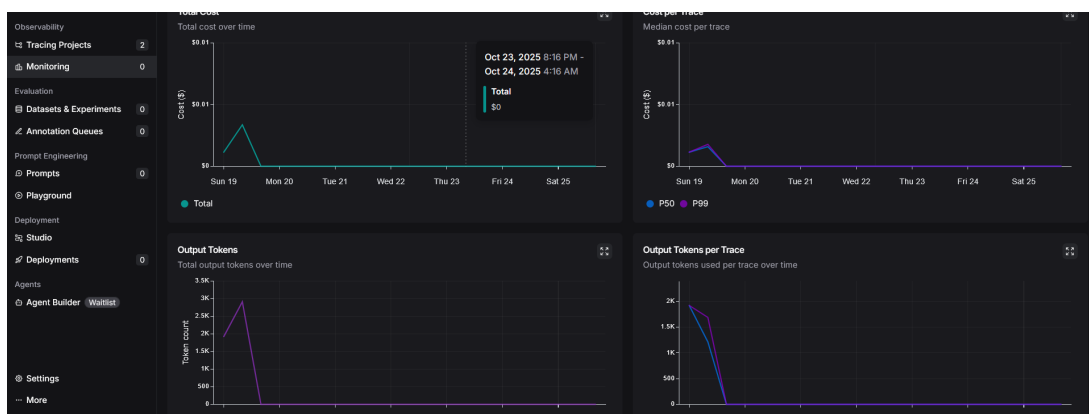
Figura 19 – Monitoramento das interações com LLM pelo LangSmith



(a) *Trace* específico de uma chamada para a LLM



(b) Lista de *traces*



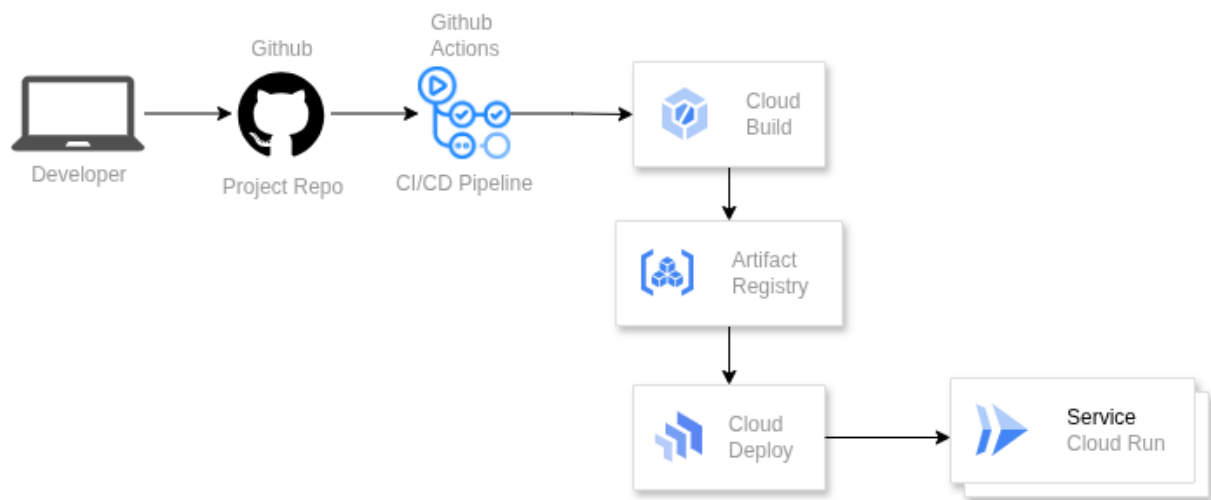
(c) *Dashboard* de custos e uso de tokens

Fonte: Produzido pelos autores

5.4 Deploy do projeto

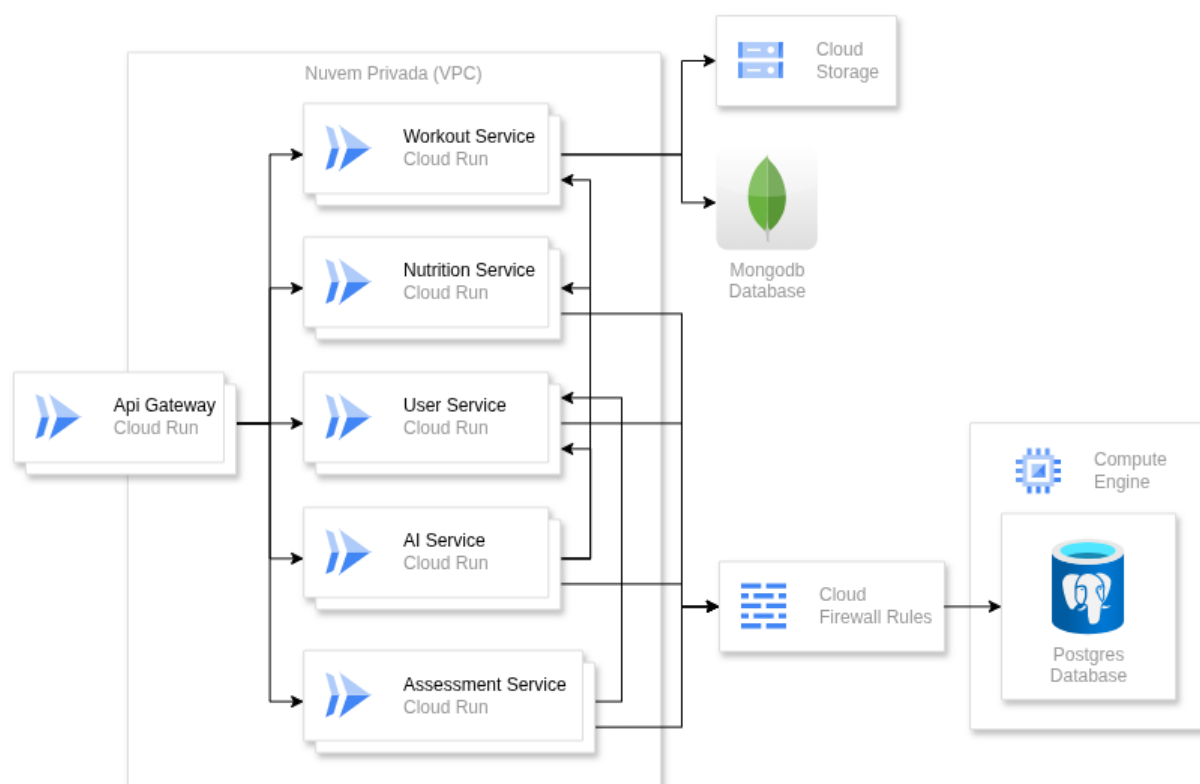
Como pode ser observado na Figura 20, o processo de *deploy* do *backend* do projeto consiste em um *pipeline* automatizado através do uso da ferramenta *Github Actions*. Por meio dessa ferramenta é possível definir gatilhos que ao serem acionados iniciam uma sequência de ações em cadeia que permitem disponibilizar uma aplicação/serviço na internet. No projeto em questão, o gatilho definido foi um *merge commit* na ramificação *main*, que ao ser acionado inicia o processo de *build* do serviço através de um *Dockerfile*. Logo após a imagem do *container docker* ser gerada, ela é registrada e utilizada para instanciar um novo *container* do serviço com as atualizações presentes no *merge commit*.

Figura 20 – Pipeline de *deploy* do *backend*



Fonte: Produzido pelos autores

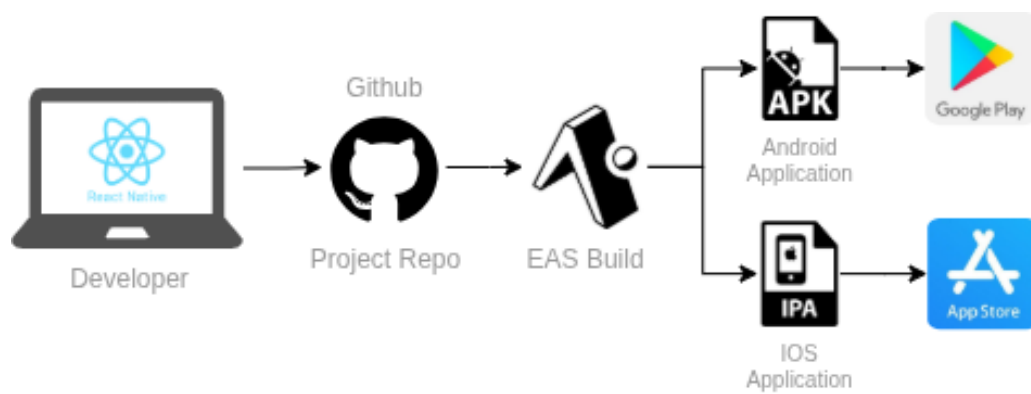
Seguindo a especificação da arquitetura do projeto mencionado na Seção 4.4 foi criado um *pipeline* de *deploy* para cada um dos serviços. Assim, realizado o *deploy* de todos os serviços do projeto temos uma infraestrutura tal qual a Figura 21. Além disso, na mesma figura, é possível notar a integração das tecnologias de armazenamento de dados na solução, sendo elas o *Cloud Storage* para o armazenamento dos *Gifs* do módulo de treinos, o *MongoDB* para guardar os dados ainda do mesmo módulo e o *PostgreSQL* para armazenar os dados dos demais módulos.

Figura 21 – Arquitetura do *deploy* na nuvem

Fonte: Produzido pelos autores

Enquanto isso, o processo de deploy do aplicativo mobile, como é possível de ver na Figura 22, começa com o desenvolvimento no *framework react native* que é então versionado em um repositório hospedado no *github*. A partir desse repositório, o serviço EAS Build é acionado para realizar a etapa de build automatizado, gerando os artefatos nativos correspondentes: um pacote APK para a versão Android e um pacote IPA para a versão iOS. Por fim, esses binários são utilizados para publicação nas respectivas lojas, Google Play e App Store, concluindo o fluxo de entrega da aplicação aos usuários finais de forma padronizada e reproduzível.

Figura 22 – Pipeline de deploy do frontend



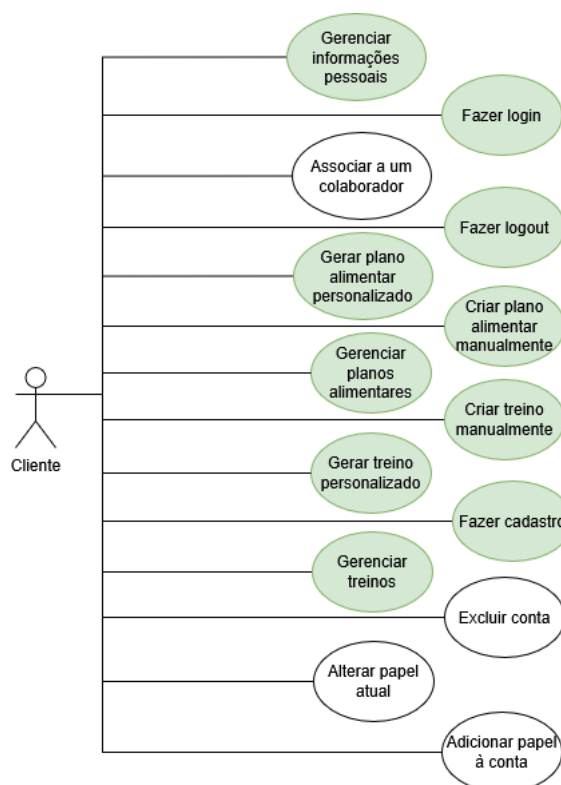
Fonte: Produzido pelos autores

5.5 Resultados do Desenvolvimento

Como resultado prático do projeto, foi possível disponibilizar uma versão beta da plataforma, a qual contém os casos de uso destacados na [Figura 23](#), [Figura 24](#) e [Figura 25](#). Nesta versão, foram priorizados os casos de uso principais para o cliente, como o gerenciamento e a geração de recomendações de treinos e planos alimentares, além da visualização de avaliações criadas para ele. Com isso, tomando como base a prototipação das telas realizadas no Figma, foram desenvolvidas as telas para o perfil do cliente. Os tópicos a seguir abordam de forma detalhada cada uma das seções implementadas e suas funcionalidades.

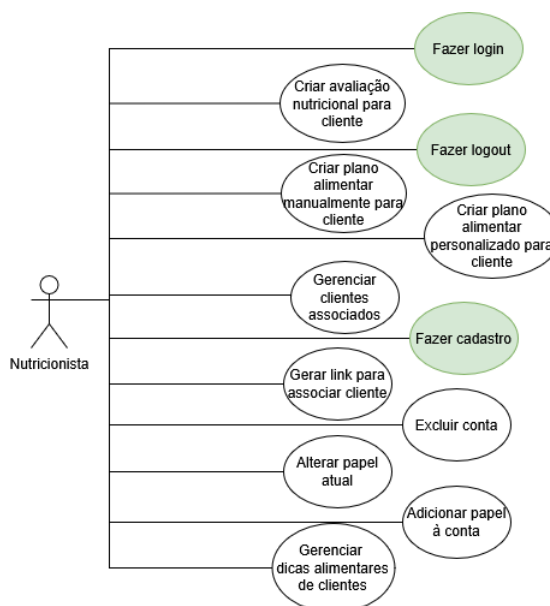
Além da interface construída, são exibidos os registros correspondentes gerados no banco de dados. Dessa forma, cada tela apresentada é acompanhada da representação de seus dados persistidos, atestando o funcionamento adequado do sistema.

Figura 23 – Casos de uso implementados do cliente

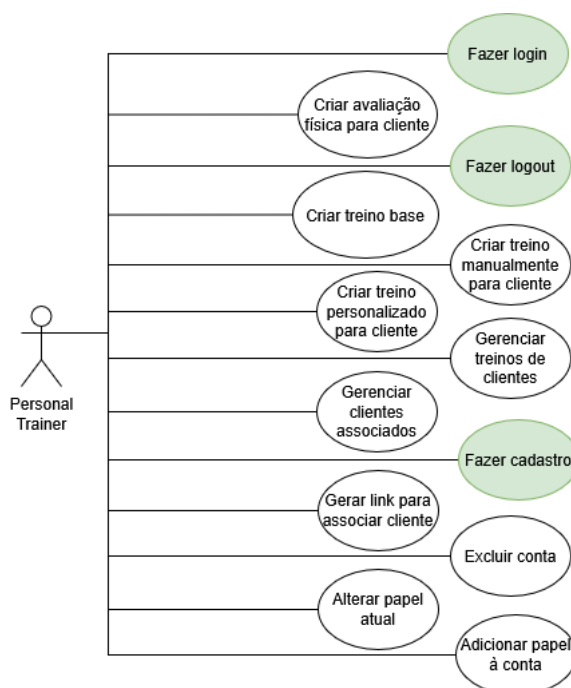


Fonte: Produzido pelos autores

Figura 24 – Casos de uso implementados do nutricionista



Fonte: Produzido pelos autores

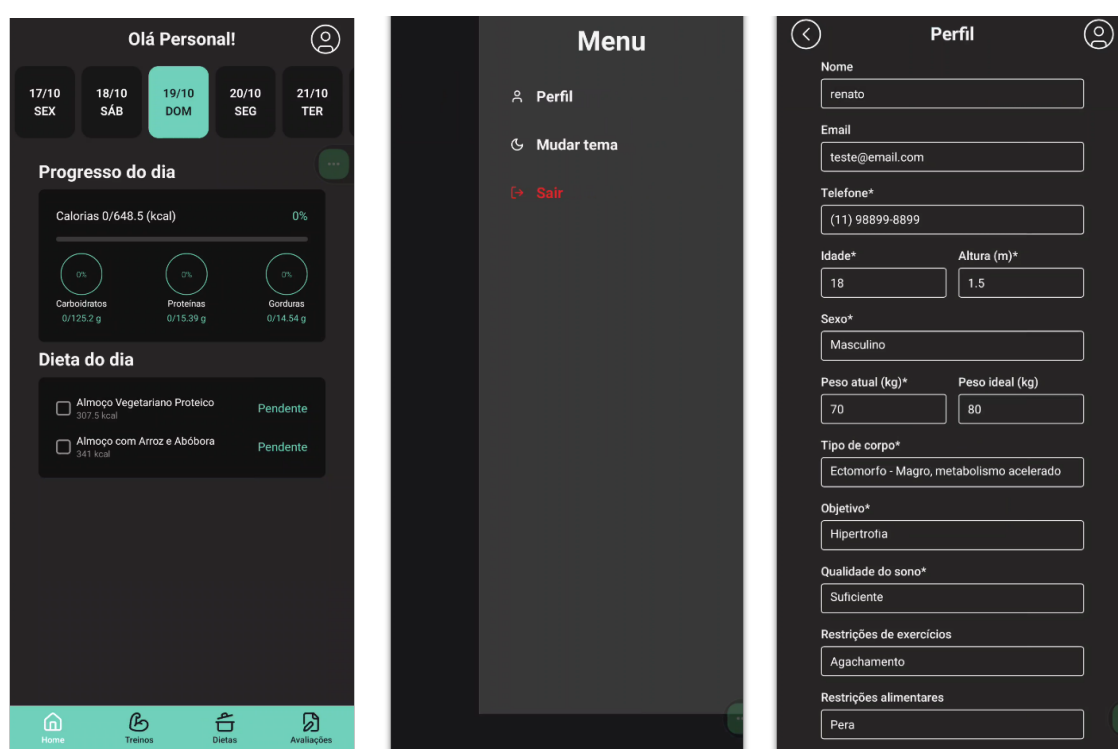
Figura 25 – Casos de uso implementados do *personal trainer*

Fonte: Produzido pelos autores

5.5.1 Seção *home*

A tela *home* (Figura 26a) é a tela inicial do app quando o usuário já está logado. Nessa tela é possível acompanhar o progresso do plano alimentar atual de forma interativa e visualizar o progresso dos dias anteriores e posteriores ao dia atual. A partir da tela inicial é possível acessar o menu da aplicação (Figura 26b) clicando no botão localizado no canto superior direito. Acessando o menu é possível ir para a tela de dados do usuário (Figura 26c), alterar o tema ou fazer o *logout* do aplicativo. Os registros associados às informações dispostas nas telas encontram-se na Figura 27.

Figura 26 – Telas da seção *home* e menu

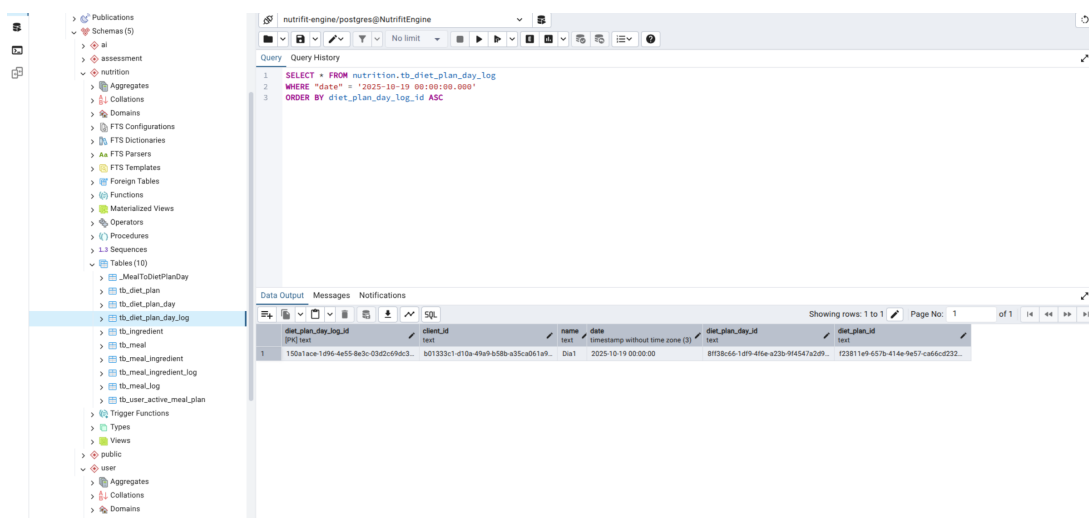


(a) Tela inicial após o desenvolvimento

(b) Tela de menu

(c) Tela de dados do usuário

Fonte: Produzido pelos autores

Figura 27 – Registros no banco de dados com as informações da *home* e menu


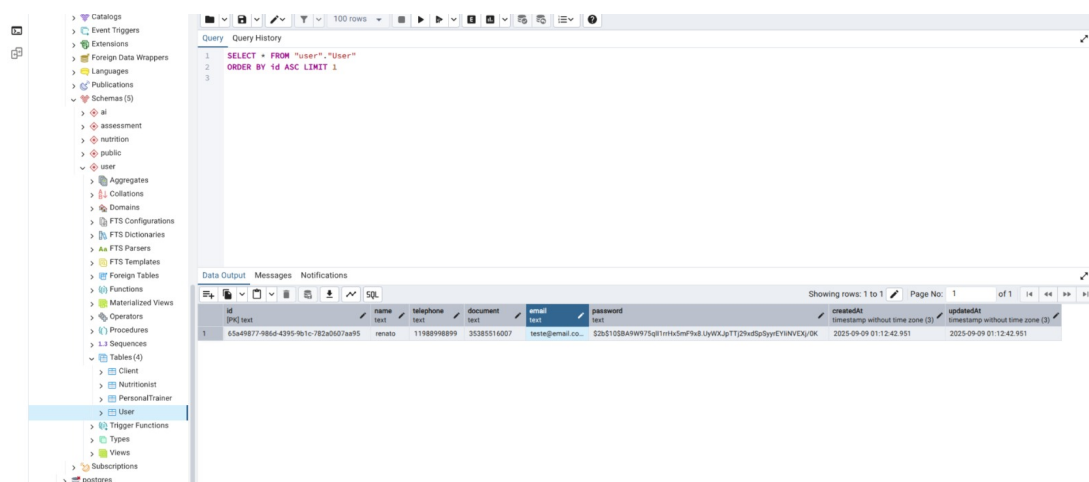
Query History

```
1 SELECT * FROM nutrition.tb_diet_plan_day_log
2 WHERE "date" = '2025-10-19 00:00:00.000'
3 ORDER BY diet_plan_day_log_id ASC
```

Data Output Messages Notifications

id	name	date	diet_plan_day_id
1	100a1ace-1d94-4e55-8a3c-03d2cd946c3	2025-10-19 00:00:00	8073b6d6-1d94-4f5e-a230-9b4547a2d9f

(a) Registro do progresso diário do cliente



Query History

```
1 SELECT * FROM "user"."User"
2 ORDER BY id ASC LIMIT 1
```

Data Output Messages Notifications

id	name	telephone	document	email	password	createdAt	updatedAt
1	65a49877-986d-4395-961c-782a0607aef5	renato	11988998899	35385516007	teste@email.co...	2025-09-09 01:12:42.951	2025-09-09 01:12:42.951

(b) Registro das informações cadastrais do cliente

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (5)

ai

assessment

nutrition

publico

user

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (4)

Client

Nutritionist

PersonalTrainer

User

Trigger Functions

Types

Views

Subscriptions

notices

No limit

Query

Query History

1

2

SELECT *

FROM "user"."Client"

ORDER BY id ASC

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1

of 1

id	age	sex	weight	height	bodyType	goal	weightGoal	sleepQuality	exerciseRestrictions	foodRestrictions	personalTrainerId	nutritionistId	userId	
1	601333c1-d10a-49a9-b58b-a35ca011a925	18	MASCUL	70	1.5	ECTOMORPH	MUSCLE_GAIN	80	SUFFICIENT	Agachamento	Pera	[null]	[null]	65a49877-986d-4395-961c-782a0607aef5

(c) Registro das informações específicas do cliente

Fonte: Produzido pelos autores

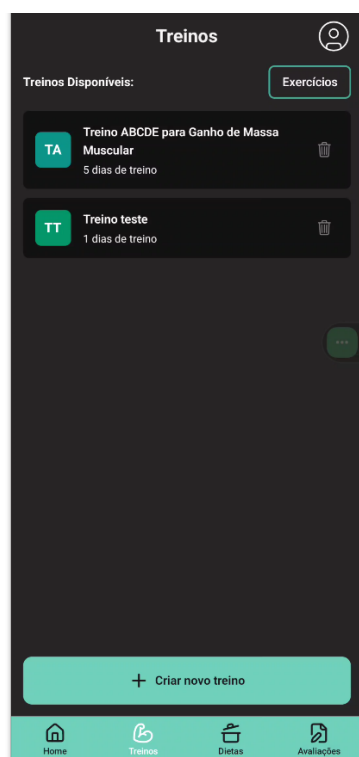
5.5.2 Seção de treinos

A seção de treinos pode ser acessado ao clicar na aba treinos no inferior da tela. Nesse fluxo é possível criar, visualizar, editar ou excluir treinos. Ao entrar no fluxo, a primeira tela a ser apresentada ao usuário é a listagem dos treinos criados anteriormente (Figura 28a). Dessa tela é possível ir para outras três telas ainda no mesmo fluxo, a tela de listagem de exercícios (Figura 28b) na qual é possível consultar uma variedade de exercícios e ir para a tela de detalhes do exercício consultado (Figura 28c). Se, na tela de listagem de treinos for clicado em algum dos treinos é apresentada a tela de detalhes do treino (Figura 29a), nela é possível consultar os exercícios de cada dia de treino, além de séries, repetições, descanso e carga de cada um deles. Também é possível editar o treino caso seja necessário.

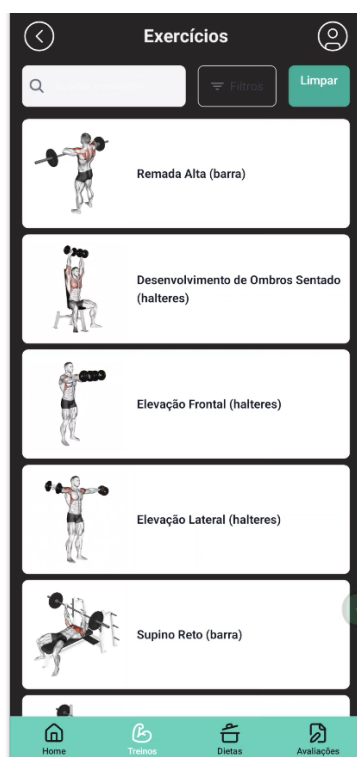
Clicando em **Criar novo treino** é apresentado ao usuário um modal (Figura 29b) no qual é possível escolher entre uma criação de treinos manual ou com o auxílio de um LLM. Na criação manual é necessário adicionar cada um dos dias de treino, exercícios, series, repetições e cargas de forma manual. A Figura 29c mostra como é a tela de criação de treinos manual ainda sem nenhum exercício adicionado. Já a tela presente na Figura 29b traz a visão do usuário ao optar por uma criação de treino com o auxílio de um LLM, nessa opção é necessário fornecer algumas informações que serão fornecidas ao modelo para auxiliar na criação do treino personalizado, dentre elas o nível de treinamento no qual o usuário se encontra (exemplo: iniciante, intermediário ou avançado), a quantidade de dias que o plano de treino deve ter e o estilo de treino na qual é possível adicionar uma observação qualquer para alinhar o treino gerado com a preferência do usuário.

Por fim, são exibidos na Figura 30 os registros nos bancos de dados de alguns treinos, tanto na *collection* existente no microserviço de Treino, quanto na tabela do microserviço da IA, no caso de treinos personalizados.

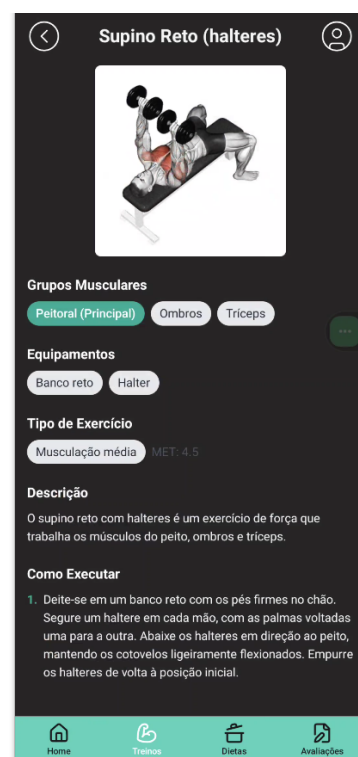
Figura 28 – Telas da seção de treinos - Parte 1



(a) Tela de listagem de treinos



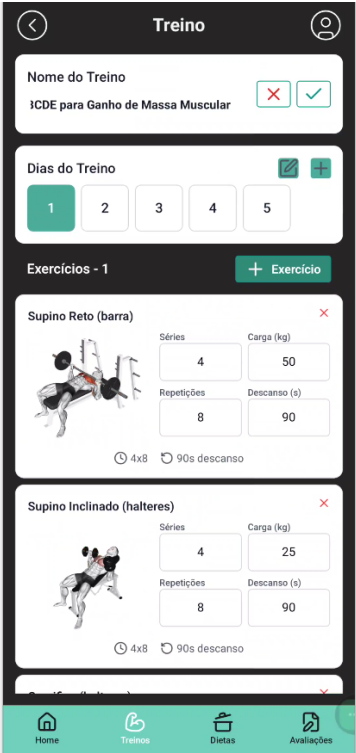
(b) Tela de listagem de exercícios



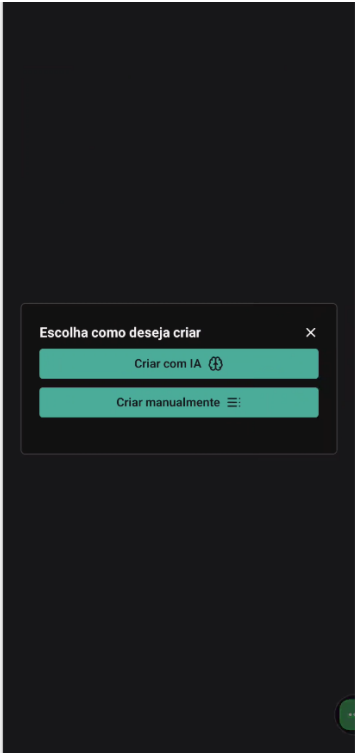
(c) Tela de detalhes do exercício

Fonte: Produzido pelos autores

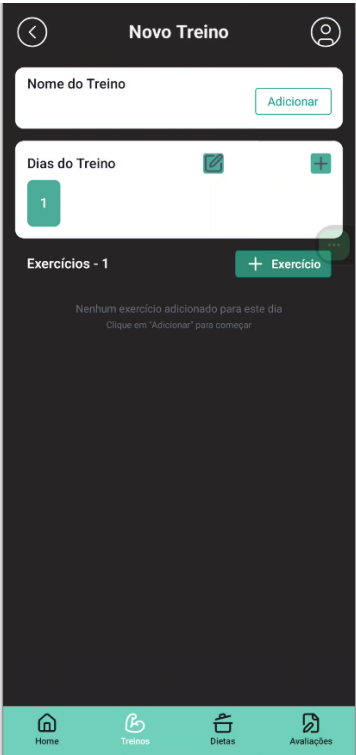
Figura 29 – Telas da seção de treinos - Parte 2



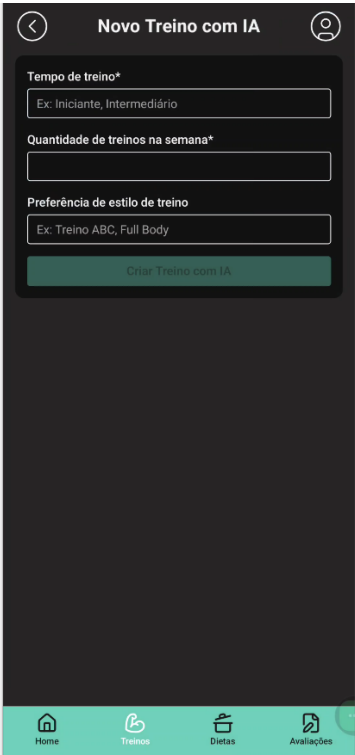
(a) Tela de detalhes do treino



(b) Tela de escolha de criação de novo treino



(c) Tela de criação de treino manual



(d) Tela de criação de treinos com LLM

Fonte: Produzido pelos autores

Figura 30 – Registros no banco de dados com as informações dos treinos

	id	clientid	input	output	editedOutput	createdAt
1	938743ee-3a	3fa85f64-5717-45	["clientid": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "workout": {"name": "Treino de Hipertrofia para Iniciantes (2 dias)", "workout_days": [{"day": 1, "exercise": "Agachamento", "sets": 3, "reps": 10}, {"day": 2, "exercise": "Flexão de Braços", "sets": 3, "reps": 10}], "created_by": "3fa85f64-5717-4562-b3fc-2c963f66afa6"}]			2025-10-01 13:53:08
2	3b8091f0-0d	ab2a371f-dc0e-4	["clientid": "ab2a371f-dc0e-4457-9e43-ad18924e77e5", "workout": {"name": "Treino de Ganho de Massa Muscular - Iniciante", "workout_days": [{"day": 1, "exercise": "Agachamento", "sets": 3, "reps": 10}, {"day": 2, "exercise": "Flexão de Braços", "sets": 3, "reps": 10}], "created_by": "ab2a371f-dc0e-4457-9e43-ad18924e77e5"}]			2025-10-01 22:47:25

(a) Registros de treinos personalizados no módulo de IA

```
id : "22fbcaeb-b7a6-4f8a-b004-47597838d8d2"
name : "Treino de Hipertrofia para Iniciantes (2 dias)"
created_at : "2025-10-01T10:53:07.161-03:00"
updated_at : "2025-10-01T10:53:07.161-03:00"
created_by : "3fa85f64-5717-4562-b3fc-2c963f66afa6"
workout_days : Array (2)
  __v : 0

_id: ObjectId('68ddaf7df60958e8b77c9cde')
id : "4cda493c-adeb-45d5-b43a-0c990e3216ff"
```

(b) Registro dos treinos criados na *collection* do módulo de Treino

Fonte: Produzido pelos autores

5.5.3 Seção de dietas

A seção de dietas pode ser acessada clicando na aba de dietas na parte inferior da tela. A tela de entrada dessa seção é a tela de listagem de planos alimentares (Figura 31a) na qual são exibidos todos os planos alimentares previamente criados pelo usuário, mostrando informações como o nome do plano e a data de criação. A partir desta tela inicial, é possível acessar os planos existentes para visualização e edição, ou criar novos planos alimentares através do botão **Criar novo plano**. Ao selecionar um plano alimentar existente, o usuário é direcionado para a tela de dias do plano alimentar (Figura 31e), onde pode visualizar a estrutura completa do plano organizada por dias, permitindo uma visão geral da distribuição das refeições ao longo do período estabelecido.

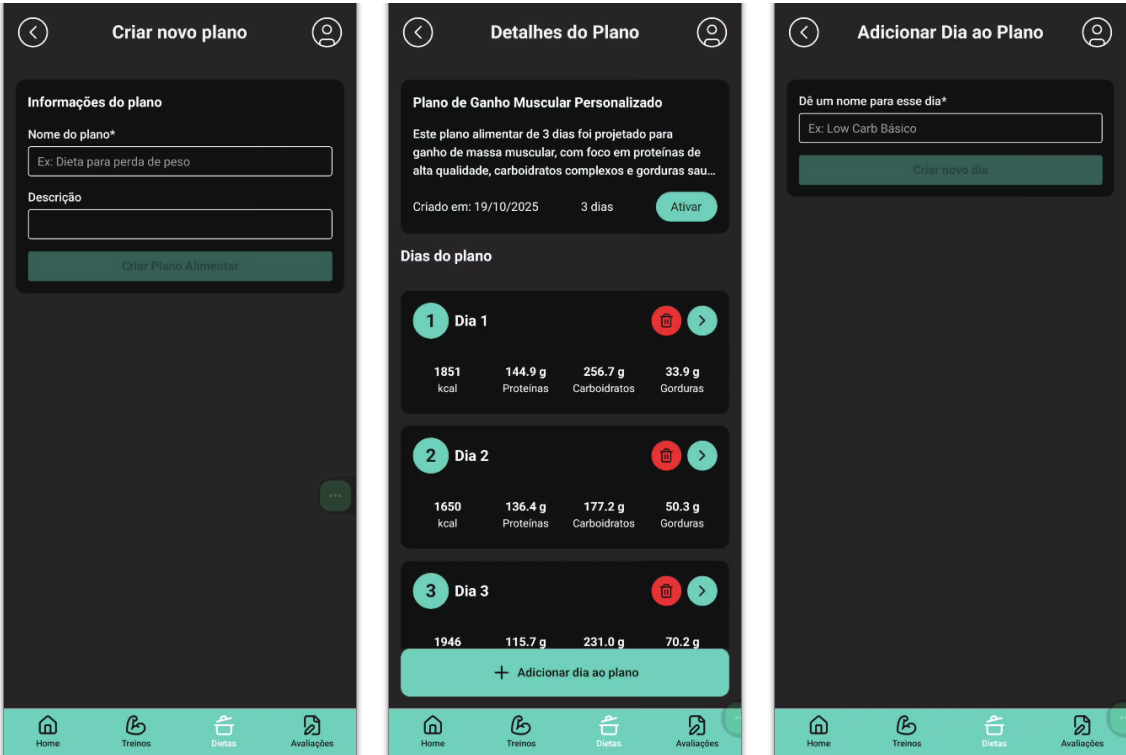
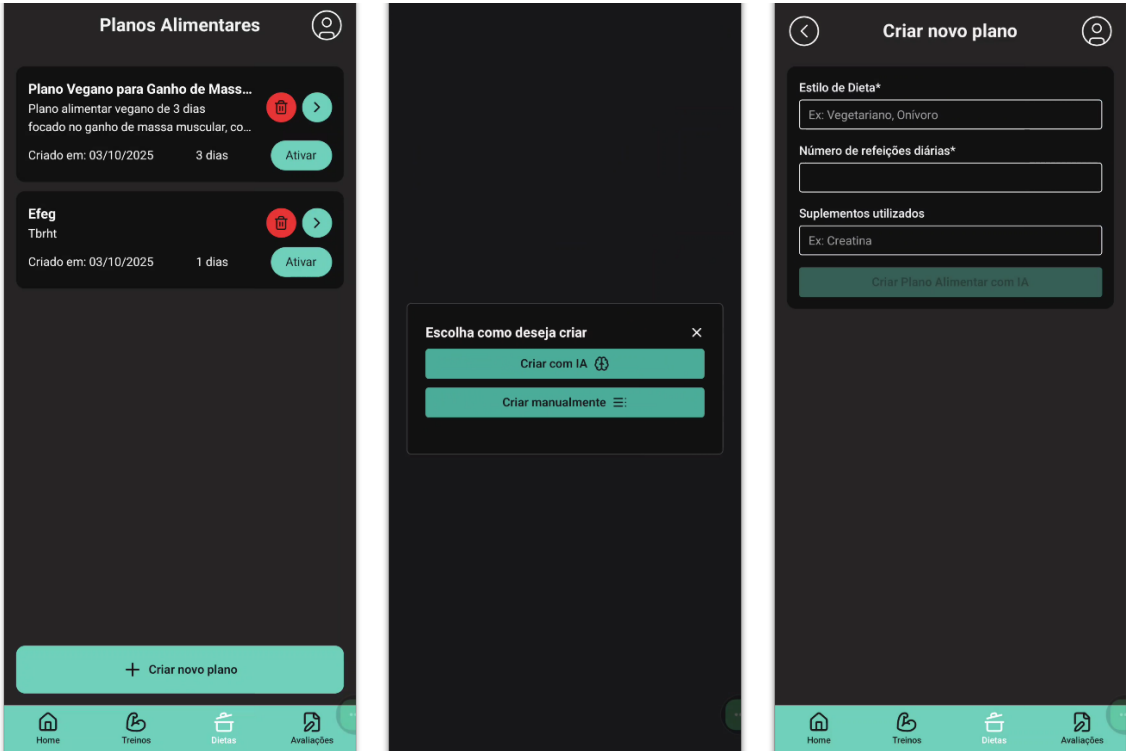
Ao optar por criar um novo plano alimentar, é apresentado ao usuário um modal de escolha (Figura 31b) que oferece duas opções de criação: manual ou assistida por LLM. Na criação assistida por inteligência artificial (Figura 31c), o usuário deve fornecer informações como estilo da dieta (por exemplo: vegetariana, vegana, focada em emagrecimento,

etc), número de refeições diárias e suplementos utilizados, permitindo que o modelo gere um plano alimentar personalizado de acordo com suas necessidades específicas. Já na opção de criação manual ([Figura 31d](#)), o usuário tem controle total sobre a estruturação do plano, podendo definir o nome do plano, adicionar uma descrição e construir cada dia de forma individualizada através da interface apresentada.

O gerenciamento detalhado das refeições é realizado através de uma estrutura hierárquica que permite ao usuário adicionar dias ao plano ([Figura 31f](#)), incluir refeições em cada dia ([Figura 32a](#) e [Figura 32b](#)), e gerenciar os ingredientes de cada refeição ([Figura 32c](#)). Para cada refeição, é possível visualizar informações nutricionais detalhadas como calorias totais e distribuição de macronutrientes (proteínas, carboidratos e gorduras). O sistema também permite a edição de ingredientes existentes ([Figura 32d](#)), adição de novos ingredientes a uma refeição ([Figura 32e](#)) e até mesmo a criação de ingredientes personalizados ([Figura 32f](#)), oferecendo flexibilidade completa na composição nutricional de cada plano alimentar.

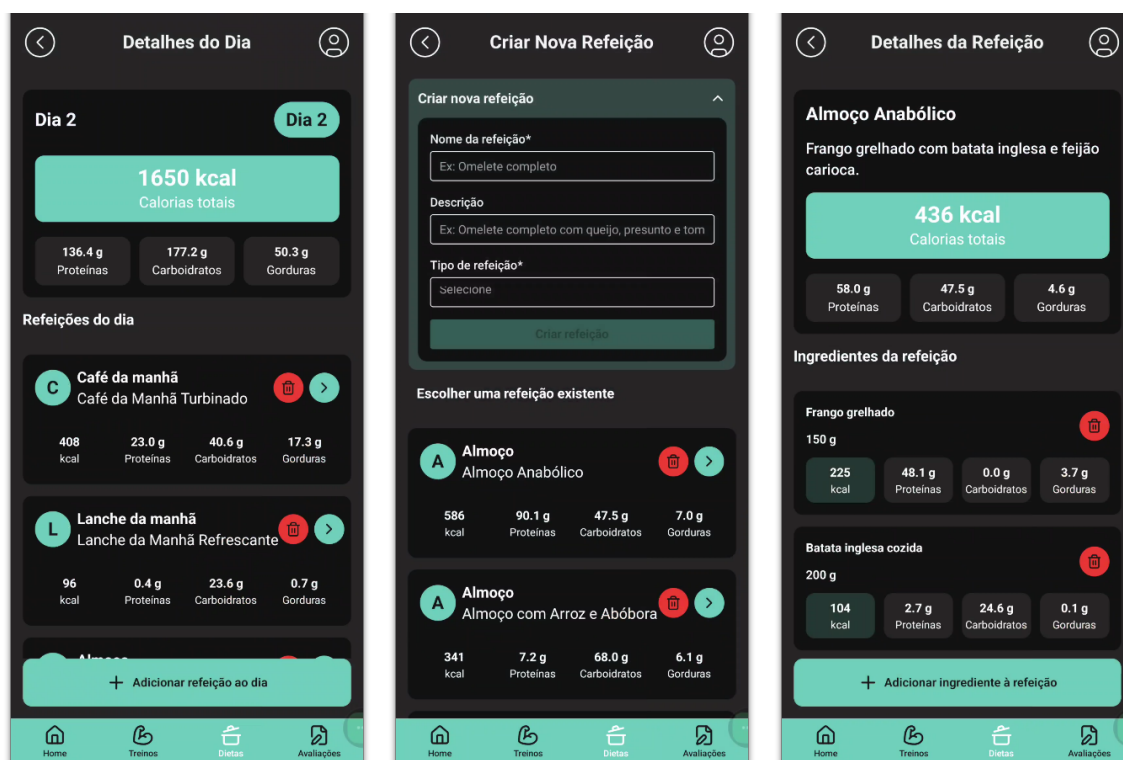
Por fim, são exibidos na [Figura 33](#) os registros nos bancos de dados de alguns planos alimentares, tanto nas tabelas existentes no microserviço de Plano Alimentar, quanto na tabela do microserviço da IA, no caso de planos alimentares personalizados.

Figura 31 – Telas da seção de planos alimentares - Parte 1

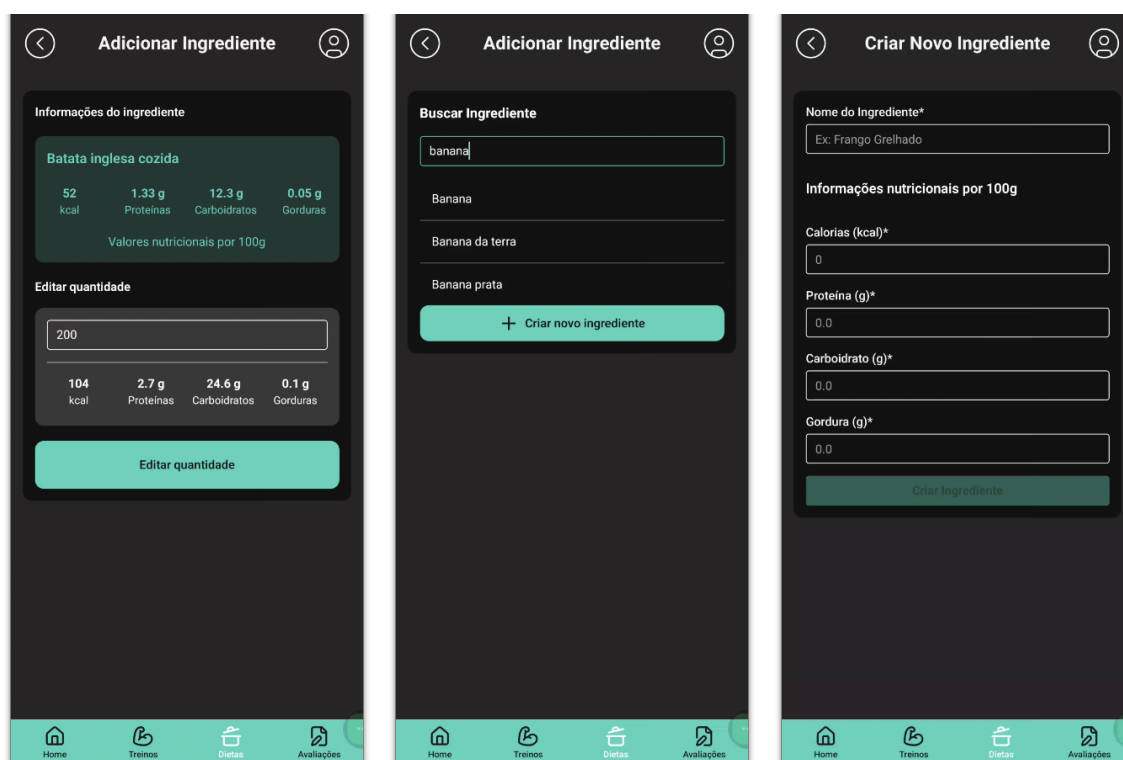


Fonte: Produzido pelos autores

Figura 32 – Telas da seção de planos alimentares - Parte 2



- (a) Tela de refeições de um dia (b) Tela para adicionar uma (c) Tela de ingredientes de
do plano alimentar refeição a um dia do plano uma refeição
alimentar



- (d) Tela para editar um ingre- (e) Tela para adicionar um in- (f) Tela para criar um novo in-
diente de uma refeição grediente a uma refeição grediente

Fonte: Produzido pelos autores

Figura 33 – Registros no banco de dados com as informações dos planos alimentares

	id	clientid	input	output	editedOutput	createdAt	updatedAt
1	15430d26-0b56	e6b1ee66-52d5-46	["clientid": "e6b1ee66-52d5-46a1-8b8b-b557af90f9ba", {"name": "Plano Alimentar para Ganho de Massa Muscular", "description": null}]			2025-09-24 01:14:07.223	2025-09-24 01:14:07.223
2	833af8f9-3ee3-4	e6b1ee66-52d5-46	["clientid": "e6b1ee66-52d5-46a1-8b8b-b557af90f9ba", {"name": "Plano de Ganho Muscular - Guilherme", "description": null}]			2025-09-24 02:05:54.488	2025-09-24 02:05:54.488
3	bc35ef54-a5ea-4	e6b1ee66-52d5-46	["clientid": "e6b1ee66-52d5-46a1-8b8b-b557af90f9ba", {"name": "Plano Alimentar para Ganho de Massa Muscular", "description": null}]			2025-09-24 02:12:32.593	2025-09-24 02:12:32.593
4	dc4a53ca-b5f3-4	e6b1ee66-52d5-46	["clientid": "e6b1ee66-52d5-46a1-8b8b-b557af90f9ba", {"name": "Plano de Ganho Muscular - Guilherme", "description": null}]			2025-09-24 02:59:31.085	2025-09-24 02:59:31.085
5	84b1a0c2-43f8-4	e6b1ee66-52d5-46	["clientid": "e6b1ee66-52d5-46a1-8b8b-b557af90f9ba", {"name": "Plano Alimentar para Ganho de Massa Muscular", "description": null}]			2025-09-24 03:08:25.385	2025-09-24 03:08:25.385

(a) Registros de planos alimentares personalizados no módulo de IA

	diet_plan_id	client_id	name	description	created_at	updated_at	created_by	updated_by
1	15a137b0-0cd1-48cc-b01333c1-d10a-4		Plano de Ganho Musc	Este plano alimentar de	2025-10-19 23:09:57.959	2025-10-19 23:09:57.959	ai-generated	ai-generated
2	45a4b1c1-cfa7-4d4c-b01333c1-d10a-4		Plano Vegano para Ga	Plano alimentar vegano	2025-10-03 14:24:41.469	2025-10-03 14:24:41.469	ai-generated	ai-generated
3	56fb15a0-b4c7-41ce-e6b1ee66-52d5-4		Plano Alimentar para f	Plano alimentar de 3 dia	2025-09-24 03:08:22.720	2025-09-24 03:08:22.720	ai-generated	ai-generated
4	55301205-e50d-49fb-0c601e70-94d7-4		Dieta 2		2025-09-18 22:42:14.079	2025-09-18 22:42:14.079	0c601e70-94d7-4a7a-826d-abb0172ed84d	0c601e70-94d7-4a7a-826d-abb0172ed84d
5	f269605-e14b-4cf4-1-0c601e70-94d7-4		Dieta 1	Descrição	2025-09-18 22:35:14.315	2025-09-18 22:35:14.315	0c601e70-94d7-4a7a-826d-abb0172ed84d	0c601e70-94d7-4a7a-826d-abb0172ed84d

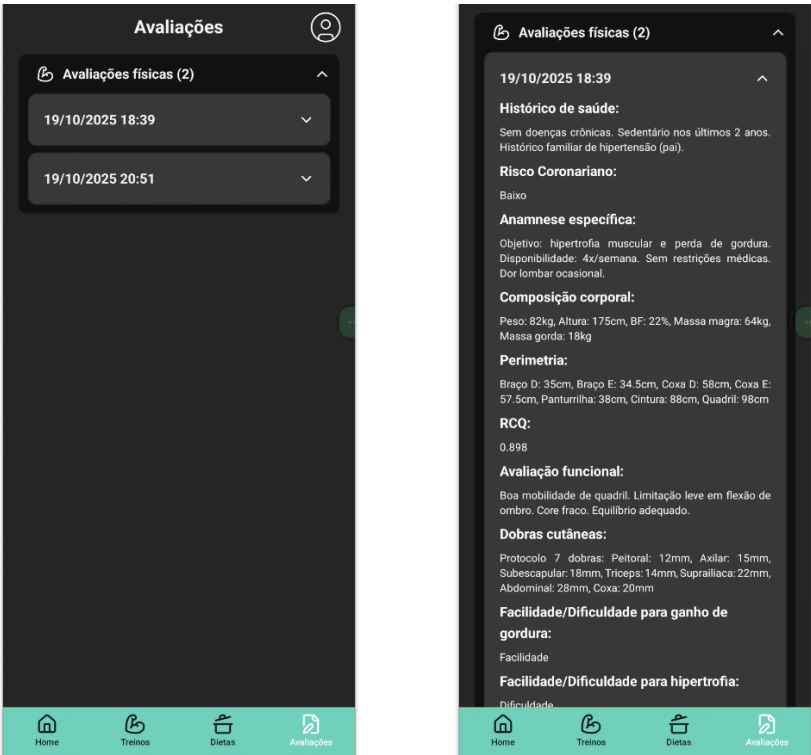
(b) Registro dos planos alimentares criados na tabela do módulo de Plano Alimentar

Fonte: Produzido pelos autores

5.5.4 Seção de avaliação

Ao entrar na seção de avaliações clicando na aba avaliações no inferior da tela o usuário é direcionado para a tela de listagem de avaliações (Figura 34a) nela é possível ver as avaliações criadas por ambos os colaboradores (*personal trainer* e nutricionista). Clicando na avaliação é possível ver os detalhes da avaliação (Figura 34b). Os registros no banco de dados com as avaliações exibidas são encontrados na Figura 35

Figura 34 – Telas da seção de avaliações

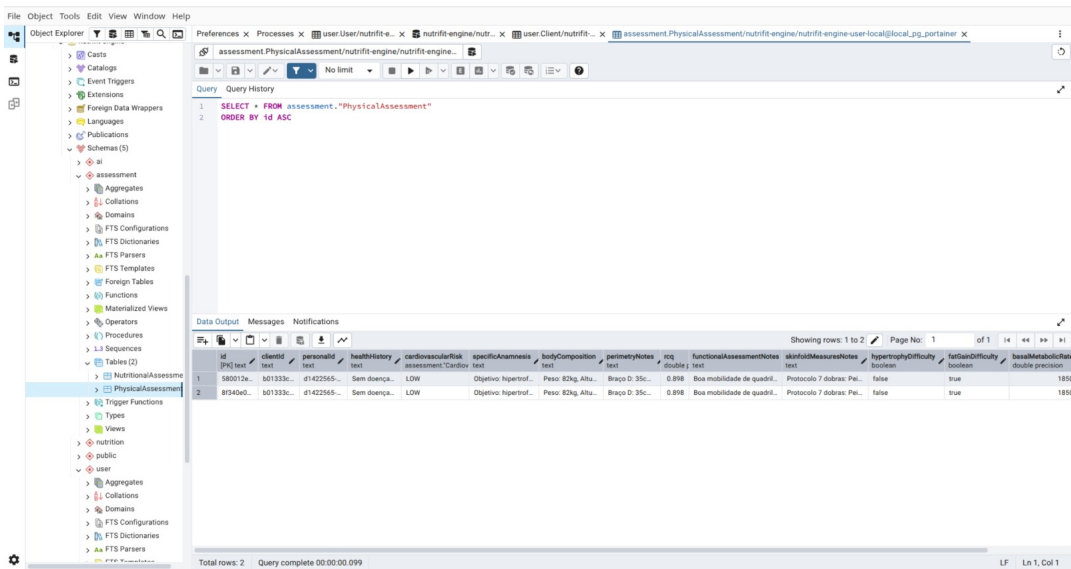


(a) Tela de listagem de avaliações

(b) Tela de listagem de avaliações mostrando os detalhes de uma avaliação

Fonte: Produzido pelos autores

Figura 35 – Registros no banco de dados das avaliações físicas



Fonte: Produzido pelos autores

O cliente só é capaz de visualizar as avaliações feitas para ele, a criação delas é reservada aos perfis dos colaboradores, ou seja, *personal trainer* ou nutricionista.

6 Considerações Finais

Nesta seção, serão discutidas as conclusões, contribuições e perspectivas de continuidade do projeto de formatura.

6.1 Conclusões do Projeto de Formatura

O desenvolvimento deste trabalho resultou na elaboração de um sistema colaborativo com arquitetura modular, segura e escalável, capaz de integrar usuários finais e profissionais especializados do segmento *fitness*. A solução fornece suporte aos processos de acompanhamento nutricional e físico, além de gerar recomendações personalizadas de treinos e planos alimentares utilizando Inteligência Artificial. O projeto atingiu seus objetivos principais ao demonstrar a viabilidade de empregar modelos generativos com técnicas de recuperação de contexto (RAG) em um cenário aplicado, bem como ao propor uma arquitetura de sistema que favorece a evolução, manutenção e extensibilidade.

As funcionalidades destinadas aos colaboradores (nutricionistas e *personal trainers*) não foram totalmente implementadas na interface do aplicativo, concluindo-se apenas o fluxo de cadastro desses usuários. Essa decisão foi tomada de forma estratégica, concentrando esforços na entrega de uma versão funcional da aplicação voltada ao cliente, contemplando as principais funcionalidades definidas no escopo do trabalho. Ainda assim, toda a estrutura de *backend* necessária para os demais casos de uso foi desenvolvida, permitindo que as telas restantes sejam implementadas de forma incremental em etapas futuras.

Do ponto de vista arquitetural, o projeto trouxe uma abordagem que combina padrões em diferentes níveis de abstração: em camadas, microsserviços e arquitetura limpa, evidenciando que essa abordagem fornece um ambiente flexível e robusto para sistemas que exigem modularidade e independência tecnológica. A aplicação desses padrões contribuiu diretamente para a organização do sistema, para a facilidade de manutenção e para a possibilidade de expansão do projeto sem impactar componentes já existentes.

Conclui-se que o trabalho desenvolvido não apenas atinge seus objetivos, mas também deixa uma base sólida para continuações futuras. A arquitetura concebida permite incorporar novos serviços, ampliar funcionalidades para profissionais especializados e integrar com outros modelos de IA. Dessa forma, o projeto evidencia o potencial de soluções computacionais escaláveis para fornecer o apoio necessário aos seus usuários.

6.2 Contribuições

O trabalho de formatura detalha um projeto que visa aproximar os campos da engenharia de *software* e Inteligência Artificial com a área da saúde, oferecendo uma proposta que pode apoiar tanto profissionais quanto usuários finais. Para isso, o grupo foi responsável por: fazer a especificação da solução, desde o levantamento de requisitos até a modelagem de uma arquitetura escalável; implementar o sistema, com o desenvolvimento dos microsserviços de *backend* e de uma primeira versão do aplicativo destinado aos clientes da plataforma, juntamente com o *deploy* dos microsserviços. Dessa forma, o trabalho contribui para o estudo de aplicações práticas de IA generativa e RAG. Ademais, reforça a importância do uso de boas práticas de arquitetura de sistemas e de *software*, além de propor um modelo replicável para sistemas que dependem de decisões automatizadas baseadas em IA.

Além disso, o projeto oferece uma solução que melhora a eficiência e reduz a carga operacional de nutricionistas e *personal trainers*, automatizando tarefas repetitivas e centralizando informações em uma única plataforma. Para os clientes, o aplicativo proporciona uma experiência mais acessível e unificada, permitindo acompanhar treinos, planos alimentares e avaliações, além de receber recomendações personalizadas rapidamente. Assim, o sistema contribui para otimizar o trabalho dos colaboradores e ampliar a autonomia dos usuários finais, demonstrando o impacto prático da Engenharia de Computação no desenvolvimento de soluções tecnológicas voltadas ao bem-estar e à saúde.

6.3 Perspectivas de Continuidade

O sistema possui grande potencial para evolução, com o desenvolvimento e a idealização de novas funcionalidades que tragam mais valor aos usuários. Dentre as possíveis melhorias no sistema, destacam-se:

1. **Treinamento de modelo específico para geração de treinos e planos alimentares:** Durante o trabalho, utilizou-se uma LLM treinada do Gemini, usando RAG para aumentar sua acurácia. Entretanto, como a modelagem do microsserviço de IA permite o rastreamento dos dados de entrada, a resposta gerada pela IA e a resposta ajustada pelo usuário, seria possível usar esses dados para treinar um modelo específico para a geração de recomendações de treinos e planos alimentares. Além disso, pode-se usar como fontes de dados treinos e planos alimentares criados previamente por *personal trainers* e nutricionistas. Para isso, pode-se realizar o *fine-tuning* de uma LLM ou explorar o treinamento de outros tipos de modelos, como os de aprendizado de máquina, por exemplo.
2. **Finalização da parte de colaboradores do aplicativo:** Como a maioria dos

endpoints necessários para o desenvolvimento das telas de colaboradores já estão implementados, é possível implementar as telas referentes aos nutricionistas e *personal trainers* do aplicativo.

3. **Atualizações regulatórias:** Acompanhar continuamente as evoluções nas legislações e normas aplicáveis, assegurando que o sistema permaneça em conformidade e mantenha a proteção e a privacidade dos usuários.

Referências

650 Industries, Inc. *Expo*. 2025. Disponível em: <<https://expo.dev>>. Acesso em: 27 set. 2025. Citado na página 57.

ALSHUQAYRAN, N.; ALI, N.; EVANS, R. A systematic mapping study in microservice architecture. In: *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*. [S.l.: s.n.], 2016. p. 44–51. Disponível em: <<https://doi.org/10.1109/MS.2016.64>>. Citado na página 17.

BALASUBRAMANIAM, S.; KADRY, Seifedine; RAJESH KUMAR, D.; ARUCHAMY, Prasanth. *Generative AI and LLMs: Natural Language Processing and Generative Adversarial Networks*. Berlin: Walter de Gruyter GmbH, 2024. 1–21 p. ISBN 9783111425078. Citado na página 15.

BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais*. 2018. Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 31 de agosto de 2023. Citado na página 21.

FERREIRA, C. S.; ANDRADE, F. B. d. Desigualdades socioeconômicas associadas ao excesso de peso e sedentarismo em adolescentes brasileiros. *Ciência & Saúde Coletiva, SciELO Public Health*, v. 26, p. 1095–1104, 2021. Citado na página 10.

Figma, Inc. *Figma*. 2025. Disponível em: <<https://www.figma.com>>. Acesso em: 28 set. 2025. Citado na página 56.

GAO, Y. et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, v. 2, 2023. Citado na página 15.

GeekyAnts. *gluestack-ui*. 2025. Disponível em: <<https://ui.gluestack.io>>. Acesso em: 27 set. 2025. Citado na página 57.

Google DeepMind. *Gemini Models*. 2025. <<https://gemini.google/overview>>. Acesso em: 27 set. 2025. Citado na página 58.

Instituto Brasileiro de Geografia e Estatística (IBGE). *Pesquisa Nacional de Saúde 2019: Informações sobre domicílios, acesso e utilização dos serviços de saúde: Brasil, grandes regiões e unidades da federação*. Rio de Janeiro: IBGE, 2020. Citado na página 10.

Instituto Brasileiro de Geografia e Estatística (IBGE). *Pesquisa Nacional de Saúde 2019: percepção do estado de saúde, estilos de vida, doenças crônicas e saúde bucal*. Rio de Janeiro: IBGE, 2020. Disponível em: <<https://biblioteca.ibge.gov.br/visualizacao/livros/liv101764.pdf>>. Citado na página 13.

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. *RFC 7519: JSON Web Token (JWT)*. 2015. IETF RFC. Acesso em: 28 de out. 2025. Disponível em: <<https://www.rfc-editor.org/rfc/rfc7519>>. Citado na página 22.

LangChain, Inc. *LangChain*. 2025. Disponível em: <<https://www.langchain.com>>. Acesso em: 27 set. 2025. Citado na página 58.

- MARTIN, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall, 2017. ISBN 9780134494166. Citado 2 vezes nas páginas 18 e 19.
- Meta Open Source. *React Native*. 2025. Disponível em: <<https://reactnative.dev/>>. Acesso em: 27 set. 2025. Citado na página 57.
- Microsoft. *TypeScript*. 2025. Disponível em: <<https://www.typescriptlang.org>>. Acesso em: 27 set. 2025. Citado na página 59.
- Ministério da Saúde. *Guia de Atividade Física para a População Brasileira*. Brasília, DF: Ministério da Saúde, 2021. Disponível em: <<https://www.gov.br/saude/pt-br/centrais-de-conteudo/publicacoes/guias-e-manuais/2021/guia-de-atividade-fisica-para-a-populacao-brasileira.pdf/view>>. Citado na página 13.
- MongoDB, Inc. *MongoDB*. 2025. Disponível em: <<https://www.mongodb.com>>. Acesso em: 27 set. 2025. Citado na página 60.
- NEGREIROS, A. et al. Quality assessment of smartphone fitness apps used to increase physical activity level and improve general health in adults: A systematic review. *Digital Health*, v. 8, 2022. Citado na página 11.
- OpenJS Foundation. *Fastify*. 2025. Disponível em: <<https://fastify.dev>>. Acesso em: 27 set. 2025. Citado na página 58.
- OpenJS Foundation. *Node.js*. 2025. Disponível em: <<https://nodejs.org>>. Acesso em: 27 set. 2025. Citado na página 58.
- OSTERWALDER, A.; PIGNEUR, Y. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Hoboken, NJ: John Wiley & Sons, 2010. ISBN 9780470876411. Citado na página 24.
- PESCATELLO, L.; MEDICINE, A. C. of S. *ACSM's Guidelines for Exercise Testing and Prescription*. Wolters Kluwer Health, 2014. (American college of sports medicine). ISBN 9781609136055. Disponível em: <<https://books.google.com.br/books?id=TtiCAwAAQBAJ>>. Citado na página 13.
- PostgreSQL Global Development Group. *PostgreSQL*. 2025. Disponível em: <<https://www.postgresql.org>>. Acesso em: 27 set. 2025. Citado na página 60.
- PRADO, R. et al. Aplicativos de smartphone para monitorar a atividade física e o comportamento sedentário: Uma revisão sistemática de estudos de validação. *Revista Brasileira de Atividade Física & Saúde*, v. 27, 2022. Citado na página 11.
- PROVOS, N. *Bcrypt Algorithm*. [S.l.], 1999. Acesso em: 28 de out. 2025. Disponível em: <https://www.usenix.org/legacy/events/usenix99/provos/provos_html/node5.html>. Citado na página 23.
- REINER, M. et al. Long-term health benefits of physical activity a systematic review of longitudinal studies. *BMC Public Health*, v. 13, n. 1, p. 813, 2013. ISSN 1471-2458. Disponível em: <<https://doi.org/10.1186/1471-2458-13-813>>. Citado na página 13.

RICHARDS, M. *Software Architecture Patterns*. Sebastopol, CA: O'Reilly Media, Inc., 2015. V. 4. Citado 2 vezes nas páginas 16 e 17.

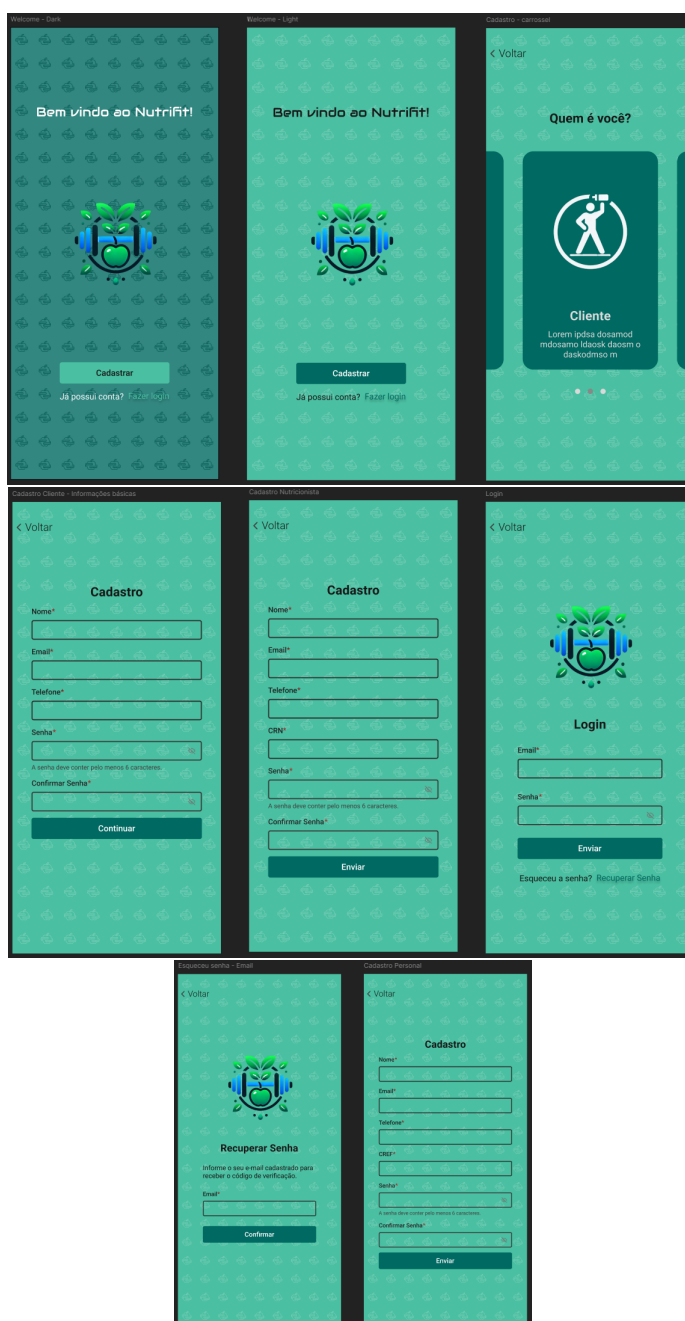
SmartBear Software. *Swagger*. 2025. Disponível em: <<https://swagger.io>>. Acesso em: 28 set. 2025. Citado na página 58.

ZHANG, M. et al. Efficacy of mobile health applications to improve physical activity and sedentary behavior: A systematic review and meta-analysis for physically inactive individuals. *International Journal of Environmental Research and Public Health*, v. 19, n. 8, p. 4905, 2022. Citado na página 11.

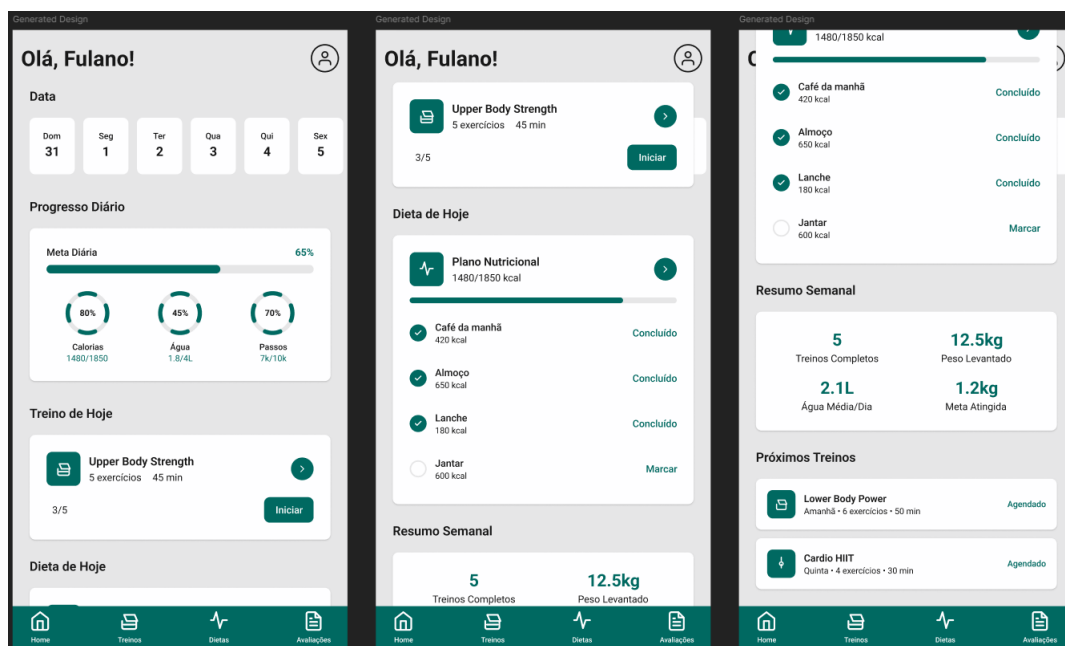
Apêndices

APÊNDICE A – Prototipação das telas

Figura 36 – Telas Iniciais

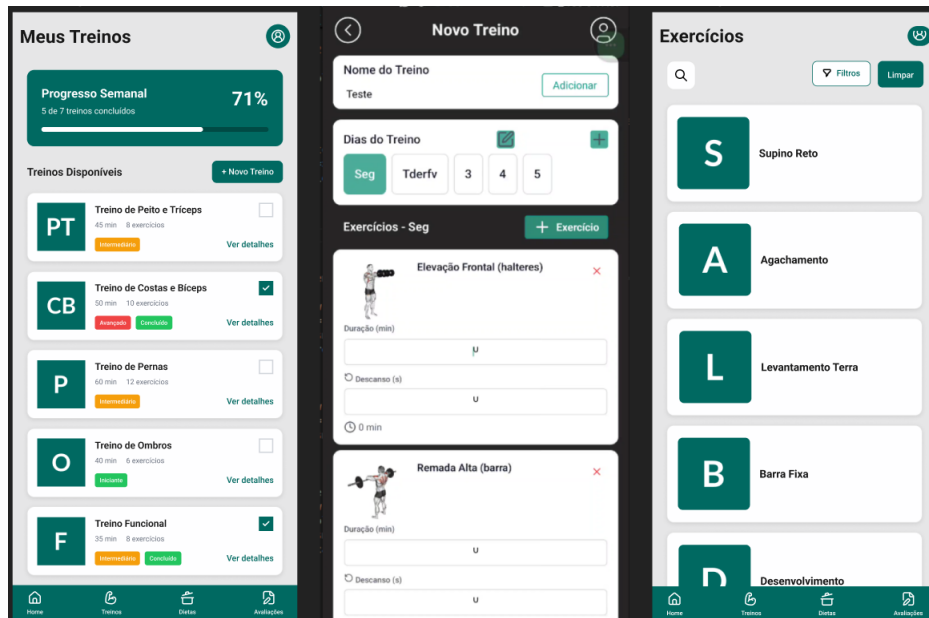


Fonte: Produzido pelos autores.

Figura 37 – Tela de *Home*

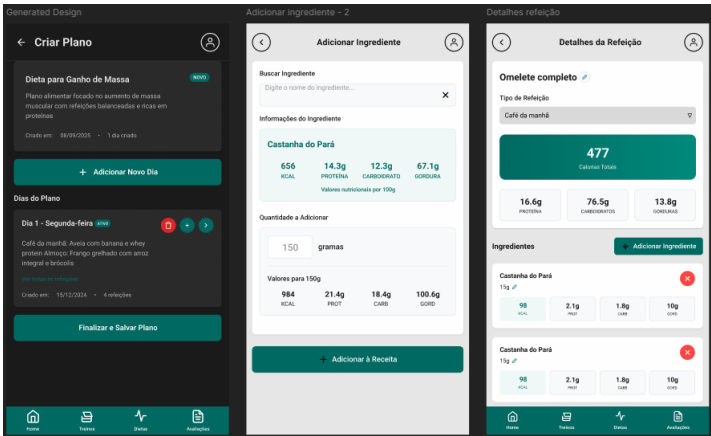
Fonte: Produzido pelos autores.

Figura 38 – Telas de treino



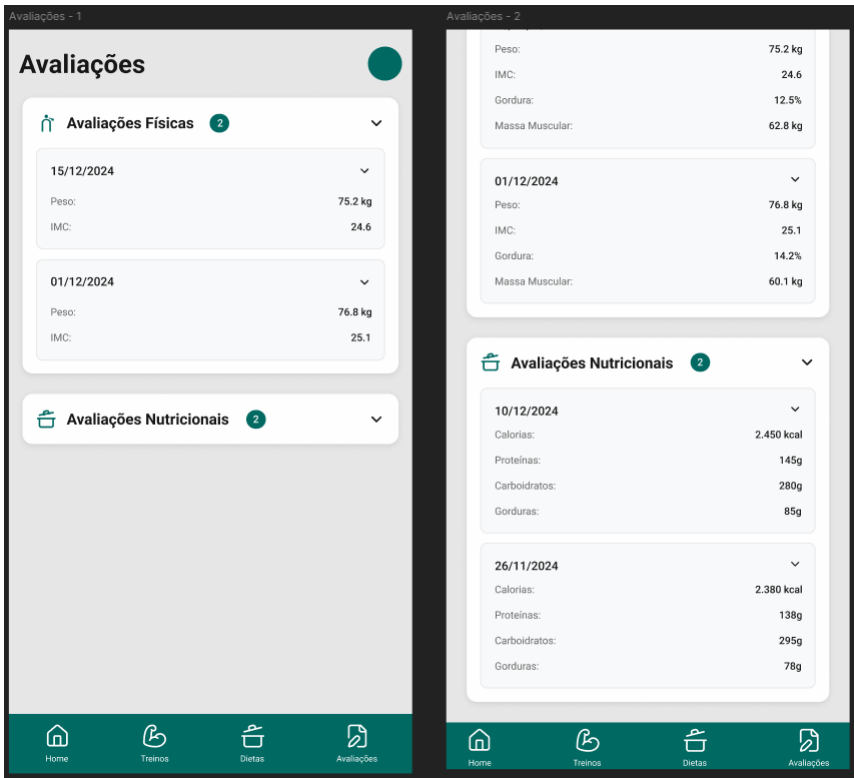
Fonte: Produzido pelos autores.

Figura 39 – Telas de nutrição



Fonte: Produzido pelos autores.

Figura 40 – Tela de avaliação



Fonte: Produzido pelos autores.